



Resumen

Este Trabajo de Fin de Grado documenta el diseño, desarrollo e implementación de ChessClassroom, una plataforma web educativa orientada a la enseñanza y gestión del ajedrez.

Actualmente, el entorno digital del ajedrez está dominado por plataformas centradas en el jugador individual sin docente, esto obliga a los profesores a utilizar varias herramientas para organizar una clase como videollamadas, almacenamiento y enseñanza de ajedrez, formando un sistema muy descentralizado. Este proyecto se basa en resolver este problema, unificando todas las necesidades en una única plataforma web.

A nivel técnico, el sistema se ha construido basándose en una arquitectura cliente-servidor con tecnologías web. El *frontend* se ha desarrollado utilizando Next.js, React y TypeScript, mientras que el *backend* utiliza Node.js y Express, integrando también Supabase como base de datos, autenticación y algunos servicios en tiempo real. La parte central del proyecto es la alta interactividad entre docente y alumnado, lograda mediante la integración de WebSockets para la sincronización de tableros en tiempo real, WebRTC (LiveKit) para la comunicación por voz y WebAssembly para ejecutar el motor de análisis Stockfish directamente en el navegador y desviar la carga en el servidor. Además, se ha diseñado un algoritmo para el procesamiento y validación de archivos PGN mediante árboles de variantes.

El proyecto es una solución de *Software as a Service* (SaaS) funcional y escalable que incluye aulas virtuales interactivas, carga y organización de material educativo, seguimiento del progreso del alumno, corrección automática de ejercicios y organización de partidas y torneos.

En conclusión, ChessClassroom cubre un vacío tecnológico existente en el sector de la docencia de ajedrez digital, ofreciendo a academias, clubs y profesores una herramienta sencilla e intuitiva que mejora el proceso educativo online.

Palabras clave: Ajedrez, Educación digital, Educación personalizada, Aplicación web, SaaS, Tiempo real, Next.js, Node.js, WebSockets, Diseño Software y Desarrollo Software.

Resum

Aquest Treball de Fi de Grau documenta el disseny, desenvolupament i implementació de ChessClassroom, una plataforma web educativa orientada a l'ensenyament i la gestió del escacs.

Actualment, l'entorn digital dels escacs està dominat per plataformes centrades en el jugador individual sense docent, cosa que obliga els professors a utilitzar diverses eines per a organitzar una classe, com ara videotrucades, emmagatzematge i ensenyament d'escacs, formant un sistema molt descentralitzat. Aquest projecte es basa a resoldre aquest problema, unificant totes les necessitats en una única plataforma web.

A nivell tècnic, el sistema s'ha construït basant-se en una arquitectura client-servidor amb tecnologies web. El *frontend* s'ha desenvolupat utilitzant Next.js, React i TypeScript, mentre que el *backend* utilitza Node.js i Express, integrant també Supabase com a base de dades, l'autenticació i alguns serveis en temps real. La part central del projecte és l'alta interactivitat entre el docent i l'alumnat, aconseguida mitjançant la integració de WebSockets per a la sincronització de taulers en temps real, WebRTC (LiveKit) per a la comunicació per veu i WebAssembly per a executar el motor d'anàlisi Stockfish directament al navegador i descarregar el servidor. A més, s'ha dissenyat un algorisme per al processament i la validació d'arxius PGN mitjançant arbres de variants.

El projecte és una solució de Software as a Service (SaaS) funcional i escalable que inclou aules virtuals interactives, càrrega i organització de material educatiu, seguiment del progrés de l'alumne, correcció automàtica d'exercicis i organització de partides i tornejos.

En conclusió, ChessClassroom cobreix un buit tecnològic existent en el sector de la docència d'escacs digital, oferint a acadèmies, clubs i professors una eina senzilla i intuïtiva que millora el procés educatiu en línia.

Paraules clau: Escacs, Educació digital, Educació personalitzada, Aplicació web, SaaS, Temps real, Next.js, Node.js, WebSockets, Disseny de programari i Desenvolupament de programari.

Abstract

This Bachelor's Thesis documents the design, development and implementation of ChessClassroom, an educational web platform aimed at chess teaching and management.

Currently, the digital chess environment is dominated by platforms focused on the individual player without a teacher, which forces instructors to use multiple tools to organise a class, such as video calls, file storage and chess teaching platforms, resulting in a highly decentralised system. This project addresses this problem by unifying all these needs into a single web platform.

From a technical standpoint, the system has been built on a client-server architecture using web technologies. The *frontend* was developed using Next.js, React and TypeScript, while the *backend* uses Node.js and Express, also integrating Supabase for the database, authentication and some real-time services. The core of the project is the high level of interactivity between teacher and students, achieved through the integration of WebSockets for real-time board synchronisation, WebRTC (LiveKit) for voice communication and WebAssembly to run the Stockfish analysis engine directly in the browser, offloading the computational burden from the server. In addition, an algorithm has been designed for the processing and validation of PGN files using variant trees.

The project is a functional and scalable Software as a Service (SaaS) solution that includes interactive virtual classrooms, loading and organisation of educational material, student progress tracking, automatic exercise correction and management of games and tournaments.

In conclusion, ChessClassroom fills an existing technological gap in the digital chess teaching sector, offering academies, clubs and teachers a simple and intuitive tool that enhances the online educational process.

Keywords: Chess, Digital education, Personalised education, Web application, SaaS, Real-time, Next.js, Node.js, WebSockets, Software design and Software development.

Índice de contenidos

1.	Introducción.....	11
1.1.	Motivación	11
1.2.	Objetivos	12
1.3.	Precedentes del proyecto	13
1.4.	Trabajo colaborativo	13
1.5.	Estructura del documento.....	14
2.	Estado del arte.....	16
2.1.	Plataformas de ajedrez online	16
2.2.	Plataformas de ajedrez dedicados al aprendizaje.....	16
2.3.	Herramientas de gestión educativo	17
2.4.	Conclusiones del análisis	18
3.	Plataforma propuesta.....	22
3.1.	Descripción de la plataforma	22
3.2.	División del trabajo	25
4.	Tecnologías utilizadas	26
4.1.	Tecnologías del <i>frontend</i>	26
4.2.	Tecnologías del backend.....	26
4.3.	Base de datos y servicios cloud	27
4.4.	Comunicación en tiempo real	27
4.5.	Lógica de ajedrez y análisis.....	27
4.6.	Integración de Chatbot	28
4.7.	Herramientas de pruebas	28
4.8.	Herramientas de desarrollo y despliegue.....	29
5.	Proceso software	30
5.1.	Metodología de desarrollo	30
5.1.1.	Metodología aplicada	30
5.1.2.	Roles del equipo	30
5.1.3.	Gestión del <i>Sprint</i> y desarrollo colaborativo.....	31
5.2.	Plan de trabajo y fases	33
5.2.1.	Fase 1: Análisis, especificación y diseño previo.....	33
5.2.2.	Fase 2: Desarrollo iterativo del MVP.....	34

5.2.3.	Fase 3: Pruebas, estabilización y validación con usuarios	34
5.3.	Control de versiones.....	37
6.	Especificación de requisitos	39
6.1.	Identificación de actores.....	39
6.2.	Técnicas de elicitación de requisitos	41
6.3.	Requisitos funcionales.....	42
6.4.	Requisitos no funcionales.....	45
6.5.	Casos de uso principales	46
6.6.	Pruebas de aceptación.....	51
7.	Modelado conceptual	53
7.1.	Modelo de dominio	53
7.2.	Modelo entidad-relación de la base de datos	54
7.3.	Modelo del árbol de movimientos	57
8.	Arquitectura y diseño	59
8.1.	Arquitectura general del sistema	59
8.2.	Arquitectura del frontend	60
8.3.	Arquitectura del backend.....	63
8.4.	Diseño del sistema de tiempo real.....	65
8.5.	Diseño de la interfaz de usuario	67
8.6.	Patrones de diseño aplicados.....	72
9.	Implementación	74
9.1.	Supabase	74
9.1.1.	Base de datos	74
9.1.2	Seguridad.....	74
9.1.3.	Manejo de archivos.....	75
9.1.4	Canales y Realtime	75
9.2	Página inicial	75
9.3	Módulo de Autenticación	76
9.4	Módulo de Gestión de clases	76
9.5	Módulo de Chat.....	77
9.6.	Módulo de Ajedrez	77
9.7.	Módulo de Planilla/Historial. Gestión de la partida: árbol de variantes	78
9.8	Módulo de Estudios.....	80
9.9.	Módulo de Ejercicios	81
9.10.	Módulo de Aula virtual en tiempo real.....	81
9.11.	Módulo de Partidas y Torneos.....	82

9.12.	Integración de Ejercicios y Torneos al módulo de Calendario.....	83
9.13.	Integración de Stockfish en Aula virtual y Partidas	83
9.14.	Despliegue	84
10.	Pruebas	85
10.1.	Estrategia de pruebas	85
10.2.	Pruebas funcionales de modulo	86
10.3.	Pruebas del árbol de variantes	88
10.4.	Pruebas de rendimiento	90
10.5.	Pruebas del sistema en tiempo real.....	90
10.6.	Pruebas de validación con usuarios	92
11.	Conclusiones y futuras líneas de trabajo	96
11.1.	Alcance y limitaciones del proyecto	96
11.2.	Cumplimiento de los objetivos	96
11.3.	Conclusiones.....	97
11.4.	Aportaciones personales	98
11.5.	Futuras líneas de trabajo	99
	Referencias bibliográficas	101
	Anexos	105
	Anexo A. Glosario	105
	Anexo B. Pseudocódigo de Proceso de construcción del árbol	107
	Anexo C. Pseudocódigo de Aplicar movimiento en el tablero.....	107
	Anexo D. Pseudocódigo de Serialización del árbol a PGN:	108
	Anexo E. Pseudocódigo de Generación de filas para la planilla	108
	Anexo F. Encuesta de satisfacción.....	108
	Anexo G. Endpoints de la API REST.....	110
	Anexo H. Imágenes de la aplicación	112
	Anexo I. Relación con los objetivos de desarrollo sostenible.....	120

Índice de figuras

Figura 1: Columnas de Backlog y Sprints	32
Figura 2: Columnas de fases de desarrollo de una UT	33
Figura 3: Diagrama de Gantt de ChessClassroom	36
Figura 4: Resultado del merge de ramas de trabajo	38
Figura 5: Diagrama de actores	40
Figura 6: Diagrama de contexto del sistema	40
Figura 7: Diagrama de características	43
Figura 8: Diagrama de casos de uso por característica	47
Figura 9: Diagrama de casos de uso por actor	48
Figura 10: Modelo de dominio de ChessClassroom	53
Figura 11: Modelo de entidad-relación de la base de datos de ChessClassroom	55
Figura 12: Modelo del árbol de movimientos	58
Figura 13: Arquitectura general del sistema	59
Figura 14: Arquitectura del frontend detallada	61
Figura 15: Arquitectura del backend detallada	64
Figura 16: Diseño del sistema en tiempo real	66
Figura 17: Wireframe página de inicio	68
Figura 18: Wireframe selección de listas de clases	68
Figura 19: Wireframe lista de clases grupales	69
Figura 20: Wireframe menú de navegación de clase	69
Figura 21: Wireframe del aula virtual	70
Figura 22: Wireframe explorador de archivos	70
Figura 23: Wireframe de ejercicios para el alumno	71
Figura 24: Wireframe de torneo	71
Figura 25: Ejemplo PGN	79
Figura 26: Distribución de respuestas para la valoración general	93
Figura 27: Distribución de respuestas para las funcionalidades	94
Figura 28: Distribución de respuestas para la usabilidad	95
Figura 29: Lista de endpoints parte 1	110
Figura 30: Lista de endpoints parte 2	111
Figura 31: Pantalla de inicio	112
Figura 32: Selección de clases	112
Figura 33: Listado de clases y creación de clase	113
Figura 34: Invitación para unirse a clase	113
Figura 35: Menú de clase	114
Figura 36: Aula virtual	114
Figura 37: Formulario de carga de posiciones en el tablero del aula	115
Figura 38: Formulario de guardado de la posición del aula	115
Figura 39: Explorador de archivos	116
Figura 40: Formulario de subida de ejercicios	116
Figura 41: Ejercicio evaluado	117
Figura 42: Formulario de creación de partida	117
Figura 43: Widget de invitación a partida amistosa	118
Figura 44: Partida	118

Figura 45: Formulario de creación de torneo	118
Figura 46: Torneo	119
Figura 47: Widget de notificación de emparejamiento de torneo	119

Índice de tablas

Tabla 1: Comparativa de funcionalidades entre las principales plataformas analizadas y ChessClassroom.....	20
Tabla 2: Ejemplos de requisitos funcionales de ChessClassroom.....	44
Tabla 3: Ejemplos de requisitos no funcionales de ChessClassroom.....	46
Tabla 4: Pruebas de aceptación para CU02 - Resolver tarea de ajedrez.....	51
Tabla 5: Pruebas del módulo de ejercicios	86
Tabla 6: Pruebas del árbol de variantes	88
Tabla 7: Resultados de pruebas con Lighthouse	90
Tabla 8: Pruebas del sistema en tiempo real.....	91
Tabla 9: Relación con los ODS	120

LISTADO DE ABREVIATURAS

API: Application Programming Interface (Interfaz de Programación de Aplicaciones).

BD: Base de datos

BI: Bussiness Intelligence (Lógica de negocio)

CI/CD: Continuous Integration / Continuous Deployment (Integración Continua / Despliegue Continuo).

CORS: Cross-Origin Resource Sharing (Intercambio de Recursos de Origen Cruzado).

CRUD: Create, Read, Update, Delete (Crear, Leer, Actualizar, Borrar).

CSS: Cascading Style Sheets (Hojas de Estilo en Cascada).

DOM: Document Object Model (Modelo de Objetos del Documento).

FEN: Forsyth-Edwards Notation (Notación Forsyth-Edwards para posiciones de ajedrez).

GUI: Graphical user interface (interfaz gráfica de usuario)

HTML: HyperText Markup Language (Lenguaje de Marcado de Hipertexto).

JSON: JavaScript Object Notation (Notación de Objetos de JavaScript).

JWT: JSON Web Token (Token Web JSON).

MVC: Model-View-Controller (Modelo-Vista-Controlador).

MVP: Minimum Viable Product (Producto mínimo viable)

PGN: Portable Game Notation (Notación Portátil de Partida de ajedrez).

REST: Representational State Transfer (Transferencia de Estado Representacional).

RLS: Row Level Security (Seguridad a Nivel de Fila).

RPC: Remote Procedure Call (Llamada a Procedimiento Remoto).

SaaS: Software as a Service (Software como Servicio).

SAN: Standard Algebraic Notation (Notación Algebraica Estándar).

SEO: Search Engine Optimization (Optimización para motores de búsqueda)

SFU: Selective Forwarding Unit (Unidad de Reenvío Selectivo).

SSR: Server-Side Rendering (Renderizado del Lado del Servidor).

TFG: Trabajo de Fin de Grado.

UCI: Universal Chess Interface (Interfaz Universal de Ajedrez).

UI: User Interface (Interfaz de Usuario).

URL: Uniform Resource Locator (Localizador Uniforme de Recursos).

UUID: Universally Unique Identifier (Identificador Único Universal).

UX: User Experience (Experiencia de Usuario).

WASM: WebAssembly.

WebRTC: Web Real-Time Communication (Comunicación Web en Tiempo Real).

1. Introducción

El ajedrez es uno de los juegos más antiguos del mundo. Nacido en el siglo VI en India, se extendió al resto del mundo a lo largo de los siglos (Murray, 1913) y con el paso del tiempo evolucionó tanto en sus reglas como en su estatus cultural, pasando de ser un pasatiempo de las élites a convertirse en un deporte reconocido a nivel internacional con millones de jugadores.

La relación entre el ajedrez y la informática es especialmente estrecha y se remonta a los primeros años de la computación moderna. Alan Turing desarrolló en 1950 uno de los primeros algoritmos capaces de jugar ajedrez, lo que se convertiría en uno de los principales campos de investigación dentro de la inteligencia artificial (Turing, 1950). Este interés culminó décadas después con hitos como la victoria de Deep Blue sobre Garry Kasparov en 1997 (Campbell, 2002) y más recientemente con AlphaZero, que demostró que un sistema basado en aprendizaje por refuerzo podía superar a los mejores motores tradicionales (Silver, 2018), abriendo nuevas líneas de investigación en inteligencia artificial aplicada al juego.

La digitalización del ajedrez ha transformado también su dimensión social y educativa. Según datos de la FIDE, el número de jugadores federados ha crecido de forma constante en los últimos años (FIDE, 2023), tendencia impulsada en parte por la popularización del ajedrez online durante la pandemia de COVID-19 (Statista, 2023). Este crecimiento, unido al avance tecnológico en torno al juego, pone de manifiesto la necesidad de herramientas digitales diseñadas específicamente para la enseñanza del ajedrez que vayan más allá de las plataformas orientadas al jugador individual.

1.1. Motivación

El ajedrez tiene una larga tradición educativa y su práctica ha demostrado tener beneficios en el desarrollo del pensamiento lógico, la concentración y la capacidad de planificación (Sala, 2016) (Trinchero, 2013). Durante los últimos años, su enseñanza ha experimentado un crecimiento notable tanto en entornos escolares como en academias, clubs y formatos online, impulsado en parte por la popularización del juego a través de plataformas digitales y contenido audiovisual (FIDE, 2023) (Statista, 2023).

Sin embargo, este crecimiento no ha venido acompañado del surgimiento de herramientas digitales diseñadas específicamente para el docente de ajedrez. Las plataformas existentes están orientadas al jugador individual y, aunque ofrecen funcionalidades de juego y análisis muy completas, carecen de las herramientas necesarias para gestionar clases, asignar ejercicios, hacer seguimiento del progreso del alumno o impartir lecciones online de forma interactiva. Esto obliga a los profesores a combinar varias aplicaciones distintas para cubrir sus necesidades docentes, generando un sistema descentralizado y poco eficiente. El análisis detallado de estas carencias se desarrolla en el capítulo 2.

Esta diferencia entre lo que existe y lo que un profesor de ajedrez realmente necesita fue la motivación principal para desarrollar ChessClassroom, una plataforma que integra en un único entorno las herramientas de enseñanza del ajedrez con las capacidades de gestión y seguimiento propias de una plataforma educativa moderna.

1.2. Objetivos

El objetivo principal de ChessClassroom es desarrollar una plataforma web educativa que unifique en un solo entorno las herramientas necesarias para la enseñanza y gestión del ajedrez, cubriendo tanto las necesidades docentes del profesor como las necesidades de aprendizaje del alumno.

Para lograr este objetivo principal, se definen los siguientes objetivos específicos:

- Analizar los procesos y necesidades existentes en la enseñanza del ajedrez con el objetivo de identificar requisitos funcionales y no funcionales.
- Diseñar e implementar una aplicación web con una arquitectura cliente-servidor desacoplada, escalable, mantenible y expandible a otras plataformas en el futuro.
- Desarrollar y ofrecer al profesor las herramientas para la gestión de su actividad docente, incluyendo la organización de clases y alumnos, la creación y asignación de material educativo y el seguimiento del progreso de sus alumnos.
- Implementar funcionalidades de comunicación y enseñanza en tiempo real que permitan al profesor dirigir clases online de forma interactiva.
- Ofrecer herramientas específicas del sector del ajedrez, como la visualización de partidas, análisis, resolución de ejercicios y la creación de competiciones.
- Ofrecer una experiencia de usuario intuitiva y accesible para perfiles no técnicos, poniendo especial atención en la usabilidad.
- Validar el sistema con pruebas funcionales y usuarios reales del público objetivo y no objetivo de la plataforma.

1.3. Precedentes del proyecto

El ajedrez ha sido tema de numerosos trabajos académicos a lo largo de los años. Sin ir más lejos, en la propia Universitat Politècnica de València existen más de diez Trabajos de Fin de Grado relacionados con el ajedrez únicamente dentro del Grado en Ingeniería Informática, por ejemplo, los siguientes:

- Vanaclocha Llorens, J. L. (1999). Implementación de un juego de ajedrez con capacidad de aprendizaje basado en técnicas de identificación de patrones. Universitat Politècnica de València.
- Tarazona Franco, J. E. (2001). Desarrollo de un motor de ajedrez. Universitat Politècnica de València.
- Navarro López, R. (2006). Programación en Java de un gestor de torneos de ajedrez. Universitat Politècnica de València.
- Bernal Martí, H. (2025). Adaptación de modelos de aprendizaje profundo para ajedrez. Universitat Politècnica de València.
- Jiménez Rivera, D. (2026). Sistema web de inventario y préstamo bibliotecario de la Federación de Ajedrez de la Comunidad Valenciana. Universitat Politècnica de València.

Como puede observarse, la mayoría de estos trabajos se centran en el desarrollar herramientas de análisis, motores de juego o aplicaciones de gestión orientadas a aspectos muy concretos del ajedrez. ChessClassroom se diferencia de estos precedentes al tratar el problema desde una perspectiva del docente, donde se plantea una plataforma web de uso diario que integra en un único entorno funcionalidades de gestión, enseñanza y práctica del ajedrez dirigidas al profesor y a sus alumnos.

1.4. Trabajo colaborativo

Este TFG ha sido desarrollado de forma colaborativa por dos estudiantes del Grado en Ingeniería Informática. Concretamente, el autor de esta memoria y Alejandro Gil López, quien además de ser ingeniero informático, es jugador de ajedrez con experiencia en la enseñanza del juego. Su conocimiento del ámbito resultó ser especialmente útil durante las fases de análisis y diseño, ya que pudo identificar rápidamente las necesidades de un docente de ajedrez y tomar las decisiones del proyecto desde una perspectiva de un usuario experto.

La división del trabajo se organizó de forma modular, asignando a cada miembro un conjunto de módulos y funcionalidades sobre los que cada uno tenía la responsabilidad de completar. Sin embargo, no quita el hecho de la coordinación entre ambos a lo largo de todo el proyecto. La división explícita del trabajo se desarrolla en el capítulo 3.

1.5. Estructura del documento

Para facilitar la lectura y comprensión de este TFG, la memoria se ha estructurado de forma secuencial por las fases de un desarrollo software. A continuación, se detalla el contenido de cada uno de los capítulos que forman este documento:

Capítulo 1: presenta la introducción, donde se contextualiza el proyecto, se expresa la motivación, se definen los objetivos y se revela la colaboración en el desarrollo del TFG.

Capítulo 2: trata el estado del arte, explorando el estado actual del ajedrez y presentando un análisis de las plataformas de ajedrez online, plataformas emergentes de ajedrez dedicadas al aprendizaje y las herramientas de gestión educativa existentes que son frecuente utilizados en conjunto para organizar clases. El capítulo finaliza justificando la necesidad de crear una solución centralizada.

Capítulo 3: presenta la propuesta de la solución, detallando cada módulo y funcionalidad a implementar. Además, se expone la división de las tareas para cada miembro que forma el equipo.

Capítulo 4: lista las tecnologías utilizadas a lo largo de todo el proyecto. En específico, el *frontend*, el *backend*, la base de datos y los servicios *cloud*, la comunicación en tiempo real, la lógica de ajedrez y análisis, la integración de Chatbot y las herramientas de desarrollo y despliegue.

Capítulo 5: describe el proceso software adoptado para el desarrollo del proyecto. Explica la metodología de desarrollo utilizada, el plan de trabajo estructurado por fases y el manejo de control de versiones.

Capítulo 6: explica la especificación de requisitos del sistema. Se identifican los actores, se exponen las técnicas de elicitación aplicadas, se muestran algunos requisitos funcionales y no funcionales, se describen los casos de uso principales y se muestran algunas pruebas de aceptación.

Capítulo 7: detalla el modelado conceptual de la aplicación. Se presenta el modelo de dominio para mostrar las entidades principales, el modelo entidad-relación que define la estructura de la BD y el diseño conceptual del árbol de movimientos (pieza clave para la gestión del tablero de ajedrez).

Capítulo 8: expone la arquitectura y diseño del sistema. Se detalla la arquitectura global cliente-servidor, centrándose en la estructura del *frontend* y del *backend*, el diseño del sistema en tiempo real y la interfaz de usuario, además de presentar algunos patrones de diseño.

Capítulo 9: documenta la fase de implementación. Se describe la configuración de la BD, los datos más relevantes del desarrollo de cada módulo, las tareas adicionales y el despliegue a producción.

Capítulo 10: explica la fase de pruebas. Se muestra la estrategia de pruebas ejemplos de pruebas funcionales de módulo, pruebas del árbol de variantes, pruebas de rendimiento, pruebas del sistema de tiempo real y pruebas de validación con usuarios.

Capítulo 11: finaliza el TFG con las conclusiones. Expone el alcance final del proyecto, el cumplimiento de los objetivos, las conclusiones, las aportaciones personales y las futuras líneas de trabajo.

Adicionalmente, el documento incluye un apartado de **Anexos** que proporciona información complementaria, como el glosario, pseudocódigo, la encuesta de validación, la lista de *endpoints* de la API REST, imágenes de la aplicación y la relación con los objetivos de desarrollo sostenible.

2. Estado del arte

En este capítulo se analiza el estado actual de las herramientas digitales relacionadas con la enseñanza del ajedrez, con el objetivo de identificar las carencias existentes en el mercado y justificar la necesidad de una solución como ChessClassroom. Para ello, se examinan las principales plataformas de ajedrez online orientadas al juego competitivo, las plataformas dedicadas específicamente al aprendizaje estructurado y las herramientas de gestión educativa generalistas. El capítulo finaliza con un análisis comparativo de todas las soluciones estudiadas y una reflexión sobre el problema que ChessClassroom pretende resolver.

2.1. Plataformas de ajedrez online

La digitalización del ajedrez ha creado un ecosistema de plataformas online que han modificado la forma en la que los jugadores estudian, practican, compiten, etc. Aun así, para un profesor que necesita gestionar un grupo de alumnos que comienzan a aprender, ninguna de las opciones actuales ofrece una solución completa.

Chess.com: la plataforma de ajedrez online más grande del mundo (Statista, 2023). Es una web muy completa para el jugador individual. Ofrece partidas en distintos formatos y controles de tiempo, puzles, lecciones estructuradas por nivel de dificultad y análisis de partidas mediante Stockfish. El problema es que está diseñada para el jugador y no para docente. No ofrece una forma real de crear un grupo de alumnos, asignar ejercicios personalizados o ver el progreso de cada uno.

Lichess: la principal alternativa de código abierto. Tiene una comunidad muy activa y unas funcionalidades similares a Chess.com. Su módulo de estudios es mejor para preparar material educativo debido a que permite crear análisis con comentarios y compartirlos con otros usuarios. Aun así, tiene los mismos problemas desde el punto de vista docente. No hay gestión de clases, no hay sistema de tareas, no hay seguimiento del alumno, etc.

2.2. Plataformas de ajedrez dedicados al aprendizaje

Dentro del ecosistema de plataformas de ajedrez online existe un subconjunto orientado específicamente al aprendizaje estructurado, que se diferencia de Chess.com y Lichess por priorizar la formación sobre el juego competitivo. Sin embargo, como se verá a continuación, ninguna de ellas resuelve completamente las necesidades del docente.

ChessKid: desarrollada por Chess.com y dirigida a jugadores más jóvenes, es la plataforma que más puede parecerse a una clase online. Permite crear clubes y manejar a los alumnos dentro de ellos, esto lo hace bastante adecuado para ámbitos educativos. Aun así, sus herramientas de gestión son básicas y no incluye funcionalidades de tiempo real, corrección automática de ejercicios, etc.

Chessable: diferente al resto, se centra la enseñanza de aperturas mediante la repetición. Es una herramienta útil para ese propósito, pero su modelo de enseñanza son cursos pregrabados que el alumno sigue de forma individual, sin ningún tipo de interacción en tiempo real con un profesor, lo cual, a nuestro parecer, lo hace la más pobre de las opciones analizadas.

OnlyChessAcademy: es una academia de ajedrez online orientada a niños de entre cuatro y dieciocho años, que ofrece clases en grupos reducidos de máximo seis alumnos. Su propuesta se diferencia de las anteriores por centrarse en la figura del docente y en la enseñanza personalizada, con sesiones en directo, demostraciones interactivas y torneos mensuales de práctica. Sin embargo, su modelo de funcionamiento se apoya en herramientas externas y no tiene una plataforma propia integrada que centralice el tablero interactivo, los ejercicios, el material educativo y el seguimiento del alumno en un único entorno.

CircleChess: plataforma de aprendizaje de ajedrez que conecta a estudiantes con entrenadores titulados en sesiones grupales de tamaño reducido y clases particulares. Su propuesta se basa en tutorías directas por parte de Grandes Maestros, análisis asistido por su sistema de inteligencia artificial propio y un sistema de mentoría entre pares. Además, ofrece una aplicación móvil con herramientas de análisis, puzzles y seguimiento del progreso del alumno. Sin embargo, al igual que OnlyChessAcademy, su modelo docente se apoya en gran medida en herramientas externas de comunicación para las sesiones en directo, sin ofrecer una plataforma propia que integre el tablero interactivo sincronizado, la gestión de ejercicios y el seguimiento del alumno en un único entorno específico para el profesor.

2.3. Herramientas de gestión educativo

Debido a la falta de una plataforma adecuada para dar clases de ajedrez, muchos profesores deciden usar herramientas educativas generalistas. Merece la pena analizarlas porque así se muestra qué existe en el mercado y por qué ninguna termina de encajar para este caso en concreto.

Google Classroom: probablemente la más conocida, ofrece la posibilidad de crear clases, asignar tareas con fechas de entrega, compartir material y hacer un seguimiento básico del progreso de los alumnos. No obstante, para un profesor de ajedrez, es limitada. Puede usarla para enviar archivos y recoger entregas, pero no tiene soporte para el ámbito del ajedrez. Funcionalidades como cargar un PGN o interactuar con un tablero son inexistentes.

Moodle; es la plataforma de gestión del aprendizaje de código abierto utilizada en entornos universitarios. Ofrece un sistema muy completo de gestión de cursos, evaluación y seguimiento, pero su configuración y mantenimiento requiere conocimientos técnicos que son muy superiores a los de un profesor de ajedrez particular o de club. Además, al igual que Google Classroom, no tiene ninguna funcionalidad específica de ajedrez.

Microsoft Teams: aunque comenzó como una herramienta de comunicación para empresas, se ha extendido bastante en entornos educativos por su capacidad de videollamada, chat, creación de grupos y gestión de archivos. Muchos profesores de ajedrez la usan para dar clases compartiendo la pantalla mientras muestran posiciones en otra aplicación. Funciona, pero es está cogida con pinzas y le obliga al profesor a usar varias aplicaciones al mismo tiempo.

2.4. Conclusiones del análisis

A lo largo de este capítulo se ha analizado el estado actual de las herramientas relacionadas con la enseñanza del ajedrez, desde las plataformas de juego más populares hasta las herramientas educativas generalistas, pasando por las plataformas dedicadas específicamente al aprendizaje estructurado. Las conclusiones que se extraen de este análisis son claras.

Las plataformas de ajedrez online generalistas como Chess.com y Lichess ofrecen herramientas de juego y análisis muy completas, pero están diseñadas pensando en el jugador individual y no en el docente. No existe en ninguna de ellas un sistema real de gestión de clases, asignación de ejercicios, seguimiento del progreso del alumno u organización del material educativo.

Las plataformas dedicadas al aprendizaje como ChessKid, Chessable, OnlyChessAcademy y CircleChess se acercan más a las necesidades docentes, pero cada una de ellas cubre únicamente una parte del problema. ChessKid ofrece gestión básica de grupos, pero carece de herramientas de tiempo real. Chessable está orientada al estudio individual mediante cursos pregrabados sin interacción directa con un profesor. OnlyChessAcademy y CircleChess ofrecen clases en directo con entrenadores titulados, pero dependen de herramientas externas de comunicación y no disponen de una plataforma propia que integre el tablero sincronizado, los ejercicios y el seguimiento del alumno en un único entorno.

Por su parte, las herramientas educativas generalistas como Google Classroom, Moodle o Microsoft Teams cubren bien la parte de gestión docente, pero carecen completamente de soporte para el dominio específico del ajedrez, obligando al profesor a combinarlas con otras aplicaciones.

La Tabla 1 resume de forma visual las funcionalidades analizadas y el grado de cobertura que ofrece cada plataforma existente, incluyendo ChessClassroom como propuesta de solución ante las carencias identificadas.

Como puede observarse, ninguna de las plataformas existentes cubre el conjunto completo de funcionalidades que un profesor de ajedrez necesita en su actividad docente diaria. La mayoría de los profesores trabajan con una combinación de varias herramientas: WhatsApp o Telegram para comunicarse con los alumnos, Google Drive para compartir material, Lichess o Chess.com para los ejercicios y Zoom o Teams para las videollamadas. Esta fragmentación de herramientas aumenta la carga cognitiva tanto del docente como del alumno y reduce la eficacia del proceso de enseñanza. Centralizar todo en un único entorno no es solo una cuestión de comodidad, sino de coherencia pedagógica (Bower, 2015).

ChessClassroom surge precisamente para cubrir este hueco, integrando en un único entorno las herramientas de enseñanza propias del ajedrez con las capacidades de gestión y seguimiento propias de una plataforma educativa moderna. La propuesta de solución se detalla en el capítulo siguiente.



Tabla 1: Comparativa de funcionalidades entre las principales plataformas analizadas y ChessClassroom

Funcionalidad	Chess.com	Lichess	ChessKid	Chessable	OnlyChessAcademy	CircleChess	ChessClassroom
Motor de análisis integrado	✓	✓	X	X	✓	✓	✓
Torneos con emparejamiento automático	✓	✓	Parcial	X	✓	X	✓
Explorador de archivos PGN	X	✓	X	Parcial	✓	X	✓
Gestión de clases y alumnos	X	X	Parcial	X	✓	X	✓
Aula virtual con tablero sincronizado	X	X	X	X	X	X	✓
Comunicación por voz	X	✓	X	X	X	✓	✓

Asignación de ejercicios con corrección automática	X	X	X	X	✓	✓	✓
Seguimiento del progreso del alumno	Parcial	Parcial	Parcial	X	✓	✓	✓
Calendario de clase	X	X	X	X	X	X	✓
Chatbot de ayuda	X	X	X	Parcial	X	X	✓

3. Plataforma propuesta

En este capítulo se presenta la solución propuesta para resolver las carencias identificadas en el análisis del capítulo anterior. Se describe en detalle la plataforma ChessClassroom, sus módulos funcionales y las decisiones tomadas sobre el alcance del proyecto, así como la división del trabajo entre los dos miembros del equipo de desarrollo.

3.1. Descripción de la plataforma

A partir del análisis realizado en los capítulos anteriores, se propone el desarrollo de ChessClassroom, una plataforma web educativa orientada a la enseñanza del ajedrez que integra en un único entorno las herramientas necesarias para que un profesor pueda gestionar su actividad docente de forma completa.

La propuesta parte de una idea sencilla, un profesor de ajedrez necesita poder gestionar sus clases y alumnos, preparar y compartir material, dar lecciones online con un tablero interactivo, asignar y corregir ejercicios, organizar competiciones entre sus alumnos, asignar objetivos a corto y largo plazo y consultar la evolución de los alumnos. Todo ello sin tener que ir cambiando entre distintas aplicaciones.

ChessClassroom satisface estas necesidades ofreciendo un único punto de acceso desde el cual el profesor puede realizar todas estas tareas y el alumno puede acceder a todo el material y actividades de sus clases, ya sean particulares o grupales. Para detallar la solución, la plataforma se organiza en torno a varios módulos funcionales:

- **Módulo de autenticación:** permite a los usuarios registrarse, iniciar sesión y maneja la verificación de la cuenta para el resto de las funcionalidades.
- **Módulo de gestión de clases:** permite al profesor crear clases particulares o grupales y añadir alumnos mediante un sistema de invitación. Además, habrá sistemas para facilitar la organización de clases, por ejemplo, un listado automático con todas las clases en las que el usuario participa e indicadores de si estos están activos o no.
- **Módulo de ajedrez:** es el componente base sobre el que se monta muchos otros módulos del proyecto. Este abarca toda la lógica de un tablero de ajedrez funcional y reutilizable. Ofreciendo de esta forma un tablero de ajedrez con funcionalidades de

carga de posiciones mediante archivos o códigos FEN y PGN, además de facilitar las jugadas al usuario mediante el uso de resaltes de movimientos, avisos y *feedback* sonoro.

- **Módulo de planilla:** es un componente que va en conjunto con el módulo de ajedrez. En este se mostrará toda la información relacionada al histórico de posiciones del tablero de ajedrez. Registrará todas las jugadas de la partida y en ciertos módulos, habilitará la posibilidad de generar variantes o líneas de juego alternativas, permitiéndole al usuario regresar a un estado pasado del tablero e introducir nuevos movimientos. También tendrá funcionalidades de desplazarse por el histórico de una línea de juego mediante botones de volver al inicio, volver a la jugada anterior, ir a la siguiente jugada e ir a la última jugada.
- **Módulo de estudios:** incorpora un sistema de explorador de archivos con carpetas donde se le permite al profesor subir y organizar archivos PGN. Tanto los alumnos como el profesor podrán cargar el material educativo en un tablero interactivo con soporte para el motor Stockfish.
- **Módulo de ejercicios:** de forma similar al de estudios, incorpora un sistema de explorador de archivos con carpetas donde se le permite al profesor subir y organizar archivos PGN. Tras ello, puede grabar una secuencia de movimientos como solución junto a un comentario para cada ejercicio y establecer los plazos de inicio y entrega de los mismo. Una vez habilitado un ejercicio, los alumnos podrán intentar resolverlo, en cuyo proceso se realiza un seguimiento del tiempo acumulado para resolver el ejercicio, los fallos, y los movimientos, además de poder guardar un comentario para el profesor. Una vez se haya completado correctamente el ejercicio o se haya finalizado el plazo de entrega, los alumnos podrán consultar la solución y los profesores podrán evaluar la respuesta del alumno, puntuarlos y dejar un comentario que el alumno podrá ver.
- **Módulo de aula virtual:** integra todas las herramientas para dar una clase online en ChessClassroom. Pondrá a disposición un tablero de ajedrez sincronizado en tiempo real con acciones de carga y guardado de posiciones, comunicación por voz, chat de texto, y una lista de alumnos conectados con controles de moderación del tablero y voz para el profesor.
- **Módulo de partidas y torneos:** por una parte, le permitirá a profesores y alumnos esperar u organizar partidas amistosas en tiempo real, donde podrán invitar directamente a otro jugador (el cual recibirá una notificación para aceptar o rechazar la propuesta), establecer el tiempo, el incremento del tiempo tras realizar un movimiento y la posición inicial del tablero. Por otra parte, el profesor podrá organizar torneos en formato Arena seleccionando los participantes de la clase y estableciendo una fecha de inicio y fin. Además, el sistema de torneos contará con las mismas posibilidades de configuración del tiempo y el tablero inicial para todas las partidas que se disputen.

Finalmente, los torneos contarán de sistema de emparejamiento automático y una tabla de clasificación en tiempo real.

- **Módulo de estadísticas:** centraliza y muestra los datos de actividad del alumno recopilados mediante los demás módulos. A través de gráficas, tablas y métricas, tanto el alumno como el profesor pueden consultar información sobre el rendimiento en ejercicios, así como los resultados de partidas y torneos jugados. El módulo le permitirá al profesor acceder de forma centralizada a los datos de cada alumno de forma individual, haciendo que sea más cómodo y rápido encontrar aspectos tanto positivos como negativos en el aprendizaje de un alumno.
- **Módulo de objetivos:** ofrece un tablero desde el cual, un profesor puede organizar y hacer seguimiento de las metas de sus alumnos. El profesor puede crear tableros grupales visibles para toda la clase o tableros individuales específicos a un alumno. Dentro de cada uno, podrá definir objetivos con un plazo de tiempo determinado. A medida que revisa el progreso de sus alumnos, el profesor puede marcar cada objetivo como cumplido.
- **Módulo de calendario:** le permite al profesor crear eventos relevantes de cada clase, como los días de docencia online, las fechas de entrega de ejercicios y los torneos programados.
- **Módulo de material adicional:** implementará un explorador de archivos que permite al profesor subir, organizar y compartir recursos educativos relacionados con el ajedrez. Por ejemplo, podrá subir archivos de distintos tipos, como PDF, imágenes y vídeos. El módulo tendrá incorporado un visualizador para cada tipo de contenido, de forma que el alumno puede ver el material directamente desde la aplicación sin tener que descargarlo o utilizar otras herramientas.
- **Módulo de gestión de alumnos:** ofrece al profesor la posibilidad de administrar los miembros de su clase. Puede consultar el listado de alumnos matriculados, asignar un alias personalizado a cada uno y expulsar a un alumno en caso de ser necesario.
- **Módulo de Stockfish:** integra el motor de análisis de ajedrez a través de su versión compilada en WebAssembly, lo cual permite ejecutarlo directamente en el navegador del usuario sin necesidad de un servidor dedicado. Su implementación será como un componente independiente y reutilizable que puede añadirse a cualquier otro módulo de la plataforma, como el módulo de estudios, el aula virtual o la revisión de partidas finalizadas.
- **Módulo de chatbot:** ofrece un asistente conversacional que ofrece ayudas contextuales en forma de guías de las distintas funcionalidades mediante un sistema de conversación en lenguaje natural. De esta forma, se facilita el uso de la aplicación para aquellos usuarios no disponen del nivel técnico suficiente o situaciones en las que el alumno necesita ayuda, pero su profesor no está disponible en ese momento.

3.2. División del trabajo

Para el desarrollo de ChessClassroom, se ha dividido la implementación de los módulos y sus trabajos adicionales entre el autor y el colaborador del TFG. A continuación, se lista para cada miembro las tareas que realizaron de forma individual (tareas como el estudio del sector, la especificación de requisitos, diseño de interfaces y valoración con usuarios reales, se realizaron en conjunto).

El autor de esta memoria se ha encargado de:

- La infraestructura de la base de datos relacionados a sus módulos.
- La página inicial de la aplicación web.
- El módulo de autenticación.
- El módulo de gestión de clases.
- El módulo del chat.
- El módulo de ajedrez.
- El módulo de la planilla con el árbol de variantes.
- El módulo de estudios.
- El módulo de ejercicios.
- El módulo de aula virtual.
- El módulo de partidas y torneos.
- La sincronización automática de eventos de ejercicios y torneos en el calendario.
- La integración del componente de Stockfish en el aula virtual y las partidas.
- El despliegue de la aplicación en producción.
- Las pruebas relacionadas a cada funcionalidad

Por su parte, el colaborador del proyecto se ha encargado de:

- La infraestructura de la base de datos relacionados a sus módulos.
- El módulo de estadísticas
- El módulo de objetivos
- El módulo del calendario
- El módulo de material adicional
- El módulo de gestión de alumnos
- El módulo de chatbot
- La creación del componente base de Stockfish en WebAssembly para el módulo de Stockfish y su integración en el módulo de estudio.
- Las pruebas relacionadas a cada funcionalidad

4. Tecnologías utilizadas

En este capítulo se presentan las tecnologías y herramientas utilizadas a lo largo del proyecto, justificando su elección y describiendo el papel que cumple cada una dentro del sistema. Las tecnologías se organizan por categorías: *frontend*, *backend*, base de datos y servicios *cloud*, comunicación en tiempo real, lógica de ajedrez y análisis, integración del chatbot, herramientas de pruebas y herramientas de desarrollo y despliegue.

4.1. Tecnologías del *frontend*

React: librería de interfaz de usuario. Sirve para desarrollar interfaces usando componentes reutilizables que manejan su estado y se actualizan de forma automática cuando hay un cambio en los datos. Se eligió por ser un estándar en el desarrollo de aplicaciones web y por la gran cantidad de librerías que tiene. En el proyecto lo hemos utilizado para construir todos los componentes.

Next.js: *framework* de desarrollo web basado en React. Añade funcionalidades como el enrutamiento automático basado en las carpetas del proyecto, la generación estática de páginas, App Router (*layouts* anidados y gestión de componentes), TypeScript nativo, etc. Se eligió como la base del *frontend* porque es de los *frameworks* más populares y documentados en la industria. En el proyecto lo hemos utilizado para definir todas las rutas de la aplicación, gestionar los layouts de las clases y desplegar la aplicación desde Vercel.

TypeScript: variante de JavaScript con tipado estático. Ayuda a detectar errores en tiempo de compilación en vez de en ejecución. Se utilizó tanto en el frontend como en el backend para mantener la consistencia de los tipos para el código.

Tailwind CSS: *framework* de estilo basado en clases para construir interfaces en HTML sin escribir hojas de estilo aparte. Se eligió porque acelera el desarrollo y por su integración con Next.js. Se utilizó para todos los estilos de la aplicación.

4.2. Tecnologías del *backend*

Node.js: entorno de ejecución de JavaScript. Permite desarrollar servidores web de alto rendimiento, orientado sobretudo a operaciones de entrada y salida asíncronos y sin bloqueos, como consultas a bases de datos, peticiones a servicios externos, etc. Se eligió por su compatibilidad con TypeScript, gran cantidad de librerías y la posibilidad de compartir código y tipos con el *frontend* y el *backend* por usar el mismo lenguaje.

Express: *framework* para Node.js. Ayuda en la creación de APIs REST con un sistema de rutas y middlewares. Se eligió por su facilidad de uso, flexibilidad y por ser la opción más habitual para construir *backends* con Node.js. En el proyecto se utiliza para construir todos los *endpoints* de la API, manejar la autenticación con middlewares y estructurar el código en tres capas.

4.3. Base de datos y servicios cloud

Supabase: servicio de plataforma de *backend* que ofrece una base de datos PostgreSQL, un sistema de autenticación con soporte para JWT, almacenamiento de archivos y un servicio de comunicación en tiempo real. Se eligió Supabase por las funcionalidades mencionadas, para acelerar el proceso de desarrollo y disminuir la complejidad del proyecto. En el proyecto se utiliza como base de datos, como gestor de la autenticación de usuarios y como motor de sincronización en tiempo real a través de su servicio Realtime.

4.4. Comunicación en tiempo real

Supabase Realtime: servicio de sincronización en tiempo real de Supabase. Está basado en WebSockets. Ofrece tres sistemas para la comunicación: Broadcast (emitir eventos entre clientes sin persistirlos en la base de datos), Presence (gestionar la lista de usuarios conectados) y Postgres Changes (escuchar cambios en las tablas de la base de datos en tiempo real). Para ChessClassroom hemos utilizado principalmente el Broadcast para sincronizar el tablero del aula virtual entre todos los usuarios y para manejar la presencia de usuarios.

LiveKit: aunque actualmente es famosa por ofrecer agentes IA, es una plataforma de comunicación de audio y vídeo en tiempo real basado en WebRTC. Tiene una arquitectura SFU que hace de servidor central y es encargado de recibir y redistribuir los audios de cada participante. Permite aumentar el número de usuarios conectados a una sala de voz con menor consumo de ancho de banda que una arquitectura punto a punto. Se eligió LiveKit porque gestiona por sí solo el audio, está bien documentada y ofrece un plan gratuito para el MVP. En el proyecto lo hemos utilizado para la sala de voz del aula virtual.

4.5. Lógica de ajedrez y análisis

Chess.js: librería de JavaScript que implementa las reglas de ajedrez. Incluye la validación de movimientos, detección de jaque, jaque mate y tablas. También puede manejar la importación y exportación de partidas en formato PGN y FEN. Se eligió para acelerar el proceso de desarrollo y no tener que implementar desde cero la lógica del juego. En el

proyecto se ha utilizado como motor de validación de movimientos en el tablero interactivo, en el módulo de estudios, en el módulo de ejercicios y en las partidas en vivo.

React-chessboard: librería de React que ofrece un componente de tablero de ajedrez con funcionalidades para arrastrar piezas, personalizar estilos, controlar el estado del tablero, etc. Se eligió para acelerar el proceso de desarrollo y no tener que implementar desde cero los componentes visuales y su lógica para un tablero de ajedrez. Además, es compatible con chess.js. En el proyecto se ha utilizado como componente base del tablero interactivo en todos los módulos que muestren o interactúen con un tablero de ajedrez.

Stockfish: motor de análisis de ajedrez de código abierto más potente disponible en la actualidad. Se eligió Stockfish frente a otras alternativas como Leela Chess Zero por su precisión, fiabilidad, precio gratuito y la característica de WebAssembly, la cual puede ejecutar el modelo en el navegador del usuario sin tener hacer peticiones a un servidor. En el proyecto se utiliza para el análisis de posiciones en el módulo de estudios, aula virtual y revisión de partidas finalizadas.

4.6. Integración de Chatbot

Grok: modelo de lenguaje. Ofrece una API que permite integrar el procesamiento de lenguaje natural en aplicaciones externas. Se eligió Grok frente a otras alternativas por su API fácil de implementar y su plan de uso gratuito para este TFG. En el proyecto se ha utilizado para alimentar el módulo de chatbot, procesar las consultas del usuario en lenguaje natural y generar respuestas contextuales sobre las funcionalidades de la plataforma.

4.7. Herramientas de pruebas

Jest: *framework* de pruebas para JavaScript. Está diseñado para funcionar sin necesidad de configuración adicional en proyectos que utilizan Node.js, React o TypeScript, lo que lo convierte en la opción más utilizada en el ecosistema de pruebas automatizadas de JavaScript. En el proyecto se utiliza la creación de pruebas unitarias tanto en *frontend* como en *backend*.

Postman: herramienta que permite ejecutar peticiones HTTP de cualquier tipo, configurar cabeceras, añadir información en el cuerpo, inspeccionar las respuestas del servidor, etc. En el proyecto se utiliza para probar y depurar los *endpoints* de la API REST antes de conectarlos al *frontend*.

Lighthouse: herramienta de auditoría de código abierto. Está integrada en las herramientas de desarrollo del navegador Chrome. Su función principal es analizar páginas web y generar informes detallados sobre su calidad de rendimiento, accesibilidad, buenas prácticas, SEO y aplicaciones web progresivas. En el proyecto se ha utilizado para analizar la calidad de las interfaces.

4.8. Herramientas de desarrollo y despliegue

Git y GitHub: herramientas de control de versiones. Se utilizaron para gestionar el trabajo en equipo y el repositorio del proyecto. GitHub se usó también para la integración continua, iniciando automáticamente el despliegue de la aplicación con cada *push* a la rama principal.

Vercel: plataforma de despliegue de aplicaciones. En nuestro caso, se ha utilizado para el *frontend* de la aplicación. Tiene integración nativa con Next.js y despliegue automático desde GitHub. Se eligió Vercel por ser la plataforma recomendada oficialmente para proyectos Next.js y por su plan gratuito para el MVP.

Render: plataforma de despliegue utilizada para el backend. También tiene despliegue automático desde GitHub y soporte para aplicaciones Node.js sin tener que gestionar la infraestructura de servidores. Se eligió por ser gratuita y su fácil configuración para el despliegue del servidor Express.

Trello: herramienta de gestión de proyectos. Se utilizó para aplicar la metodología ágil explicada en el capítulo 5.

Discord: plataforma de creación de comunidades online. Se utilizó para realizar reuniones de seguimiento y comunicación general.

LucidChart y Excalidraw: herramientas de dibujo online que permiten crear diagramas de forma colaborativa en tiempo real. Ofrecen varias plantillas y elementos para crear diagramas UML, diagramas de flujo, modelos entidad-relación, etc. En el proyecto se utilizó principalmente durante la fase de especificación de requisitos y para crear ayudas visuales para la memoria del TFG.

Balsamiq Wireframes: herramienta de diseños de interfaz de bajo nivel (Wireframes). Se utilizó para crear los bocetos de la interfaz de usuario y definir la estructura visual y flujo de navegación antes de comenzar el desarrollo.

Visual Studio Code: editor de texto con soporte para crear proyectos de código y trabajar con repositorios y extensiones. Es el editor que se utilizó para programar todo el proyecto.

5. Proceso software

En este capítulo se describe el proceso software seguido durante el desarrollo de ChessClassroom. Se explica la metodología de trabajo adoptada, los roles asumidos por cada miembro del equipo, la planificación temporal del proyecto estructurada en fases y el sistema de control de versiones utilizado para coordinar el desarrollo colaborativo.

5.1. Metodología de desarrollo

La elección de una metodología de desarrollo adecuada es una de las decisiones más importantes en cualquier proyecto software, ya que determina cómo se organiza el trabajo, cómo se gestionan los cambios y cómo se garantiza la entrega de un producto funcional en los plazos establecidos.

5.1.1. Metodología aplicada

Para el desarrollo de la plataforma, se ha aplicado una metodología de trabajo ágil basada en *Scrum* (Schwaber, 2020), combinada con organización visual de las fases de trabajo de la técnica *Kanban* (Skarin, 2010).

La elección de una metodología ágil iterativo e incremental ha sido debido a que está ampliamente extendida en el sector de la Ingeniería del Software moderna para proyectos con requisitos evolutivos, ya que permite una rápida adaptación a los cambios y permite la entrega de software funcional en ciclos cortos de tiempo (Pressman, 2020).

5.1.2. Roles del equipo

Al tratarse de un equipo formado por dos desarrolladores, los roles estándares de Scrum se adaptaron de esta forma:

- **Product Owner:** Asumido por Alejandro Gil López (colaborador de proyecto), encargado de definir la visión del negocio y establecer las prioridades del *backlog*.
- **Scrum Master:** Asumido por el autor de la memoria, responsable de coordinar los tiempos, organizar las reuniones, facilitar recursos y regular el ritmo de trabajo y entrega.
- **Development Team:** Ambos miembros del equipo desarrollaron la aplicación.

5.1.3. Gestión del *Sprint* y desarrollo colaborativo

Para la gestión diaria del *Sprint Backlog*, se usó la herramienta Trello. Esta herramienta permite crear un tablero Kanban y crear unidades de trabajo agrupadas por temática, a los cuales se les puede asignar miembros, establecer fecha de entrega y registrar el tiempo invertido para cada tarea.

En nuestro caso, se ha creado un tablero Kanban con varias columnas para el *Backlog* y cada *Sprint*, como puede verse en la Figura 1. Además de las diversas columnas para el proceso de desarrollo de las funcionalidades (Refactorización, Diseño, Desarrollo, Testing, Integración) Figura 2 (Sommerville, 2016).

El seguimiento se realizó mediante reuniones diarias de planificación de corta duración (*Daily Stand-ups*) a través de Discord.



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica



Figura 1: Columnas de Backlog y Sprints

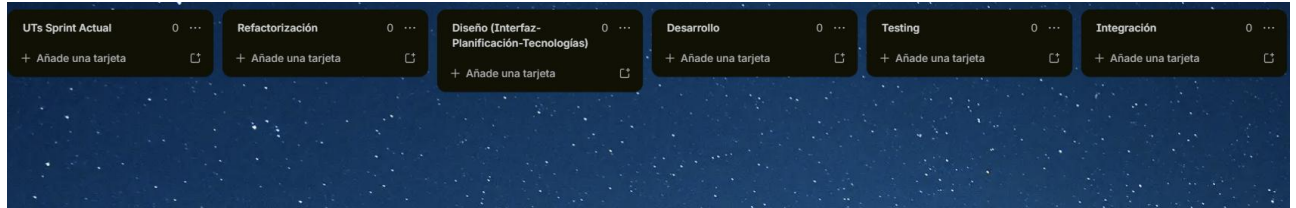


Figura 2: Columnas de fases de desarrollo de una UT

5.2. Plan de trabajo y fases

El ciclo de vida del proyecto se estructuró en tres grandes bloques: una fase inicial de ideas, análisis y diseño, seguida de una fase de desarrollo iterativo enfocado en crear un Producto Mínimo Viable (MVP) en su totalidad y finalizando con una fase de pruebas de validación con usuarios reales. Esta planificación por fases reduce los riesgos técnicos y asegura que el desarrollo se ajuste a las necesidades del *Product Owner* (Sommerville, 2016).

5.2.1. Fase 1: Análisis, especificación y diseño previo

Antes de iniciar la codificación, se estableció una etapa de preparación y diseño de la aplicación, estructurado en las siguientes actividades:

- **Estudio de viabilidad y entorno:** Se realizó un análisis de mercado y un estudio mediante una matriz DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades) para validar la necesidad y viabilidad de la idea.
- **Especificación de requisitos:** Extracción de las historias de usuario, requisitos funcionales y no funcionales y la creación del *Product Backlog*, organizando las funcionalidades por módulos de trabajo y prioridad.
- **Modelado y Arquitectura:** Se han elaborado diversos modelos: diagrama de actores, diagrama de contexto, diagrama de casos de uso, diagrama de características, modelo de dominio y el modelo de datos para la estructura de la base de datos (modelo entidad-relación). Al mismo tiempo, se definió la arquitectura de software (cliente-servidor) y se inicializó la infraestructura de los repositorios.

- **Diseño de interfaces de baja fidelidad (Wireframes):** Se elaboraron y valoraron diversos diseños de baja fidelidad para cada funcionalidad a implementar y el flujo de navegación que los une.

Se detallará más sobre estas actividades en los Capítulos 6, 7 y 8.

5.2.2. Fase 2: Desarrollo iterativo del MVP

La fase de implementación se dividió en tres *Sprints* de desarrollo de dos semanas cada uno. Cada iteración tuvo como objetivo entregar un incremento de software funcional e integrable a producción. Para ello, se estructuró el trabajo de la siguiente forma:

- **Sprint 1. Estructura y funcionalidades base:** Durante este primer *Sprint*, se construyó la infraestructura base de la aplicación. Se implementó la página de inicio (*Landing Page*), los menús de navegación, el módulo de Autenticación, el módulo de Gestión de clases, el módulo de Chat, el módulo de Ajedrez y el módulo de Planilla. El colaborador desarrolló el módulo de Gestión de alumnos y el módulo de Material adicional.
- **Sprint 2. Funcionalidades pedagógicas:** El segundo *Sprint* se centró en las funcionalidades de mayor peso educativo. Se implementó el módulo de Aula virtual, el módulo de Estudios y el módulo de Ejercicios. Además, el colaborador completó el módulo de Estadísticas, el módulo de Objetivos, el módulo de Calendario.
- **Sprint 3. Sistema de juego, elementos avanzados:** En el último *Sprint* se implementaron los módulos de mayor complejidad y concurrencia. Se implementó el módulo de Partidas y Torneos. El colaborador implementó el módulo de Stockfish y el módulo de Chatbot.

Se detallará más sobre la implementación de cada funcionalidad en el Capítulo 9.

5.2.3. Fase 3: Pruebas, estabilización y validación con usuarios

Tras finalizar los *Sprints*, se estableció una fase final dedicada exclusivamente al aseguramiento de la calidad (QA) y la validación del proyecto. Esta etapa se estructuró en las siguientes actividades:

- **Validación con usuarios finales (Beta Testing):** Despliegue de la plataforma en un entorno de pruebas controlado (*staging*). Se realizaron sesiones de evaluación. Durante las sesiones se tomó la opinión de los usuarios mediante una encuesta.
- **Refinamiento y corrección de defectos:** Se analizó el feedback recibido en las sesiones de prueba con el objetivo de identificar y resolver defectos críticos de software (*bugs*) y errores en la usabilidad (Nielsen, 1993).

Se detallará más sobre las pruebas en el Capítulo 10.

A continuación, en la Figura 3 se muestra un diagrama de Gantt que resume el desarrollo temporal del proyecto.



ChessClassroom

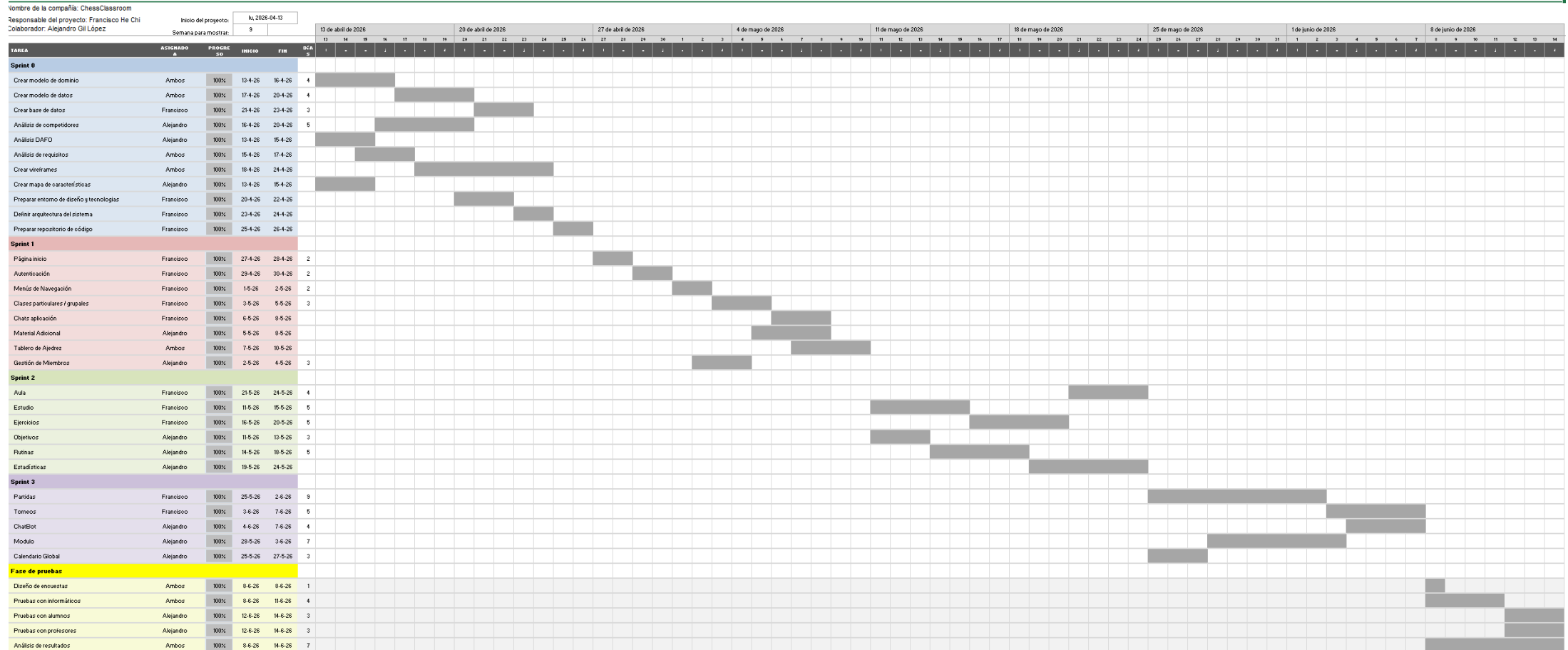


Figura 3: Diagrama de Gantt de ChessClassroom



5.3. Control de versiones

Para evitar errores en el código fuente, mantener un historial de los cambios y coordinar el trabajo colaborativo simultáneo, se utilizó Git como sistema de control de versiones distribuido, utilizando GitHub como plataforma de alojamiento remoto de los repositorios. Estas herramientas ya son un estándar en la industria del desarrollo de software, ya que permiten gestionar el historial del proyecto de forma estructurada y deshacer los cambios del código a estados anteriores en caso de error (Chacon, 2014).

A nivel estructural, se tomó la decisión de arquitectura de separar el proyecto en dos repositorios distintos en GitHub: uno dedicado exclusivamente a la capa de presentación (*frontend*) y otro a la lógica de servidor (*backend*). Esta separación de responsabilidades evita el acoplamiento del código y permite crear *pipelines* de Integración Continua y Despliegue Continuo (CI/CD) independientes y automatizadas para cada parte de la arquitectura.

El flujo de trabajo aplicado para la gestión de ramas se basó en el modelo *Feature Branch Workflow*. Se evitó trabajar directamente sobre la rama principal (*main*), ya que este se reserva para tener únicamente el código estable, funcional y listo para el despliegue en producción. El ciclo de desarrollo para cada nueva funcionalidad sigue los siguientes pasos:

1. **Creación de ramas independientes:** Por cada nueva tarea del *Sprint*, se creaba una rama derivada (*branch*) con un nombre descriptivo de la funcionalidad a implementar.
2. **Desarrollo aislado:** El código y las pruebas correspondientes se realizaban y confirmaban (*commits*) de forma local y remota en dicha rama, para no afectar al trabajo del compañero.
3. **Revisión cruzada:** Una vez finalizado el desarrollo, se abría una petición de integración (*Pull Request*). Para mantener la calidad del código, ninguna rama se fusionaba (*merge*) directamente a la rama principal sin haber sido previamente revisada y aprobada por el otro miembro del equipo.

En la Figura 4 se puede observar los resultados del merge de ramas de trabajo del frontend y backend.



Figura 4: Resultado del merge de ramas de trabajo

6. Especificación de requisitos

En este capítulo se presenta la especificación de requisitos del sistema, siguiendo las directrices del estándar IEEE 830 (IEEE) como referencia para su documentación. En primer lugar, se identifican los actores que interactúan con la plataforma. A continuación, se describen las técnicas de elicitación empleadas para recopilar las necesidades de los usuarios, seguidas de los requisitos funcionales y no funcionales del sistema. El capítulo finaliza con la descripción de los casos de uso más representativos y las pruebas de aceptación asociadas.

6.1. Identificación de actores

Para definir los límites del sistema y comprender qué entidades interactúan con él, se han identificado dos categorías de actores: actores primarios, que corresponden a personas que utilizan directamente la plataforma, y actores secundarios, que representan sistemas externos con los que ChessClassroom se comunica para ofrecer sus funcionalidades.

Como se puede observar en la Figura 5, los actores primarios comparten un actor base llamado Usuario, del cual heredan los tres actores primarios que existen en el sistema.

El Usuario no registrado es cualquier persona que accede a la plataforma sin iniciar sesión. Su interacción se limita al chatbot y a la página de inicio de la aplicación, donde puede consultar información general sobre la plataforma y acceder al registro o inicio de sesión.

El Profesor es el actor principal del sistema y es el tipo de usuario más importante de la aplicación. Una vez iniciada la sesión, tiene conjunto de funcionalidades: crear y gestionar clases, dar clases en el aula virtual, asignar y corregir ejercicios, organizar partidas y torneos, gestionar el material educativo y realizar el seguimiento del progreso de sus alumnos.

El Alumno es el segundo actor registrado del sistema. Como alumno, puede acceder a las clases mediante un código de invitación creado por el profesor y puede participar en el aula virtual, resolver ejercicios asignados, consultar su progreso y participar en partidas y torneos organizados dentro de su clase.

En cuanto a los actores secundarios, estos representan servicios externos que el sistema integra para diversas funcionalidades. Supabase forma parte de la infraestructura de *backend*, maneja la BD, el sistema de autenticación mediante JWT y los canales de chat en tiempo real. LiveKit es la plataforma externa encargada de gestionar la comunicación de audio en tiempo real dentro del aula virtual. El Motor de Análisis de Ajedrez hace referencia a Stockfish, un motor de código abierto que se ejecuta directamente en el navegador del usuario mediante WebAssembly, evalúa posiciones y calcula variantes. Por último, el Chatbot representa la integración con la API de Grok, ofrece al usuario un asistente de texto que ayuda a resolver dudas sobre funcionalidades de la plataforma.

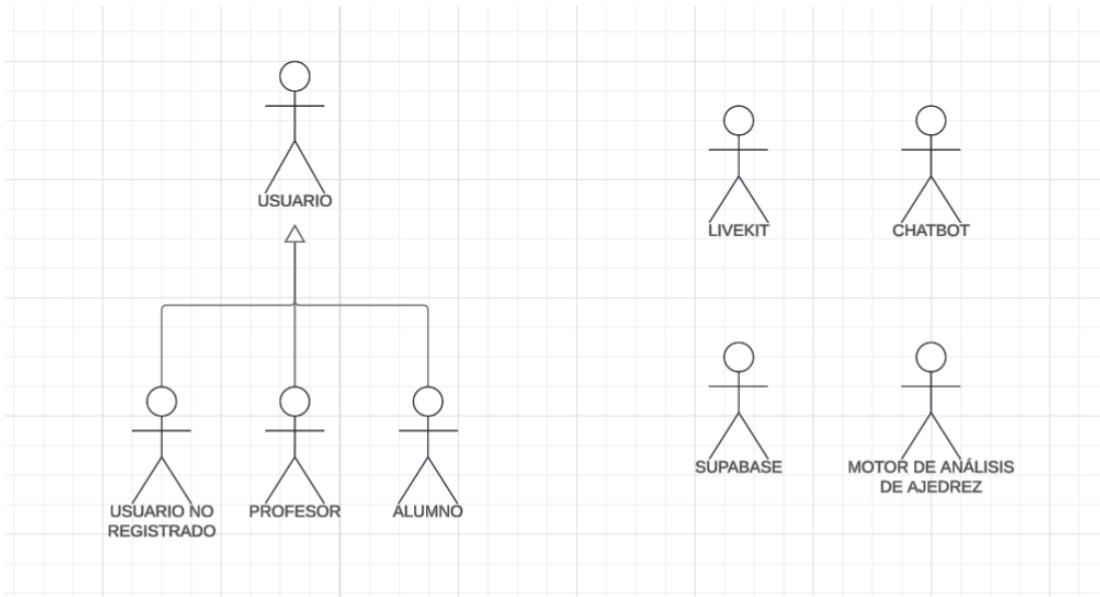


Figura 5: Diagrama de actores

Como resultado de la identificación de actores, también se realizó el diagrama del contexto del sistema. En la Figura 6 se puede observar en el lado izquierdo del sistema los actores primarios y en el lado derecho los actores secundarios.

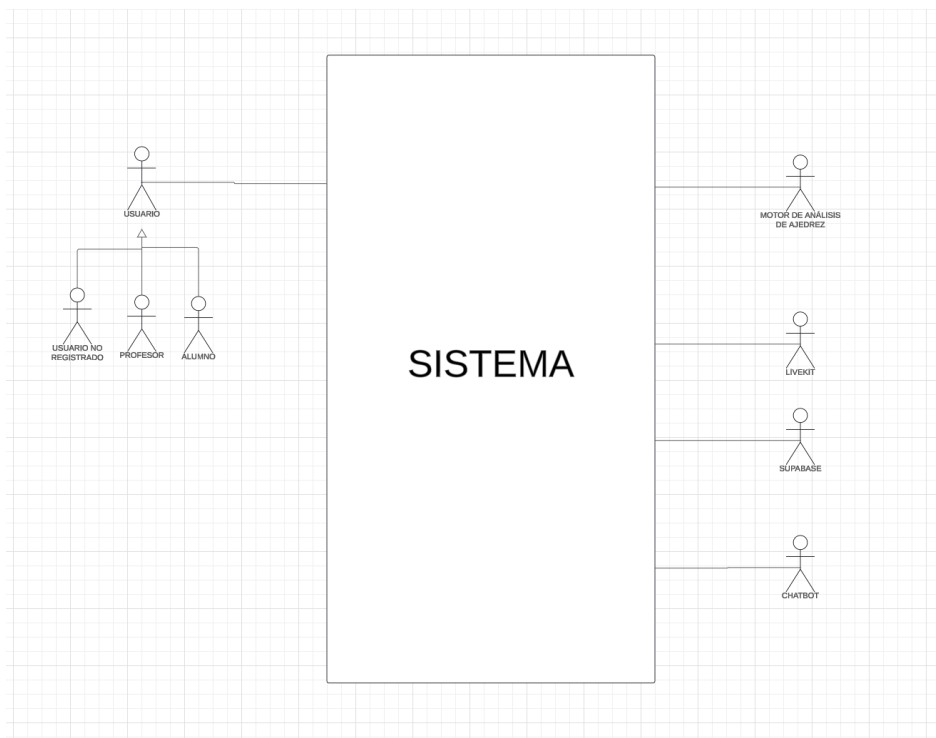


Figura 6: Diagrama de contexto del sistema

6.2. Técnicas de elicitación de requisitos

La identificación y definición de los requisitos del sistema se realizó combinando varias técnicas de elicitación. El objetivo del uso de estas técnicas es obtener una visión completa de las necesidades de los usuarios finales y del contexto del sector en el que se encuentra la plataforma.

Análisis de competidores y entorno, que consistió en un estudio detallado de las plataformas de ajedrez más relevantes del sector, como Lichess, Chess.com, ChessKid y Chessable. A esto también se le añade herramientas educativas generales. El objetivo establecer el estándar del mercado e identificar las faltas de estas plataformas, es decir, las funcionalidades que no ofrecen y que representan una oportunidad para ChessClassroom. Este análisis se detalla en el apartado 2.4 del estado del arte y sirvió como base para definir los requisitos del sistema.

Entrevistas con usuarios potenciales: Se realizaron quince entrevistas a personas de diferentes perfiles, formado principalmente por profesores y alumnos de clubs de ajedrez y tres personas con experiencia en desarrollo de software. Las entrevistas fueron de forma abierta y no estructurada, pero dirigidas a tres temas principales: qué aspectos valoran más en una herramienta de estudio, qué les gusta y qué echan en falta en las plataformas de ajedrez que ya utilizan y qué funcionalidades considerarían útiles. Esto nos permitió recopilar información de gran valor, detectando necesidades que habrían sido difíciles de encontrar con una encuesta.

Brainstorming: realizado entre los miembros del equipo. A partir de la información recogida en las entrevistas y del conocimiento del sector, se generaron ideas de forma libre sobre funcionalidades, módulos y flujos de uso. Esto permitió aumentar el alcance inicial de la especificación e identificar relaciones entre funcionalidades.

Técnica Persona: una herramienta de análisis centrado en el usuario, utilizada en ingeniería de requisitos y diseño de experiencia de usuario. Esta técnica consiste en definir perfiles arquetípicos que representan a los distintos tipos de usuarios del sistema con el objetivo de centrarse en sus necesidades durante el proceso de desarrollo (Cooper, 2007). Para este proyecto se definieron dos perfiles principales:

- Carlos, profesor de ajedrez de 38 años que lleva diez años dando clases en un club local y que acaba de comenzar a dar clases online. Su principal frustración es tener que combinar varias herramientas para gestionar a sus alumnos: WhatsApp para comunicarse, Google Drive para compartir material, Lichess para ejercicios y Zoom para videollamadas. Lo que más valora en una herramienta es que sea fácil de usar y que le permita ver de un rápidamente el progreso de cada alumno sin tener que consultar varias aplicaciones.

- Laura, alumna de ajedrez de 17 años que asiste a clases semanales en su club y que estudia por su cuenta con recursos de su profesor. Su principal frustración es que el material que recibe está separado en distintos formatos y plataformas. Valora poder revisar los ejercicios que ha hecho, ver sus errores y tener un seguimiento claro de su mejora a lo largo del tiempo.

Finalmente, la especificación de los requisitos funcionales se formalizó con historias de usuario siguiendo el formato estándar de Scrum y se documentaron tomando como referencia las reglas del estándar IEEE 830 (IEEE). Estas reglas indican que una especificación de requisitos software de calidad debe de ser correcta, completa, consistente, verificable y trazable.

6.3. Requisitos funcionales

Los requisitos funcionales definen lo que debe hacer la plataforma respecto a las acciones de los usuarios. Para su identificación se utilizaron las técnicas descritas en el apartado anterior, y el resultado se organizó por características y módulos, añadiéndose a un Product Backlog ordenado por prioridad y gestionado mediante Trello a lo largo de los tres *Sprints* de desarrollo. La Figura 7 muestra el diagrama de características elaborado durante la fase de especificación de requisitos (a medida que fue progresando el proyecto, se identificaron más).

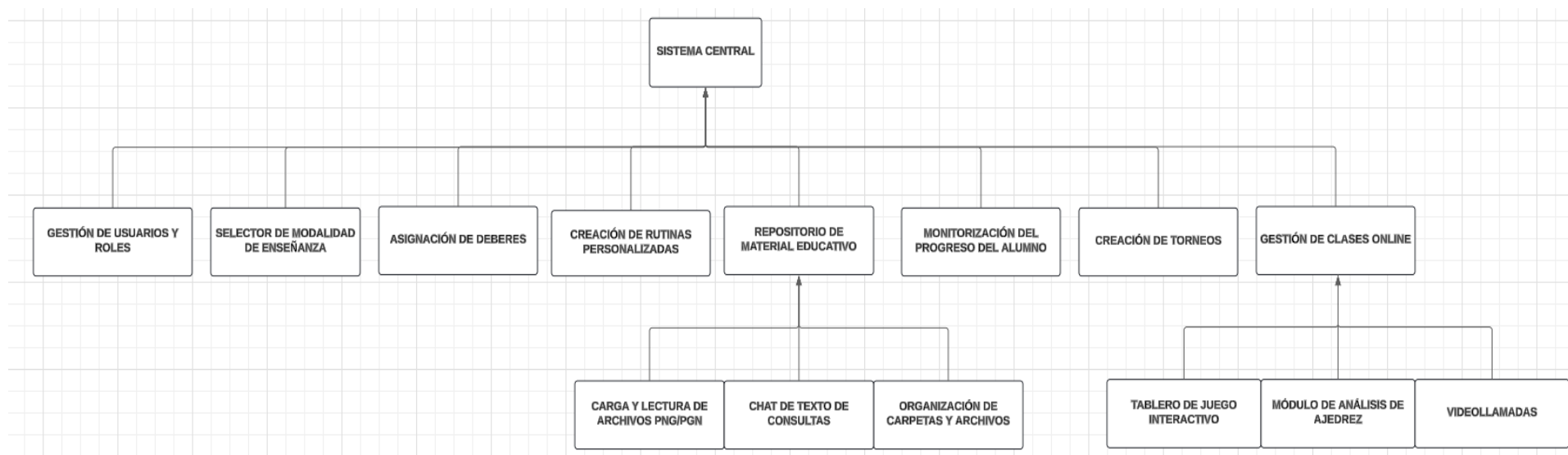


Figura 7: Diagrama de característiques

A continuación, se presenta la Tabla 2 donde se muestra algunos de los requisitos funcionales de la aplicación, algunas implementadas por el autor, algunas por el colaborador y otras que han permanecido en *Backlog* y que quedaron fuera del alcance.

Tabla 2: Ejemplos de requisitos funcionales de ChessClassroom

Identificador	Descripción	Módulo	Estado
RF01	El sistema debe permitir el inicio de sesión mediante correo electrónico y contraseña con autenticación basada en JWT	Gestión de usuarios y roles	Implementado
RF02	El sistema debe gestionar roles diferenciados entre profesor y alumno, con permisos y vistas distintas para cada uno	Gestión de usuarios y roles	Implementado
RF03	El sistema debe permitir la comunicación por voz entre los participantes del aula virtual	Gestión de clases online	Implementado
RF04	El sistema debe permitir al profesor cargar una posición en el tablero mediante texto en formato FEN, un archivo PGN, o contenido del módulo de estudios o ejercicios	Gestión de clases online	Implementado
RF05	El sistema debe permitir organizar el material educativo en carpetas con nombre personalizado	Repositorio de material educativo	Implementado
RF06	El sistema debe crear torneos en formato Arena con emparejamiento automático entre participantes	Creación de torneos	Implementado

RF07	El sistema debe integrar un asistente conversacional basado en un modelo de lenguaje para resolver dudas funcionales y teóricas	Chat de texto de consultas	Implementado (colaborador)
RF08	El sistema debe mostrar estadísticas de rendimiento por categoría de ejercicio y análisis de aperturas	Monitorización del progreso del alumno	Implementado (colaborador)
RF09	El sistema debe analizar automáticamente posiciones tácticas mediante un clasificador de IA y etiquetarlas por temática	Repositorio de material educativo	<i>Backlog</i>
RF10	El sistema debe permitir crear torneos en formato suizo	Creación de torneos	<i>Backlog</i>

6.4. Requisitos no funcionales

Los requisitos no funcionales definen las restricciones y atributos de calidad que debe satisfacer el sistema independientemente de las funcionalidades concretas que ofrece. Para su definición se tomaron como referencia las reglas del estándar ISO/IEC 25010 (ISO, 2011), en este, se establece un modelo de calidad del software, ampliamente utilizado en ingeniería del software.

A continuación, se presenta la Tabla 3 donde se muestran algunos de los requisitos no funcionales que se han tomado en cuenta para el desarrollo de la aplicación.

Tabla 3: Ejemplos de requisitos no funcionales de ChessClassroom

Identificador	Descripción	Tipo
RNF01	El sistema debe permitir la sincronización del tablero de ajedrez en el aula virtual con un tiempo inferior a 500 milisegundos	Rendimiento y tiempo real
RNF02	La interfaz de usuario debe responder menos de 100 milisegundos para acciones que no requieran comunicación con el servidor	Rendimiento y tiempo real
RNF03	Todas las comunicaciones entre el cliente y el servidor deben realizarse sobre HTTPS	Seguridad y privacidad
RNF04	El sistema debe implementar autenticación basada en JWT	Seguridad y privacidad
RNF05	La interfaz debe ser utilizable por usuarios sin conocimientos técnicos, incluyendo alumnos de corta edad y siguiendo principios de usabilidad reconocidos como los heurísticos de Nielsen (Nielsen, 1993)	Usabilidad
RNF06	El sistema debe proporcionar retroalimentación visual inmediata al usuario	Usabilidad
RNF07	El sistema debe soportar múltiples clases y torneos activos de forma simultánea sin deterioro apreciable del rendimiento	Escalabilidad y disponibilidad
RNF08	En caso de desconexión y reconexión de un usuario durante una partida, el sistema debe recuperar el estado del reloj y del tablero	Escalabilidad y disponibilidad
RNF09	El código fuente debe estar organizado en módulos independientes con separación clara entre la capa de presentación, la lógica de negocio y el acceso a datos	Mantenibilidad y portabilidad
RNF10	La arquitectura desacoplada entre <i>frontend</i> y <i>backend</i> debe permitir la sustitución o actualización de cualquiera de las dos capas sin afectar al funcionamiento del sistema completo	Mantenibilidad y portabilidad

6.5. Casos de uso principales

Para organizar el desarrollo de los requisitos funcionales, estos se han agrupado en casos de uso, los cuales a su vez fueron agrupados por características/módulos. A continuación, se muestran los casos de uso agrupados por características en la Figura 8 y los casos de uso agrupados por actores en la Figura 9.



Figura 8: Diagrama de casos de uso por característica

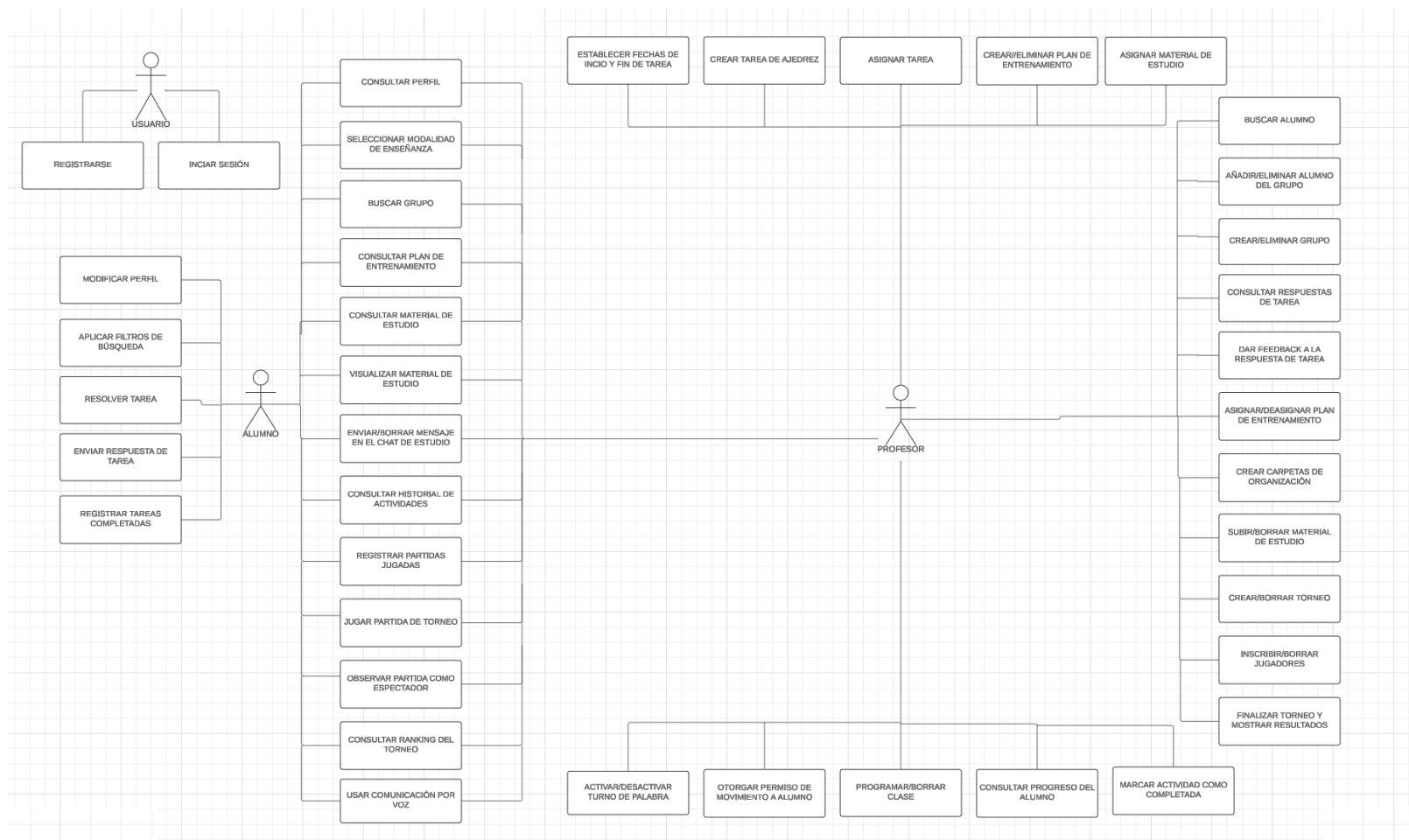


Figura 9: Diagrama de casos de uso por actor



A continuació, se describen alguns de los casos de uso más representativos del sistema. Para cada uno se indica el actor principal, las precondiciones necesarias, el flujo principal de interacción y la postcondición.

CU01 - Crear clase particular o grupal

- **Actor principal:** Profesor.
- **Precondición:** El profesor ha iniciado sesión en el sistema.
- **Flujo principal:** El profesor accede a su panel principal y selecciona la opción de crear una nueva clase. El sistema le presenta un formulario donde debe indicar el tipo de clase, particular o grupal, y un nombre. Tras confirmar el formulario, el sistema crea la clase en la base de datos, genera un código de invitación único asociado a ella y redirige al profesor a la vista de la clase recién creada.
- **Postcondición:** La clase queda registrada en el sistema y el profesor tiene un código de invitación para compartir con sus alumnos.
- **Flujo alternativo:** Si el profesor no completa los campos obligatorios, el sistema muestra un mensaje de error y no continúa con la creación.

CU02 - Resolver tarea de ajedrez

- **Actor principal:** Alumno.
- **Precondición:** El alumno ha iniciado sesión y tiene al menos una tarea asignada con fecha de inicio activa.
- **Flujo principal:** El alumno accede a la sección de ejercicios de su clase y selecciona una tarea disponible. El sistema carga la posición inicial del ejercicio en el tablero interactivo y el alumno comienza a realizar movimientos. Por cada movimiento introducido, el sistema lo valida automáticamente contra la solución pregrabada por el profesor. Si el movimiento es correcto, el tablero muestra un aviso positivo y el sistema aplica el siguiente movimiento de la solución como respuesta del oponente. Si el movimiento es incorrecto, el sistema muestra un aviso negativo y registra el intento fallido. El alumno puede añadir un comentario opcional para el profesor antes de enviar la entrega. Una vez completado el ejercicio, el sistema guarda el tiempo acumulado, el número de intentos fallidos y el PGN de la partida jugada.
- **Postcondición:** La respuesta del alumno queda registrada en el sistema y el profesor puede consultarla, evaluarla y añadir un comentario de revisión.
- **Flujo alternativo:** Si el alumno abandona la página sin completar el ejercicio, el sistema conserva el progreso parcial y el tiempo acumulado hasta ese momento.

CU03 - Crear y jugar partida de torneo

- **Actor principal:** Profesor y Alumno.
- **Precondición:** El profesor ha iniciado sesión y existe una clase con al menos dos alumnos registrados.
- **Flujo principal:** El profesor accede al módulo de torneos de su clase y crea un nuevo torneo en formato Arena. Para ello, configura el nombre, las fechas, el tiempo de partida, el incremento tras cada jugada, los participantes y opcionalmente el estado inicial del tablero. El sistema registra el torneo y lo publica como disponible para los alumnos.

Una vez iniciado el torneo, el sistema genera automáticamente emparejamientos entre los jugadores disponibles. Cada par de jugadores recibe una notificación de inicio de partida y accede al tablero de juego. Los jugadores realizan sus movimientos de forma alternada, con el reloj descontando el tiempo de cada uno. Al finalizar la partida por jaque mate, tiempo agotado, abandono o tablas acordadas, el sistema registra el resultado y actualiza la clasificación del torneo en tiempo real. El proceso de emparejamiento se repite automáticamente hasta que el torneo finaliza.

- **Postcondición:** Los resultados de todas las partidas quedan registrados, la clasificación final del torneo se muestra a todos los participantes y el evento del torneo aparece sincronizado en el calendario de la clase.
- **Flujo alternativo:** Si un jugador no realiza su primer movimiento en los primeros treinta segundos, el sistema aborta la partida automáticamente.

CU04 - Cargar archivo PGN en el módulo de estudios

- **Actor principal:** Profesor.
- **Precondición:** El profesor ha iniciado sesión y accede al módulo de estudios de una clase.
- **Flujo principal:** El profesor selecciona una carpeta de organización existente o crea una nueva. A continuación, sube un archivo PGN individual o una base de datos PGN multi-partida desde su dispositivo. El sistema procesa el archivo, extrae los metadatos de cada partida y lo almacena en Supabase Storage. Una vez cargado, el archivo queda disponible en el explorador de estudios de la clase.

El profesor puede seleccionar cualquier partida para visualizarla en el tablero interactivo, navegar por el historial de movimientos mediante la planilla con soporte de variantes y activar el motor Stockfish para analizar posiciones concretas.

- **Postcondición:** El archivo queda almacenado y accesible para el profesor y los alumnos de la clase según los permisos configurados.
- **Flujo alternativo:** Si el archivo subido no tiene un formato PGN válido, el sistema muestra un mensaje de error y no realiza el almacenamiento.

6.6. Pruebas de aceptación

Las pruebas de aceptación definen los criterios que debe satisfacer el sistema para que una funcionalidad se considere correctamente implementada desde el punto de vista del usuario. Cada prueba se especifica con un identificador, una descripción, las precondiciones necesarias, los pasos a seguir y el resultado esperado. A continuación, y a modo de ejemplo, en la Tabla 4 se detallan las pruebas de aceptación diseñadas para el CU02 - Resolver tarea de ajedrez.

Tabla 4: Pruebas de aceptación para CU02 - Resolver tarea de ajedrez

Identificador y descripción	Precondición	Pasos	Resultado esperado
PA01 - Acceso a un ejercicio asignado con fecha activa	El alumno ha iniciado sesión y tiene al menos un ejercicio asignado cuya fecha de inicio ya ha llegado	El alumno accede al módulo de ejercicios de su clase y selecciona el ejercicio disponible	El sistema carga la posición inicial del ejercicio en el tablero interactivo y el alumno puede comenzar a interactuar con él
PA02 - Movimiento correcto en un ejercicio	El alumno tiene un ejercicio abierto y en estado activo	El alumno realiza un movimiento que coincide con la solución pregrabada por el profesor	El tablero muestra un aviso positivo, el sistema aplica automáticamente el siguiente movimiento de la solución como respuesta del oponente y el contador de intentos fallidos no se incrementa
PA03 - Movimiento incorrecto en un ejercicio	El alumno tiene un ejercicio abierto y en estado activo	El alumno realiza un movimiento que no coincide con la solución pregrabada	El tablero muestra un aviso negativo, el movimiento no se aplica y el sistema incrementa el contador de intentos fallidos
PA04 - Abandono y reanudación de un ejercicio	El alumno tiene un ejercicio en curso y abandona la página sin completarlo	El alumno cierra la página del ejercicio y vuelve a acceder a él	El sistema conserva el progreso parcial y el tiempo acumulado hasta el momento del abandono

PA05 - Entrega de un ejercicio completado	El alumno ha completado todos los movimientos correctos del ejercicio	El alumno añade opcionalmente un comentario para el profesor y confirma la entrega	El sistema registra la entrega con el tiempo total acumulado, el número de intentos fallidos y el PGN de la partida jugada. Además, marca el ejercicio como completado
PA06 - Consulta de entregas por parte del profesor	Al menos un alumno ha completado o iniciado un ejercicio asignado	El profesor accede al módulo de ejercicios y selecciona el ejercicio correspondiente	El sistema muestra el estado de la entrega de cada alumno, incluyendo el tiempo empleado, los intentos fallidos y el PGN jugado
PA07 - Evaluación de una entrega por parte del profesor	El profesor visualiza la entrega de un alumno en el módulo de ejercicios	El profesor asigna una puntuación del uno al cinco y añade un comentario de revisión	La evaluación queda registrada y es visible para el alumno en su vista del ejercicio

Las entidades **Sistema de Pago**, **Cámara** y **Test**, visibles en el diagrama, corresponden a funcionalidades planificadas durante la fase de análisis que quedaron fuera del alcance y no han sido implementadas en la versión final del TFG.

7.2. Modelo entidad-relación de la base de datos

El modelo entidad-relación define la estructura de información de las entidades del sistema. Este modelo sirve para el diseño de la BD, mostrando los atributos, las relaciones y las cardinalidades de cada elemento. Su elaboración se realizó siguiendo la notación estándar UML (Object Management Group, 2017) lo que permite expresar de forma precisa y sin ambigüedades tanto la estructura de los datos como las restricciones de integridad que debe tener el sistema.

Al igual que el modelo de dominio, este diagrama fue elaborado durante la fase de análisis previa al desarrollo y muestra la visión completa del sistema en aquel momento. Como consecuencia, faltan algunas entidades presentes en el estado final del proyecto y algunas de las entidades y relaciones que aparecen en el diagrama pertenecen a funcionalidades que quedaron fuera del alcance del TFG.

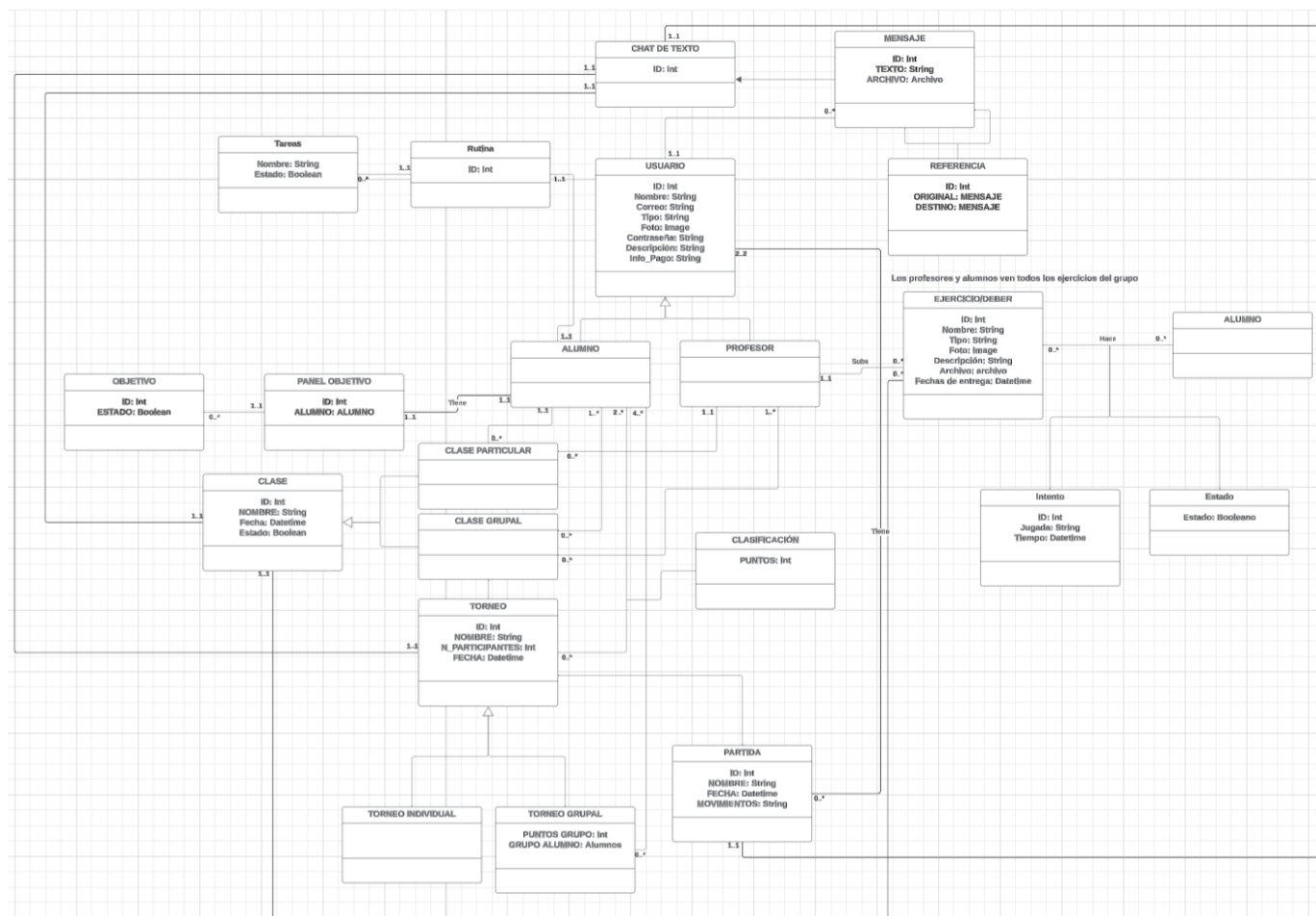


Figura 11: Modelo de entidad-relación de la base de datos de ChessClassroom



Como se puede observar en la Figura 11, la entidad central del modelo es **Usuario**. Esta entidad almacena los atributos comunes a todos los usuarios: identificador, nombre, correo electrónico, tipo de rol, fotografía de perfil, contraseña y descripción. De ella se especializan las entidades **Alumno** y **Profesor**, que heredan estos atributos y añaden las relaciones específicas de cada rol dentro de la plataforma. Cabe recalcar que este enfoque de generalización y especialización es coherente con las recomendaciones de (Larman, 2004) para el modelado de sistemas con roles diferenciados, ya que permite centralizar la lógica común y extender el comportamiento de forma limpia en las subclases.

La entidad **Clase** es un elemento importante del sistema, y se especializa en dos subtipos: **Clase Particular**, representando a la relación entre un profesor y un único alumno, y **Clase Grupal**, agrupando a múltiples alumnos bajo la gestión de un mismo profesor. Cada clase tiene un nombre, una fecha de creación y un estado activo o inactivo. Un profesor puede gestionar múltiples clases, y una clase particular está asociada a un único alumno, mientras que una clase grupal puede tener varios.

La entidad **Torneo** está asociada a una clase y se especializa a su vez en **Torneo Individual** y **Torneo Grupal**, siendo el segundo el que almacena la puntuación por grupo y la lista de alumnos participantes. Cada torneo puede agrupar múltiples **Partidas**, que registran los movimientos en formato de cadena de texto, la fecha y el nombre de la partida. La entidad **Clasificación** registra los puntos acumulados por cada participante a lo largo del torneo.

El módulo de ejercicios se representa mediante la entidad **Ejercicio**, que almacena el nombre, tipo, descripción, archivo adjunto y fechas de entrega. Esta entidad se relaciona con el **Profesor** que la crea y con los **Alumnos** que la realizan. Cada intento de solución queda registrado en la entidad **Intento**, que almacena la jugada realizada y el tiempo demorado correspondiente. La entidad **Estado** representa el progreso actual del alumno en ese ejercicio. Esta estructura permite al profesor consultar el historial de resolución de cada alumno de forma detallada.

La comunicación entre usuarios se gestiona a través de la entidad **Chat de Texto**, que agrupa los **Mensajes** intercambiados. Cada mensaje contiene un texto y opcionalmente un archivo adjunto, y puede referenciar a otro mensaje previo mediante la entidad **Referencia**, que almacena el mensaje original y el mensaje de destino para implementar la funcionalidad de respuesta en hilo.

El seguimiento del alumno se gestiona mediante las entidades **Panel de Objetivo** y **Objetivo**. Un alumno puede tener uno o varios paneles de objetivos, cada uno de los cuales agrupa múltiples objetivos con su estado de completitud. La entidad **Rutina** representa los planes de entrenamiento periódicos asociados a un usuario, y la entidad **Tareas** agrupa las actividades concretas de cada rutina con su nombre y estado de realización.

7.3. Modelo del árbol de movimientos

Uno de los aspectos técnicos más relevantes de la aplicación es la representación del historial de una partida de ajedrez. En las plataformas con menos impacto en el sector, el historial de movimientos se almacena como una lista lineal ordenada, lo que imposibilita representar líneas alternativas o variantes sobre una misma posición. Para superar esta limitación, ChessClassroom implementa una estructura de datos no lineal basada en un árbol de nodos, donde cada nodo representa una posición concreta del tablero y sus hijos representan las distintas continuaciones posibles desde esa posición.

Esta decisión de diseño es relevante en el contexto de la enseñanza del ajedrez donde el análisis de variantes es una práctica habitual en la educación y entrenamiento a alto nivel. Un profesor que analiza una partida en el aula virtual puede retroceder a un punto concreto del historial y explorar una jugada alternativa sin perder la línea principal, generando así una bifurcación en el árbol. Del mismo modo, esto le permite al módulo de ajedrez y planilla cargar archivos PGN con variantes anidadas, cuya estructura es compatible con la representación en árbol.

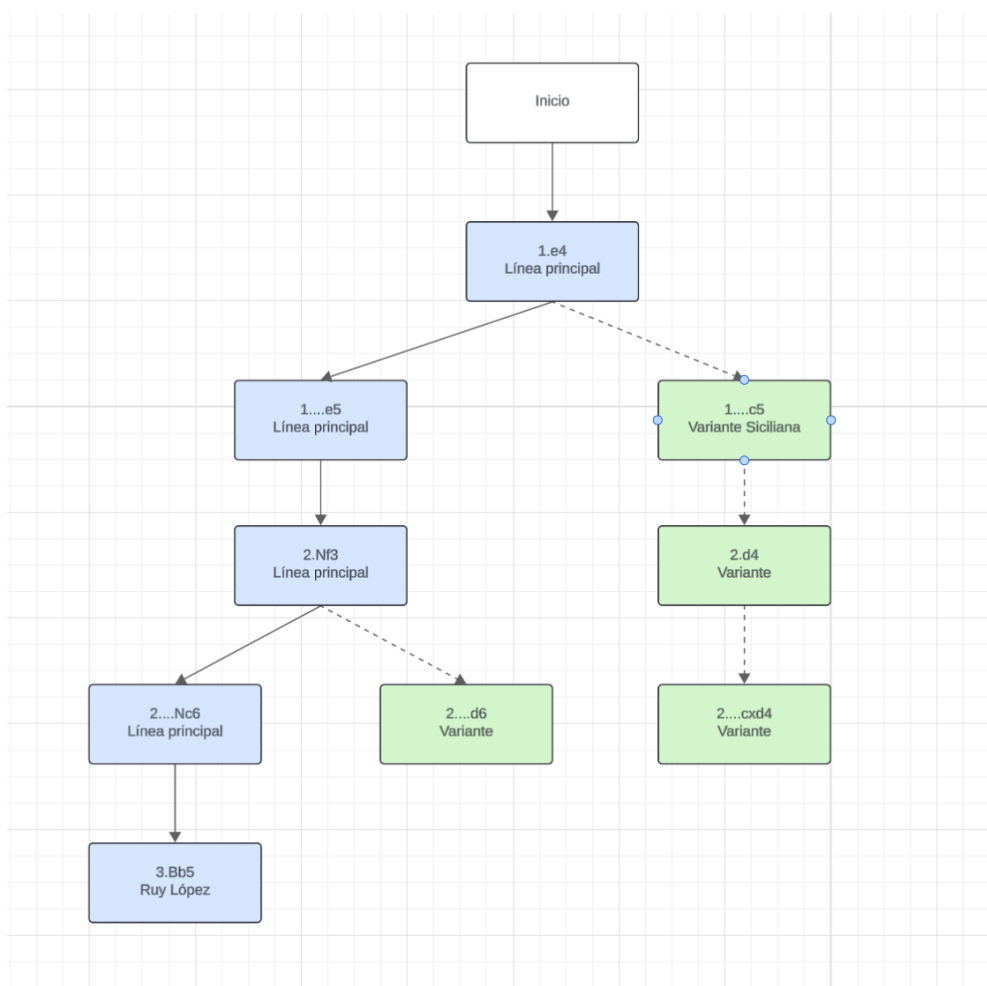


Figura 12: Modelo del árbol de movimientos

Como se puede observar en la Figura 12, el árbol comienza en un nodo raíz que representa la posición inicial de la partida. A partir de este, la línea principal azul desciende en vertical siguiendo la secuencia de movimientos jugados, mientras que las variantes se representan como ramas laterales verdes que se bifurcan en el momento donde se hace una jugada alternativa.

Cada nodo del árbol almacena la siguiente información: el identificador único del nodo, el identificador del nodo padre, la notación del movimiento en formato SAN, la posición resultante en formato FEN (FIDE, 2023), el color que realizó el movimiento y un indicador de si hubo captura. Esta estructura permite navegar por el árbol en cualquier dirección, avanzar o retroceder en el historial y saltar entre variantes sin necesidad de recalcular las posiciones desde el inicio (Knuth D. E., 1997).

La serialización del árbol se realiza en formato PGN estándar, que representa la línea principal de forma secuencial e incluye las variantes entre paréntesis de forma anidada.

Se desarrollará con más detalle la implementación del árbol y su sistema de nodos en el Capítulo 9.

8. Arquitectura y diseño

En este capítulo se describe la arquitectura y el diseño del sistema de ChessClassroom. En primer lugar, se presenta la arquitectura general del sistema, mostrando cómo se organizan las tres capas principales de la aplicación. A continuación, se detalla la arquitectura interna del *frontend* y del *backend* por separado. Después, se explica el diseño del sistema de comunicación en tiempo real y se presentan los modelos de interfaz de usuario elaborados durante la fase de diseño previo. El capítulo finaliza con una descripción de los patrones de diseño aplicados a lo largo del desarrollo.

8.1. Arquitectura general del sistema

En ChessClassroom se aplica una arquitectura cliente-servidor desacoplada organizada en tres capas: la capa de presentación, la capa de lógica de negocio y la capa de persistencia. Esta separación de responsabilidades mejora el mantenimiento independiente para cada área de la aplicación. Además, permite escalar cada componente de forma autónoma y crea una base de código abierta a una futura migración a otros entornos o tecnologías, como aplicaciones móviles sin necesidad de reescribir la lógica de negocio (BI) (Sommerville, 2016).

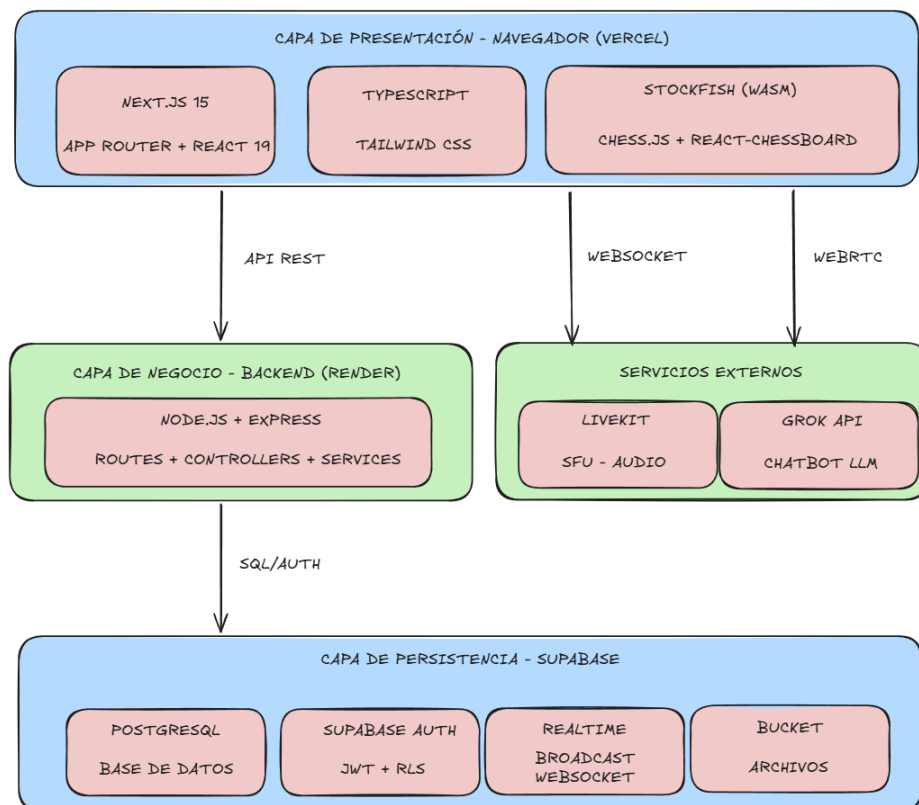


Figura 13: Arquitectura general del sistema

Como se puede observar en la Figura 13, la capa de presentación se encuentra en el navegador del usuario y se despliega a través de Vercel mediante integración continua desde el repositorio de GitHub. Está desarrollada con Next.js 15 utilizando el App Router, React 19 y TypeScript, con Tailwind CSS para los estilos. En esta capa se destaca la integración del motor de análisis Stockfish mediante WebAssembly, esto permite ejecutar cálculos de alto consumo computacional directamente en el navegador del usuario sin consumir recursos del servidor.

La capa de lógica de negocio forma el servidor backend. Está desarrollada con Node.js, Express y TypeScript y desplegado en Render. Este servidor expone una *API REST* que recibe las peticiones del cliente, las valida, aplica la lógica de negocio correspondiente y accede a la base de datos a través de los servicios de Supabase. Además, esta capa gestiona la autenticación mediante *JWT* y las reglas de autorización según el rol del usuario.

La capa de persistencia está gestionada por Supabase. Esta plataforma proporciona cuatro servicios para la aplicación web: una base de datos PostgreSQL para la persistencia de los datos, un sistema de autenticación con soporte para *JWT*, políticas de seguridad a nivel de fila (RLS), el servicio Supabase Realtime para la sincronización de eventos en tiempo real mediante *WebSockets* y un sistema de buckets para almacenar y referenciar archivos.

Además de estas tres capas, el sistema integra dos servicios externos especializados. LiveKit actúa como servidor *SFU* para la gestión de la comunicación de audio en tiempo real dentro del aula virtual, donde internamente integra el protocolo *WebRTC*. La *API* de Grok proporciona el motor de lenguaje natural que controla el chatbot de ayuda.

8.2. Arquitectura del frontend

El frontend de ChessClassroom está desarrollado utilizando el *App Router*, que organiza la aplicación en torno a una estructura de carpetas donde cada carpeta corresponde a un segmento de la URL y puede definir su propio *layout*, página y renderizado. Esta estructura mejora la organización modular del código y aprovecha las capacidades de renderizado del servidor de Next.js, por ejemplo, los React Server Components para las partes de la aplicación que no necesitan interacción con el cliente.

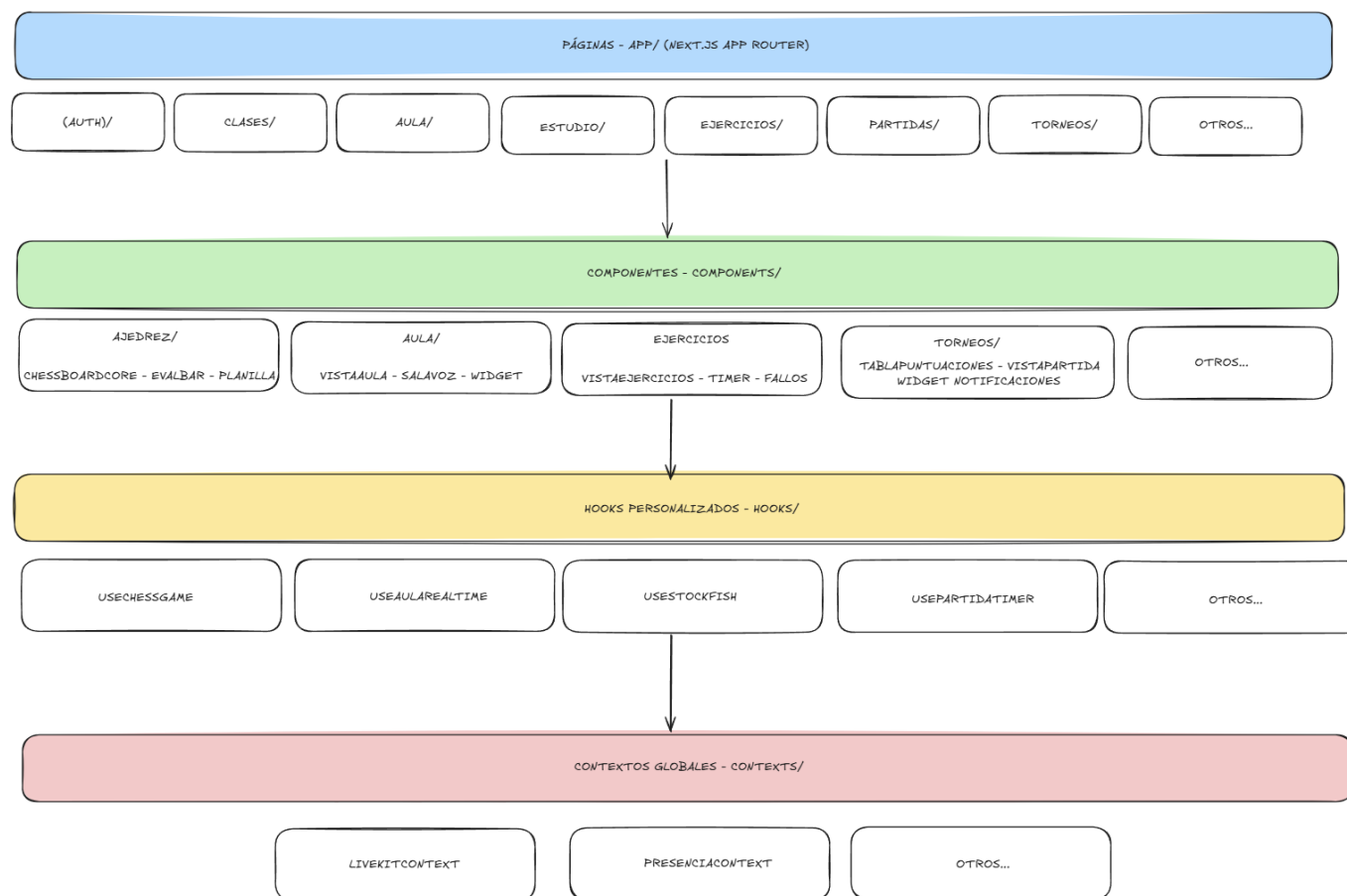


Figura 14: Arquitectura del frontend detallada



Como se puede observar en la Figura 14, la arquitectura del frontend se organiza en cuatro capas: páginas, componentes, *hooks* personalizados y contextos globales.

La capa de páginas, bajo `app/`, establece las rutas de la aplicación y actúa como punto de entrada de cada módulo. Las rutas verifican que exista un *token JWT* válido antes de renderizar el contenido, redirigiendo al usuario al formulario de inicio de sesión en caso contrario. Dentro de esta capa se destaca el uso de *layouts* anidados, especialmente en la ruta `clases/[id]/`, cuyo `layout.tsx` actúa como contenedor para los contextos globales de presencia y comunicación por voz, esto asegura que se mantengan activos, aunque el usuario navegue entre los distintos módulos de una misma clase.

La capa de componentes, bajo `components/`, agrupa los elementos reutilizables organizados por módulo. Los componentes de esta capa son principalmente aquellos que reciben datos y *callbacks* mediante *props* y pasan la lógica de negocio a las capas inferiores. Entre los más relevantes se destacan `ChessboardCore`, que encapsula el tablero interactivo con soporte para arrastre de piezas, marcadores de movimientos legales y promoción de peones. Otro componente es `VistaAula`, que integra varios subcomponentes, como el tablero sincronizado, la lista de participantes y los controles de moderación del profesor.

La capa de *hooks* personalizados, bajo `hooks/`, encapsula la lógica compleja y con estado que sería demasiado pesada para estar en los componentes. Cada *hook* tiene una funcionalidad clara y delimitada: `useChessGame` gestiona el árbol de variantes y la navegación por el historial de movimientos, `useAulaRealtime` gestiona el canal de Supabase Realtime del tablero compartido, `useStockfish` gestiona la comunicación con el motor de análisis mediante Web Workers, y `usePartidaTimer` gestiona el reloj de las partidas con sincronización de *timestamps* del servidor.

La capa de contextos globales, bajo `contexts/`, gestiona el estado compartido que debe persistir entre navegaciones dentro de una misma clase. `LiveKitContext` mantiene la conexión a la sala de voz, aunque el usuario abandone la página del aula, mostrando un *widget* flotante arrastrable que permite seguir participando en la comunicación por voz desde cualquier módulo. `PresenciaContext` gestiona la lista de usuarios conectados a la clase en tiempo real mediante un sistema de *broadcast* manual implementado con Supabase Realtime.

8.3. Arquitectura del backend

El backend de ChessClassroom está desarrollado con Node.js y Express en TypeScript. Sigue una arquitectura de tres capas similar al patrón Modelo-Vista-Controlador (MVC), adaptada al contexto de una API REST donde la capa de vista es sustituida por las respuestas JSON que el servidor devuelve al cliente. Esta organización separa claramente las responsabilidades de cada capa y facilita tanto el mantenimiento del código como la incorporación de nuevas funcionalidades sin afectar al resto del sistema (Pressman, 2020).

Como se puede observar en la Figura 15, las tres capas del backend son las siguientes:

La capa de rutas, bajo `routes/`, define los *endpoints* de la API REST y los asocia a los controladores correspondientes. Cada módulo tiene su propio archivo de rutas, esto permite localizar rápidamente el punto de entrada de cualquier funcionalidad. Las rutas protegidas incorporan un middleware de autenticación que verifica el *token* JWT incluido en la cabecera de cada petición antes de permitir el acceso al controlador correspondiente.

La capa de controladores, bajo `controllers/`, actúa como intermediario entre la capa de rutas y la capa de servicios. Se encarga de *parsear* los parámetros de la petición HTTP, invocar el servicio correspondiente con los datos necesarios y construir la respuesta que se devuelve al cliente. Los controladores no contienen lógica de negocio, esto hace que sean sencillos y fácilmente *testeables*. En esta capa se gestiona también la configuración de CORS, que permite el acceso desde el dominio del frontend desplegado en Vercel. Cada módulo tiene su propio controlador.

La capa de servicios, bajo `services/`, contiene toda la lógica de negocio de la aplicación. Es la capa que interactúa directamente con la BD a través del cliente de Supabase. También es donde se implementan las operaciones más complicadas del sistema, como la validación de movimientos en partidas, la generación de emparejamientos en torneos con formato Arena, la gestión de permisos en el aula virtual o el procesamiento de archivos PGN. Cada módulo tiene su propio servicio con responsabilidades definidas: auth, clases, aula, ejercicios, partidas, torneos y estudios, entre otros.

Se utilizó TypeScript en toda la capa de backend. Esto aporta tipado estático que reduce los errores en tiempo de ejecución, sobre todo en las operaciones con la base de datos, donde los tipos de las respuestas de Supabase se definen para mantener la coherencia entre el esquema de la base de datos y el código de la aplicación.

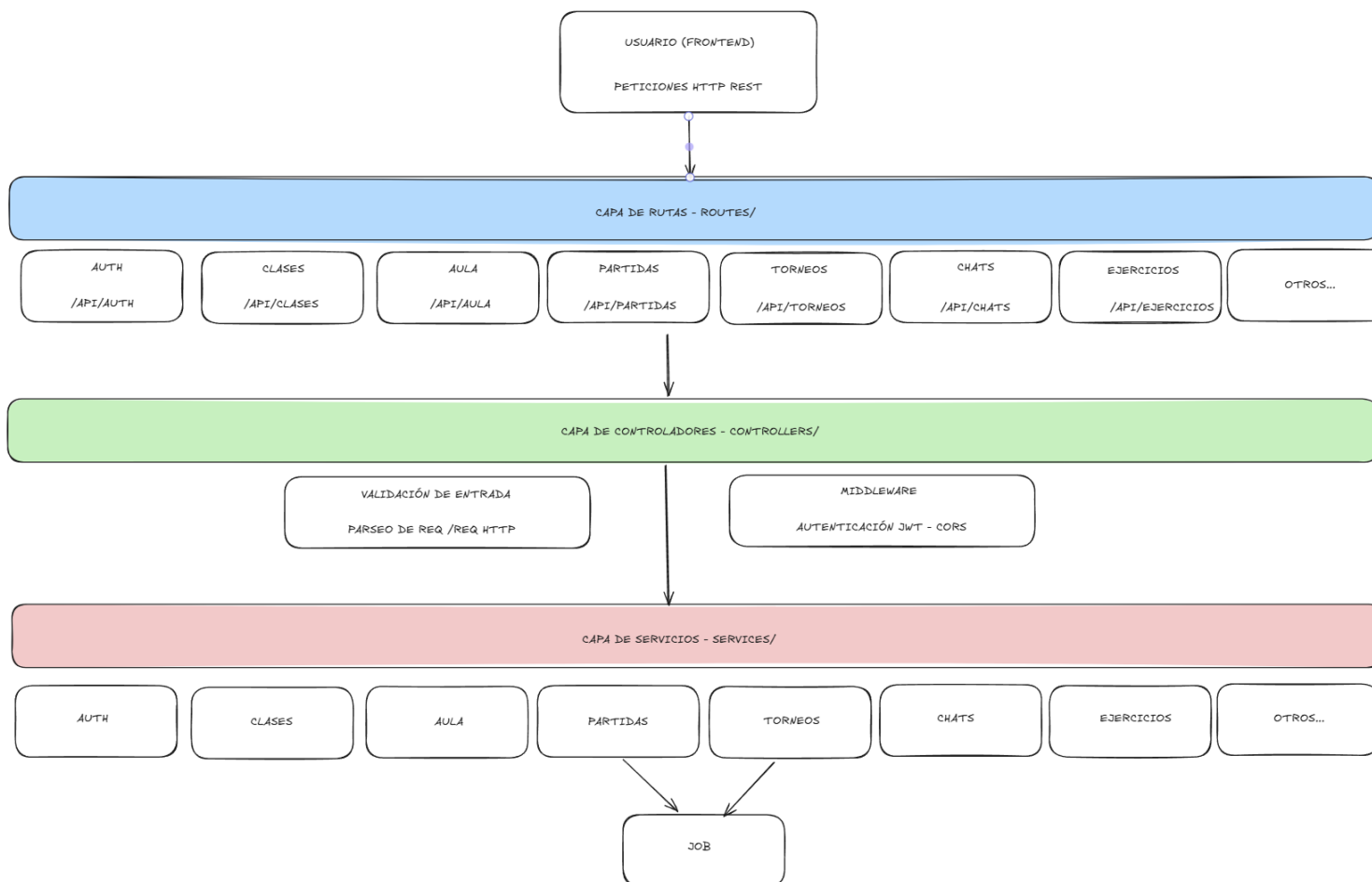


Figura 15: Arquitectura del backend detallada

8.4. Diseño del sistema de tiempo real

La comunicación en tiempo real es uno de los aspectos más importantes de ChessClassroom, ya que varios módulos de la plataforma necesitan una sincronización precisa e instantánea entre múltiples clientes de forma simultánea. El sistema de tiempo real funciona con tres canales de comunicación independientes. Cada uno de los canales se especializa en un tipo de interacción distinta: el canal de chat de texto, el canal de sincronización del tablero y la presencia y el canal de comunicación por voz.

Como se puede observar en la Figura 16, estos tres canales funcionan de forma paralela e independiente, esto permite que el aula virtual mantenga simultáneamente el chat de texto en tiempo real, el tablero sincronizado, la sala de voz activa y, evitando que el fallo o la latencia de uno afecte al funcionamiento de los otros.

El primer canal gestiona la mensajería de texto en tiempo real usando una suscripción a la tabla de mensajes de Supabase. Esto hace que cada nuevo mensaje insertado en la base de datos se propague automáticamente a todos los clientes suscritos al canal correspondiente sin tener que realizar otras peticiones al servidor.

El segundo canal utiliza el servicio Supabase Realtime con su sistema de Broadcast, este emite eventos a todos los clientes suscritos a un mismo canal sin necesidad de almacenar los mensajes en la base de datos. El canal del aula se identifica mediante el patrón aula: {aulaid} y maneja dos tipos de eventos: los eventos del tablero compartido que son: movimientos, navegación por el historial, carga de posiciones, reinicio, cambio de orientación, permisos de movimiento y sincronización del análisis de Stockfish; y los eventos de presencia, que mantienen una lista actualizada de los usuarios conectados al aula en tiempo real. El sistema de presencia ha sido implementado de forma manual mediante eventos de Broadcast, esto es debido a que el mecanismo de Presence de Supabase tuvo problemas durante el desarrollo. La solución para la presencia es un protocolo de cuatro eventos: CONECTADO, cuando un usuario emite su llegada y solicita que los demás respondan con su estado; PRESENCIA, como respuesta al anuncio de llegada; ACTUALIZAR, para modificar campos como el estado de conexión a la sala de voz; y DESCONECTADO, cuando el usuario sale del aula. Además, se emite un *heartbeat* cada treinta segundos para mantener la lista actualizada por desconexiones inesperadas.

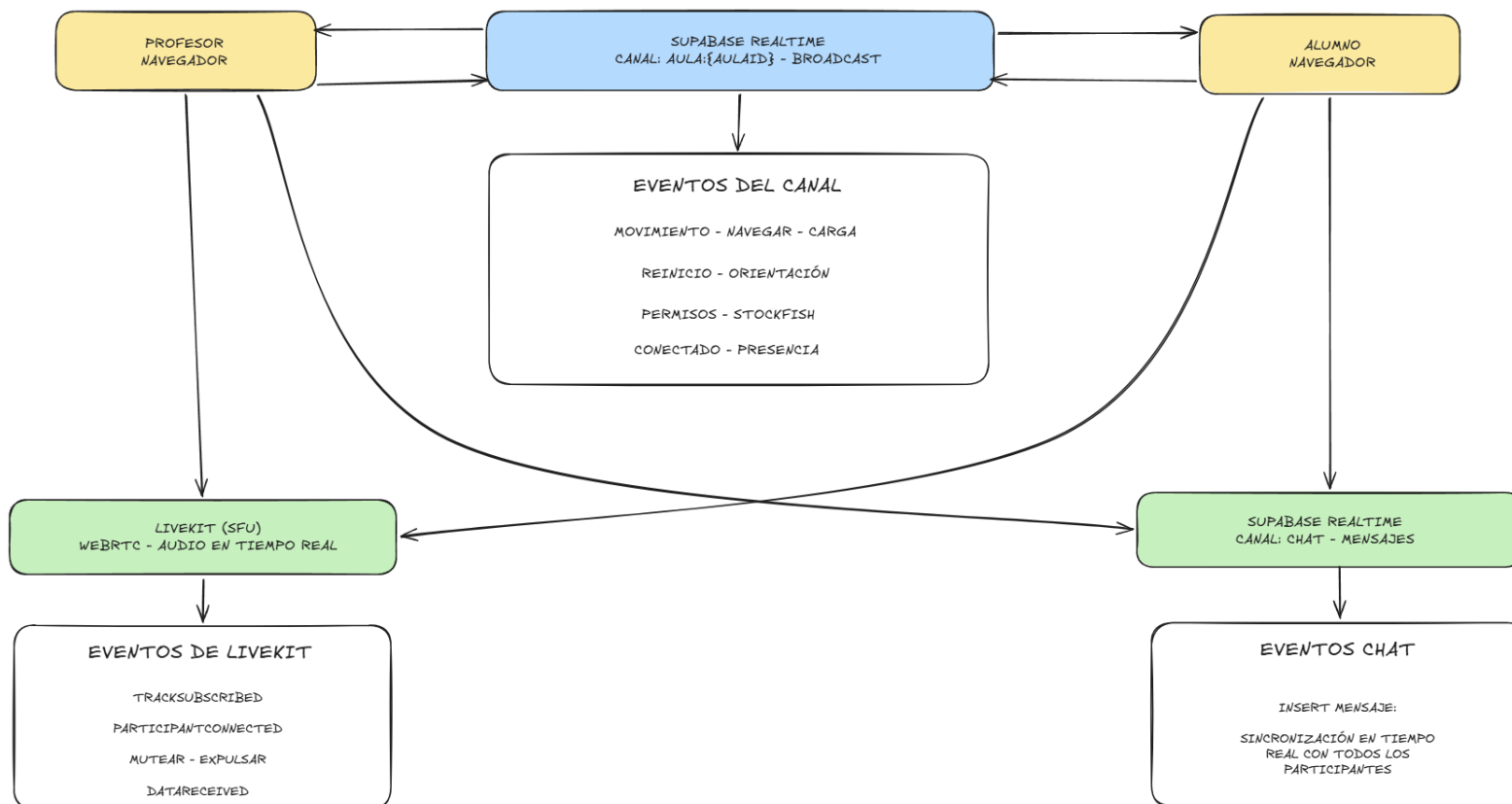


Figura 16: Diseño del sistema en tiempo real



El tercer canal maneja la comunicación de audio en tiempo real a través de LiveKit, una plataforma basada en WebRTC que implementa una arquitectura SFU. Su diferencia respecto a la arquitectura punto a punto es que actúa como servidor central que recibe el audio de cada participante y los reenvía selectivamente al resto. Esto reduce el consumo de ancho de banda del usuario y escala la sala de voz a más participantes (Grigorik, 2013). La integración de LiveKit en el *frontend* se hace mediante un contexto React que persiste en el *layout* de clases, así, la sala de voz permanece activa, aunque el usuario navegue a otros módulos. Las acciones de control de audio de cada participante se añaden directamente al documento HTML en lugar de gestionarse con componentes React. Esto se ha hecho para evitar problemas de sincronización con el ciclo de vida de los componentes cuando los *tracks* de audio llegan antes de que los efectos del componente se hayan ejecutado.

8.5. Diseño de la interfaz de usuario

Durante la fase previa al desarrollo, se realizaron varios modelos de interfaz de baja fidelidad (*wireframes*) utilizando la herramienta Balsamiq. Estos *wireframes* definen la estructura visual y la organización de los componentes para cada módulo antes de implementarse. Al ser modelos de baja fidelidad, el objetivo es probar las funcionalidades y detectar problemas de usabilidad antes de crear un diseño final. En este apartado, se mostrarán algunos de ellos.

Hay que destacar que el diseño de la interfaz de usuario se ha basado en los principios de usabilidad (Nielsen, 1993), centrándose sobre todo en la respuesta inmediata y la reducción de la carga mental para ejecutar funcionalidades. Debido a que la plataforma está orientada a profesores y alumnos de bajo nivel técnico, se priorizó una interfaz clara e intuitiva que no necesite mucha formación previa.

La Figura 17 muestra la página de inicio o *landing page*. Esta página lista la información de la plataforma a los usuarios no registrados. Lista las funcionalidades principales e incluye accesos al registro y al inicio de sesión.

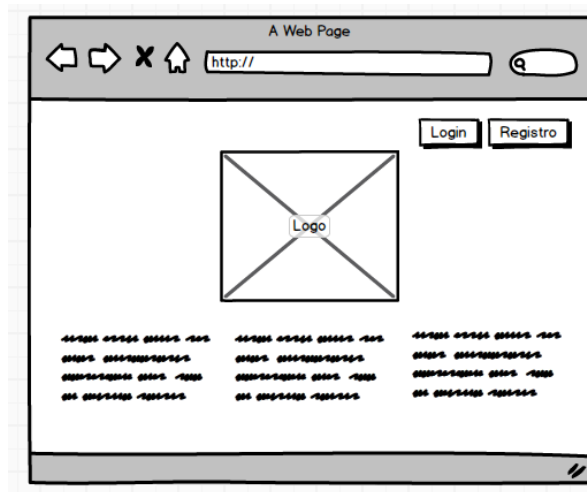


Figura 17: Wireframe página de inicio

Una vez autenticado, el profesor pasa a su panel principal, donde puede acceder y crear sus clases. El alumno pasa a un panel similar que muestra las clases en las que está matriculado. La Figura 18 muestra la pantalla de selección de listas de clases y la Figura 19 muestra la lista de clases grupales con una clase en estado inactivo.

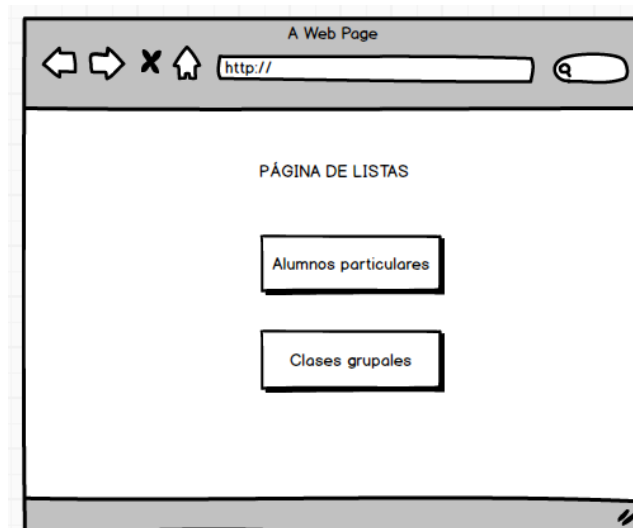


Figura 18: Wireframe selección de listas de clases

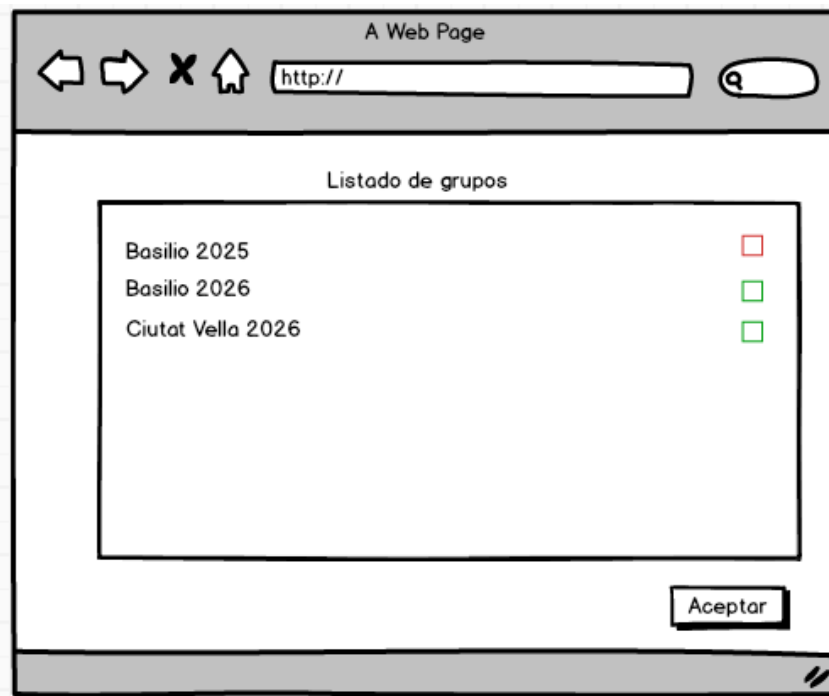


Figura 19: Wireframe lista de clases grupales

La Figura 20 muestra el menú de navegación de la clase, desde el cual el usuario puede acceder a los distintos módulos.

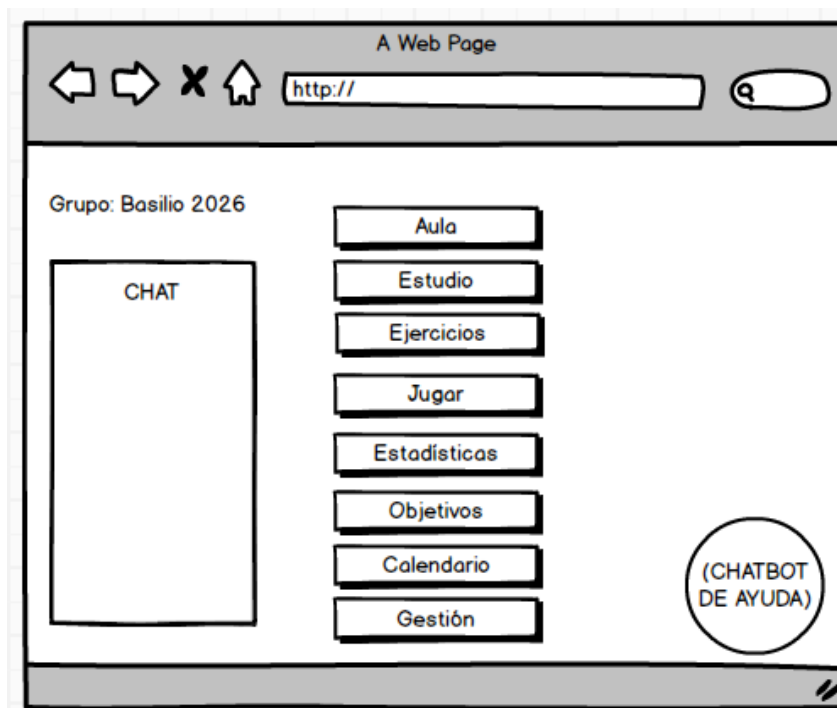


Figura 20: Wireframe menú de navegación de clase

La Figura 21 muestra la interfaz del aula virtual donde hay varios componentes: el tablero de ajedrez interactivo, la planilla de jugadas, la lista de participantes conectados con su estado de presencia, la conexión a la sala de voz, los controles de permisos del profesor, el panel de chat de texto y el módulo de Stockfish.

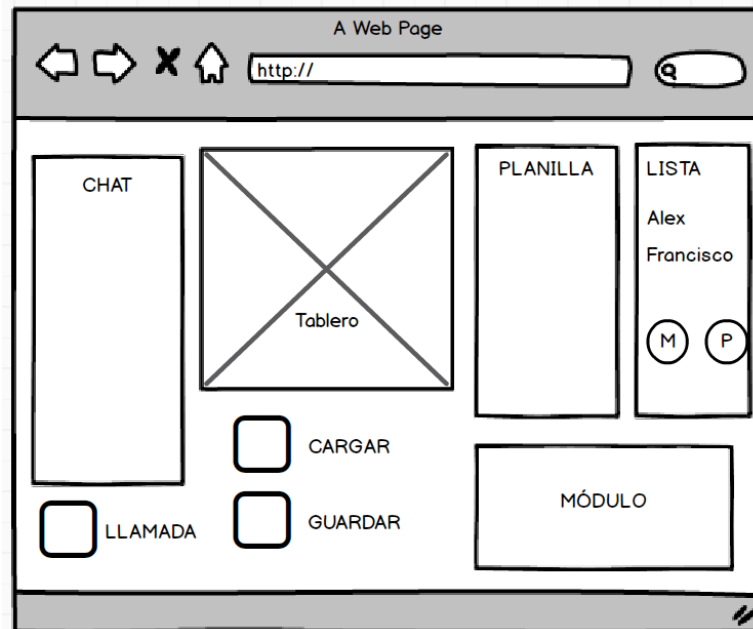


Figura 21: Wireframe del aula virtual

La Figura 22 muestra el explorador de archivos junto a un chat de texto integrado que aparecen en los módulos de estudios y ejercicios.

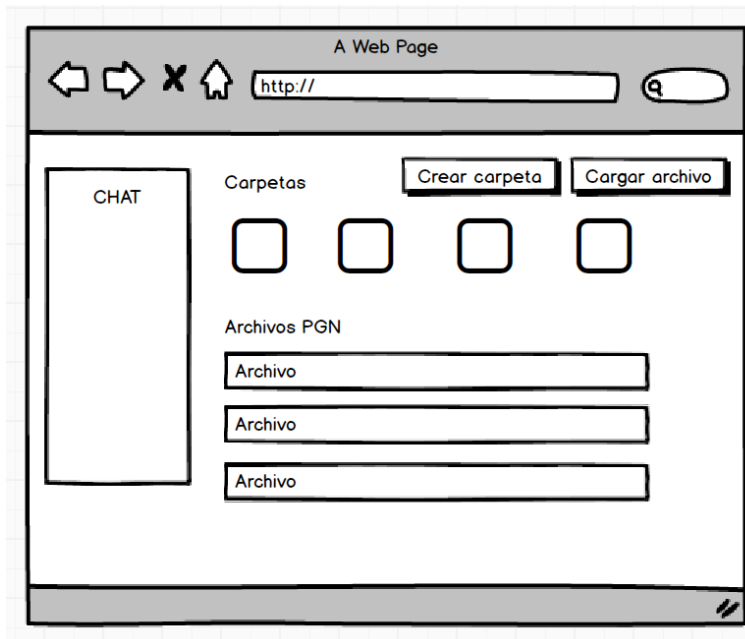


Figura 22: Wireframe explorador de archivos

La Figura 23 muestra el módulo de ejercicios que le carga al alumno la posición inicial del ejercicio en el tablero interactivo junto a la información del estado de la respuesta a la tarea (temporizador y fallos). También se indica la fecha de entrega y la sección de comentario para el profesor. El tablero mostrará un resaltado verde cuando el movimiento es correcto y un resaltado rojo cuando es incorrecto.

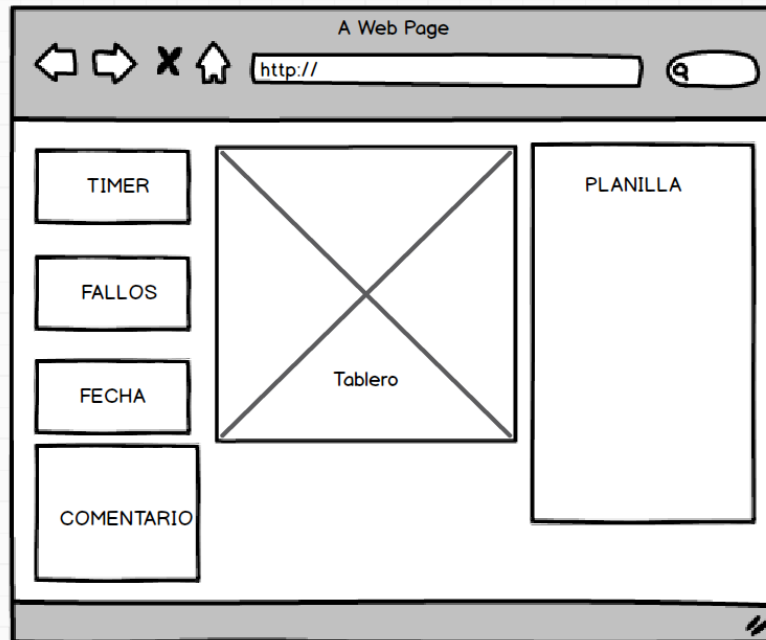


Figura 23: Wireframe de ejercicios para el alumno

La Figura 24 muestra el módulo de torneos que integra un chat general, la tabla de clasificación en tiempo real y un listado partidas activas o finalizadas en el historial.

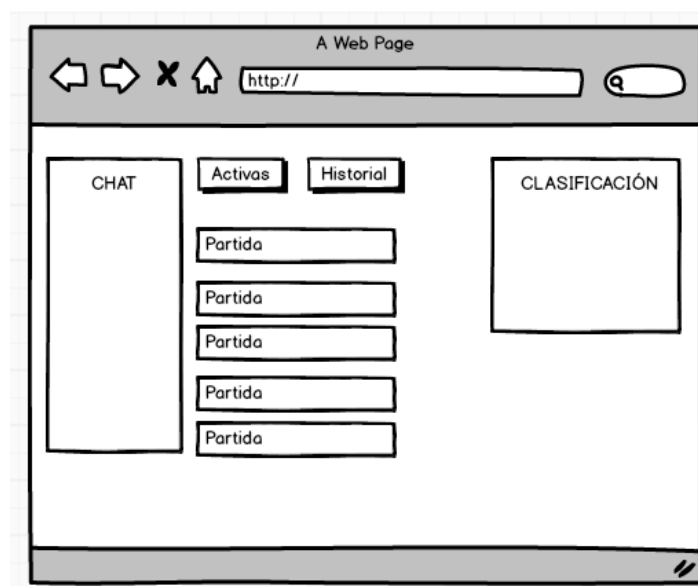


Figura 24: Wireframe de torneo

8.6. Patrones de diseño aplicados

Para el desarrollo de ChessClassroom se han aplicado varios patrones de diseño software. Tanto en el *frontend* como en el *backend*, los patrones han sido aplicados para mejorar mantenibilidad, escalabilidad, claridad del código, etc. A continuación, se describirán algunos de los patrones.

Modelo-Vista-Controlador: como ya se ha explicado en el apartado 8.3, el servidor se divide en tres capas diferentes: rutas, controladores y servicios. Este patrón se usa de forma estándar para el desarrollo de APIs REST gracias a que puede separar la lógica de acceso a los datos almacenados en la base de datos (servicio), como de la lógica información de presentación (controlador) y de la gestión de peticiones HTTP (rutas) (Pressman, 2020).

Lifting state up: aplicado en el *frontend*, es la elevación del estado. Este patrón se basa en transmitir el estado compartido por varios componentes hijos a su componente padre, de esta forma, se convierte en la única fuente de información en la aplicación. Este patrón se aplicó a módulos complejos que integran varios componentes. Las páginas son como contenedores de componentes que centralizan el estado y lo distribuyen a los hijos usando *props* y *callbacks*, de forma que se evita la duplicación del estado entre componentes y evita inconsistencias, como, por ejemplo, la desincronización entre el tablero y la planilla de movimientos.

Hooks: este patrón permite encerrar la lógica compleja en pequeñas unidades reutilizables independientemente de la interfaz. Los *custom hooks* son la forma que tiene React para separar la lógica de negocio de la presentación sin tener que añadir componentes (Wieruch, 2020). Para nuestro caso, se crearon *hooks* como *useChessGame*, *useAulaRealtime*, *useStockfish* o *useGameTimer*, los cuales encierran la lógica que en casos normales estaría en los componentes de cada página.

Inversión de dependencias entre contextos: este patrón permite mantener los contextos desacoplados y ayuda a que sean modificados. En nuestro caso, lo hemos implementado entre *LiveKitContext* y *PresenciaContext*. Se hizo por un sistema de *callbacks* debido a que ambos contextos están en el mismo *layout* y necesitan coordinarse sin conocerse mutuamente. *LiveKitContext* acepta una función *onSalir* que se activa cuando el usuario (profesor o alumno) sale de la sala de voz, esto permite a *PresenciaContext* actualizar el estado de presencia del usuario sin que los dos contextos tengan una referencia al otro.

Caché: hemos implementado un sistema de referencias como caché mediante `useRef` de React, se aplica a situaciones donde es necesario mantener un valor que cambia entre renderizados sin provocar un nuevo ciclo de renderizado. Por ejemplo, tenemos “muteandoYoMismoRef” en el contexto de LiveKit, este es referencia de tipo *bool* que se activa antes de llamar a la función de *mutear* y se desactiva después, cuando llega el evento de *track* silenciado, se comprueba el valor de esta referencia para saber si el silenciado fue causado por el usuario o por el profesor, así se evita una comparación de estados de forma asíncrona que podría causar condiciones de carrera.

Event channel: en los sistemas de tiempo real se aplicó el patrón de canal de eventos, en este, los clientes se suscriben a un canal comunitario y responden solo a los eventos que les corresponden según su rol. Este patrón separa al emisor del receptor y permite añadir nuevos tipos de eventos sin tener que modificar la lógica, lo cual termina facilitando la escalabilidad de los sistemas (Freeman, 2021).

9. Implementación

En este capítulo se documenta la fase de implementación de ChessClassroom. Se describe el proceso de desarrollo de cada módulo de la plataforma, destacando las decisiones técnicas más relevantes, los problemas encontrados durante el desarrollo y las soluciones aplicadas. Los apartados siguen el mismo orden que la propuesta de solución del capítulo 3, comenzando por la infraestructura base de Supabase y finalizando con el despliegue de la aplicación en producción.

9.1. Supabase

Supabase proporciona en una única plataforma los servicios de base de datos, autenticación, almacenamiento de archivos y comunicación en tiempo real. Esta decisión de centralizar estos servicios en Supabase simplificó considerablemente la arquitectura del sistema, evitando la necesidad de configurar y mantener soluciones independientes para cada uno de ellos. A continuación, se describen los aspectos más relevantes de su configuración e integración en el proyecto.

9.1.1. Base de datos

La base de datos de ChessClassroom está implementada con PostgreSQL a través de Supabase. Las tablas de la base de datos se diseñaron tomando en cuenta el modelo entidad-relación explicado en el capítulo 5. A medida que se desarrollaba el proyecto, también se añadían elementos nuevos, ya sean nuevas columnas o tablas.

Una de las decisiones importantes fue el uso de claves UUID como id primaria para las tablas. Esta decisión fue tomada ya que a diferencia de *ints* incrementales, las claves UUID se crean automáticamente sin tener que hacer nada, esto es útil en los módulos de tiempo real donde la latencia de más consultas podría ser notable.

Otra decisión ha sido el añadir la columna “created_at” con tipo TIMESTAMPTZ en algunas tablas, como los mensajes del chat o los movimientos de las partidas. Esto es debido a que el uso de *timestamps* con zona horaria permite que no haya problemas en los datos entre usuarios de distintos lugares del mundo.

9.1.2 Seguridad

La seguridad de la base de datos se gestiona mediante dos sistemas. El primero es la autenticación con JWT el cual asegura que solo los usuarios autenticados pueden realizar peticiones al API del backend. El segundo es el sistema de políticas de seguridad a nivel de fila de PostgreSQL, Row Level Security o RLS que permite crear reglas sobre la BD para limitar las filas que puede leer o modificar cada usuario según su identidad y rol.

En nuestro caso, se activó RLS en las tablas más importantes como podrían ser las de ejercicio o datos, donde un alumno solo puede consultar los ejercicios asignados a su clase y un profesor solo puede acceder a los datos de los alumnos matriculados en sus propias clases.

En conjunto, estos dos sistemas aseguran que, en el caso de una vulnerabilidad en la capa de *backend*, los datos de los usuarios se mantengan protegidos y viceversa.

9.1.3. Manejo de archivos

El almacenamiento de archivos se gestiona mediante Supabase Storage, que ofrece un sistema de almacenamiento. En el proyecto lo hemos utilizado para almacenar los archivos PGN que los profesores suben al módulo de estudios y ejercicios.

Los archivos se guardan en *buckets* (contenedores de almacenamiento con sus propias políticas de acceso). Se creó un *bucket* para los archivos de estudios y ejercicios, con políticas que permiten el acceso solo a los usuarios de la clase. La subida de archivos se hace desde el *frontend* mediante el cliente de Supabase, esto reduce la carga sobre el servidor *backend* y mejora el rendimiento de la operación.

Una limitación de este sistema es el 50 MB como tamaño máximo de archivo del plan gratuito de Supabase Storage. Sin embargo, esto es suficiente para los archivos PGN, que normalmente ocupan kilobytes, salvo en el caso de los PGN database que pueden tener cientos de partidas.

9.1.4 Canales y Realtime

Supabase Realtime permite la comunicación en tiempo real entre clientes mediante WebSockets. Como se explicó en el Capítulo 8, en el proyecto se usa el Broadcast, que permite enviar eventos entre clientes suscritos a un mismo canal sin tener que guardarlos en la BD.

9.2 Página inicial

La página inicial es la primera pantalla que ve cualquier usuario que accede a la web y su objetivo es presentar la plataforma de forma atractiva, listar las funcionalidades principales y guiar al usuario al registro o el inicio de sesión.

Respecto al aspecto técnico, está implementada como un componente de servidor de Next.js, esto significa que se renderiza en el servidor y se envía al navegador como HTML estático. La principal razón es que mejora el tiempo de carga de la página, como extra, también favorece indexación de los motores de búsqueda.

9.3 Módulo de Autenticación

El módulo de autenticación gestiona el registro e inicio de sesión de los usuarios y el control de acceso a las rutas protegidas de la aplicación. El sistema se basa en autenticación mediante JWT, este *token* se genera tras iniciar la sesión, se almacena en el `localStorage` del navegador y se incluye en la cabecera de autorización de cada petición a la API. Este *token* tiene una duración configurada de 7 días, pero que se refresca cada vez que manda el usuario una petición API. El propio *frontend*, al cambiar de página comprueba si el token existe y aún es válido, redirigiendo al usuario al formulario de inicio de sesión en caso contrario.

Para su implementación se eligió por combinar la tabla de autenticación de Supabase con una tabla pública de usuarios. Esto es debido a que Supabase ofrece servicios como el cifrado de contraseñas, pero también se necesita almacenar información adicional del usuario, por lo que se creó una tabla pública cuya clave primaria es el UUID generado por Supabase Auth.

El registro realiza dos operaciones. La creación del usuario en Supabase Auth y la inserción como fila en la tabla pública. Ambas se hacen desde el *backend* en una operación atómica para asegurar que no puedan existir usuarios autenticados sin un perfil asociado en la base de datos.

El inicio de sesión valida las credenciales contra las que se encuentra en Supabase Auth y si son correctas, devuelve al cliente un token JWT junto al perfil completo del usuario. Incluir los datos del perfil en la respuesta evita hacer una petición extra al servidor y hace el proceso sea más fluido.

9.4 Módulo de Gestión de clases

El módulo de gestión de clases es la pantalla principal tras el inicio de sesión. Muestra las clases asociadas al usuario, permite al profesor crear nuevas clases y gestiona el flujo de invitación y unión a clases existentes.

El aspecto más destacable de este módulo es el sistema de invitación. Para evitar que el código se comparta de forma descontrolada y que otras personas puedan unirse, el código de invitación se genera como un *token* de un solo uso con un temporizador activo. De esta forma, se controla que el alumno disponga de un tiempo límite para utilizarlo. Si el alumno se une o el temporizador llega a 0, el *token* queda inutilizable y el sistema genera automáticamente uno nuevo para la siguiente invitación.

9.5 Módulo de Chat

El módulo de chat maneja la comunicación por mensajes de texto en tiempo real dentro de cada clase, las cuales tendrán varias salas de chat que se generan automáticamente al crear la clase, por ejemplo, general, aula virtual, estudios y ejercicios.

La sincronización en tiempo real se desarrolla mediante una suscripción a la tabla de mensajes a través de Supabase Realtime con Postgres Changes y cada vez que se inserta un nuevo mensaje en la base de datos, Supabase envía un evento a todos los clientes suscritos al canal, que hace actualizar el componente del chat de forma inmediata sin tener que realizar ninguna petición al servidor.

Al acceder a una sala, el historial de mensajes previos se carga mediante una petición a la API del *backend*.

El componente diferencia los mensajes propios de los del resto de participantes y tiene incorporado sistema de *auto-scroll* que baja la vista automáticamente hasta el mensaje más reciente cuando llega uno nuevo. Esto está implementado con una referencia al último elemento de la lista de mensajes.

9.6. Módulo de Ajedrez

El módulo de ajedrez es el componente base sobre el cual se construyen muchos otros módulos del proyecto, por lo que lo más importante durante su desarrollo fue diseñarlo para que fuese reutilizable y con la menor cantidad de dependencias posibles.

Para la validación de movimientos se utilizó chess.js, que comprueba en cada jugada si el movimiento es legal, detecta situaciones especiales (jaque, jaque mate y tablas) y mantiene el estado de la partida en formato FEN. La comunicación entre chess.js y el componente de react-chessboard se gestiona a través del estado del componente, de forma que cualquier cambio en la posición FEN del tablero causa una actualización del componente.

Una de las funcionalidades más importantes fue el sistema de resaltado de casillas, ya que este no está incorporado en las librerías. Cuando el usuario arrastra una pieza, el tablero muestra automáticamente los movimientos legales disponibles y resalta la casilla de origen. Cuando el rey está en jaque, su casilla se resalta en rojo y los únicos resaltes disponibles para otras fichas son los movimientos que protegen al rey. Tras cada movimiento, las casillas de origen y destino quedan marcadas.

Dado que react-chessboard promociona automáticamente un peón a reina sin consultar al usuario, se creó un componente modal que intercepta la promoción, muestra las cuatro piezas disponibles (reina, torre, alfil y caballo) y espera la selección antes de aplicar el movimiento.

El módulo incluye también un botón para girar el tablero, útil en el aula virtual y en estudios. También tiene otro botón para reiniciar la posición, que reinicia a la posición inicial estándar o la del archivo cargado según el módulo en el que se esté utilizando. Además, se añadió retroalimentación sonora para las acciones de movimiento, captura, jaque y jaque mate, ya que consideramos que es una parte importante de la experiencia y que transmiten información sin tener que mantener la vista en la pantalla.

Respecto a los problemas encontrados, el más destacable fue el rediseño de react-chessboard en su última versión, donde se cambiaron nombres, funcionalidades y comportamientos de casi toda la librería (React-chessboard, n.d.). Esto ha causado que la mayoría de los tutoriales y guías existentes queden inservibles y ha limitado bastante el alcance del proyecto.

Otro problema importante fue un error de hidratación en Next.js provocado por react-chessboard donde al renderizarse en el servidor durante el proceso de SSR, el tamaño del tablero no podía calcularse correctamente al no existir un DOM real, lo que generaba una diferencia entre el HTML del servidor y el renderizado del cliente que React terminaba detectando como un error. La solución fue implementar un patrón de renderizado solo en el cliente usando un estado “isMounted” que devuelve un esqueleto de carga durante el SSR y renderiza el tablero solo cuando el componente se ha montado en el navegador.

9.7. Módulo de Planilla/Historial. Gestión de la partida: árbol de variantes

La planilla es el componente que muestra el historial de movimientos de una partida y permite al usuario navegar por ellos de forma no lineal. Su desarrollo es uno de los aspectos más complejos del proyecto, ya que debe representar y permitir navegar por un árbol de variantes donde cada nodo puede tener múltiples bifurcaciones.

La decisión de desarrollar este sistema desde cero se debe a que chess.js no tiene soporte para variantes anidadas en formato PGN, por lo que solo es capaz de mantener una única línea principal de la partida.

Para la implementación de la lógica, se diseñó una estructura de datos en forma de árbol de nodos, donde cada nodo almacena la información necesaria para reconstruir la posición del tablero en cualquier momento. Específicamente, se guarda el identificador del nodo, el movimiento en notación SAN, la posición resultante en formato FEN, las casillas de origen y destino, el identificador del nodo padre, la lista de identificadores de los nodos hijos, el número de jugada, el color que realizó el movimiento y un indicador de si hubo captura. El primer hijo de cada nodo representa siempre la línea principal, mientras que los hijos siguientes representan variantes alternativas.

Cuando en un tablero se carga la posición mediante un archivo PGN, se debe construir previamente el árbol a partir de su contenido. Para ello se desarrolló la función “construirArbol”. El proceso comienza eliminando las cabeceras, los comentarios y las anotaciones del PGN (Figura 25) para quedarse solo con los tokens de movimientos y los paréntesis que delimitan las variantes. Luego, se recorre la lista de *tokens* de forma secuencial mediante una pila para gestionar el anidamiento. Cuando se encuentra un paréntesis de apertura, el estado actual del juego se guarda en la pila y se retrocede al nodo padre para explorar una rama alternativa. Cuando se encuentra un paréntesis de cierre, se restaura el estado guardado y se continúa por la línea principal. Este sistema permite gestionar variantes anidadas a cualquier nivel de profundidad. Pseudocódigo de la función “construirArbol” disponible en el Anexo B.

```
[Event "?"]
[Site "?"]
[Date "????..??.??"]
[Round "?"]
[White "?"]
[Black "?"]
[Result "**"]
[Link "https://www.chess.com/analysis"]

1. d4 f5 2. b3 (2. h3 a5 3. f3 Nf6 4. Be3 (4. d5 Ra6 5. b4 Re6 6. Qd4) 4... h6 5. Bf4) 2... g6
```

Figura 25: Ejemplo PGN

Cuando el usuario realiza un movimiento desde el tablero, el sistema comprueba si ya existe un nodo hijo con la misma posición resultante y si es el caso, navega a ese nodo sin duplicar información, en el caso contrario, crea un nuevo nodo y lo añade como hijo del nodo actual. Si el usuario se encuentra en un punto intermedio del historial y realiza un movimiento diferente al de la línea principal, este se añade automáticamente como una nueva variante sin eliminar la línea existente. Pseudocódigo de la aplicación del movimiento sobre el tablero disponible en el Anexo C.

Respecto a la serialización del árbol, este se convierte a formato PGN mediante un recorrido recursivo donde se escribe primero el movimiento de la línea principal y a continuación las variantes entre paréntesis. Esta serialización se utiliza para guardar el estado completo del árbol en la BD. Pseudocódigo de la serialización del árbol a formato PGN disponible en el Anexo D.

Para la implementación de la planilla, el componente recibe el árbol de nodos y lo transforma en una lista plana de filas para su renderizado. Cada fila agrupa el movimiento de blancas y el de negras para ese turno y las variantes se muestran tabuladas con un color que indica el nivel de profundidad. El nodo del estado del tablero se resalta en color azul y el componente auto-scrollea tras cada movimiento. Pseudocódigo de la construcción de filas a partir de un PGN disponible en el Anexo E.

La navegación por el árbol se implementa mediante cuatro operaciones: ir al inicio, retroceder un movimiento, avanzar un movimiento e ir al final. Avanzar siempre sigue la línea principal, mientras que el usuario puede saltar a cualquier variante haciendo clic directamente sobre el movimiento en la planilla.

9.8 Módulo de Estudios

El módulo de estudios es el repositorio de recursos para la clase. Su funcionalidad principal es permitir al profesor organizar, subir y visualizar archivos PGN en un tablero de análisis interactivo, y poner ese material a disposición de los alumnos de forma ordenada.

Se ha implementado un explorador de archivos con soporte para carpetas donde el profesor puede crearlas y nombrarlas libremente para organizar el material. Dentro de cada carpeta pueden coexistir subcarpetas, archivos PGN individuales y bases de datos PGN multi-partida; se clasifica automáticamente entre ambos tipos.

La subida de archivos se hace directamente desde el frontend a Supabase Storage sin pasar por el servidor backend, esto disminuye el tiempo de la operación y evita sobrecargar el servidor. Una vez subido, los metadatos del archivo se registran en la base de datos para que el explorador pueda mostrarlo en la carpeta correspondiente. El profesor puede además ocultar o eliminar cualquier archivo subido desde el propio explorador.

Al seleccionar una partida, el sistema la carga en el tablero interactivo mediante un *hook*, construyendo el árbol de variantes completo a partir del PGN del archivo. Esto permite al usuario navegar libremente por el historial de movimientos, explorar variantes y retroceder a cualquier punto de la partida.

9.9. Módulo de Ejercicios

La implementación del módulo de ejercicios se ha basado en permitir al profesor crear tareas a partir de archivos PGN, asignarlas a sus alumnos con fechas de inicio y entrega, y realizar un seguimiento detallado de sus respuestas. Mientras que, desde el punto de vista del alumno, el módulo presenta los ejercicios disponibles según su estado, permite resolverlos de forma interactiva en el tablero y una vez evaluado por el profesor, ver la revisión.

La implementación de respuestas del ejercicio se realiza mediante un *hook* para gestionar el tablero interactivo, con una lógica de validación que compara cada movimiento del alumno con la secuencia de la solución grabada. Cuando el movimiento es correcto, el tablero muestra un aviso positivo y aplica automáticamente el siguiente movimiento de la solución como respuesta del oponente. Si es incorrecto, el tablero muestra un aviso negativo y el movimiento no se aplica. Durante todo el proceso se registra el tiempo acumulado, el número de intentos fallidos y el PGN de los movimientos realizados. Todos estos elementos quedan guardados en la base de datos junto con un comentario opcional del alumno al completar la entrega.

Una decisión de diseño importante fue el aislamiento entre alumnos dentro de un mismo grupo. Cada alumno únicamente puede ver su propia entrega y no la de sus compañeros, mientras que el profesor tiene acceso centralizado a todas las respuestas. Este aislamiento se implementa mediante las políticas RLS de Supabase, que restringen el acceso a las filas de la tabla de respuestas según el identificador del usuario autenticado.

9.10. Módulo de Aula virtual en tiempo real

El aula virtual es uno de los módulos más complejos del proyecto, ya que integra simultáneamente cuatro sistemas de comunicación en tiempo real: el tablero sincronizado, la sala de voz, el chat de texto y la lista de usuarios conectados.

El tablero sincronizado se implementa mediante el canal de Supabase Realtime descrito en el capítulo 8. Una decisión de implementación importante fue incluir un campo `emisor_id` en cada evento de movimiento con el identificador del usuario que lo generó, de esta forma, cada cliente comprueba si el `emisor_id` coincide con el suyo propio y, en caso positivo, descarta el evento para no aplicar el mismo movimiento dos veces. Esta solución es más robusta que desactivar temporalmente la suscripción al canal, ya que evita momentos tiempo en los que podrían perderse otros eventos.

Los permisos de movimiento de cada alumno se registran en la base de datos para que sobrevivan a desconexiones y reconexiones. Cuando un alumno entra al aula, sus permisos se cargan desde la base de datos y se pasan al componente como permisos, provocando que el alumno recupere exactamente el mismo estado que tenía antes de desconectarse.

La integración de LiveKit para la sala de voz presentó varios problemas. El primero fue la reproducción del audio. El componente oficial de LiveKit no funcionaba porque los eventos de llegada de nuevos *tracks* se producían antes de que el componente se hubiera montado en el DOM. La solución fue gestionar el audio directamente dentro del contexto de LiveKit, añadiendo elementos de audio al documento HTML en el evento de suscripción a un nuevo *track* en vez de depender del ciclo de vida de un componente React.

El control de volumen individual se implementó con el atributo *data-identity* en cada elemento de audio, que tiene el identificador del participante al que pertenece. Para modificar el volumen de un participante concreto, se busca el elemento con ese atributo y se modifica directamente su propiedad *volume*, evitando mantener un estado React extra.

El muteo y desmuteo remoto de participantes necesitó una solución alternativa porque LiveKit no permite desmutear remotamente a un participante desde el cliente. La solución fue enviar un mensaje a través del canal de LiveKit de tipo DESMUTEAR con el identificador del participante seleccionado, de esta forma el cliente receptor ejecuta localmente la operación. El mismo mecanismo se utilizó para la expulsión de la sala de voz, enviando un mensaje de tipo EXPULSAR que hace que el cliente del participante ejecute la desconexión.

Por último, se implementó un widget flotante que aparece cuando el usuario está conectado a la sala de voz y navega fuera del aula. Su posición es arrastrable y se guarda en el almacenamiento local del navegador para que el usuario no tenga que recolocarlo en cada sesión.

9.11. Módulo de Partidas y Torneos

La lógica de validación de movimientos se realiza en el backend. Cuando un jugador envía un movimiento, el servidor reconstruye la posición aplicando el PGN acumulado mediante chess.js y valida el movimiento antes de almacenarlo en la BD. El tiempo restante de cada jugador se calcula mediante una función de BD que calcula el tiempo transcurrido desde el último movimiento y aplica el incremento correspondiente, de esta forma, se asegura que el reloj es siempre coherente con el servidor y no depende del reloj del cliente que podría desincronizarse.

Toda la supervisión de partidas y torneos se centraliza en un *job* que se ejecuta en el servidor cada cinco segundos, comprobando de forma secuencial el tiempo transcurrido de las partidas activas, las condiciones de aborto por inactividad, el inicio y fin automático de torneos por fecha y los emparejamientos pendientes. Este sistema evita depender del cliente para detectar condiciones de fin de partida, lo que podría generar inconsistencias o condiciones de carrera si el jugador pierde la conexión.

El motor de emparejamiento Arena implementa un sistema de *heartbeat* donde se selecciona el primer par de jugadores disponibles que cumplan dos condiciones: haber enviado un ping de presencia en los últimos treinta segundos y no haber jugado entre sí en los últimos dos minutos. Estos dos minutos están implementados para evitar enfrentamientos consecutivos entre los mismos jugadores, que es una de las características principales del formato Arena.

Para la notificación de partidas se desarrollaron componentes widget que, en el caso de una partida amistosa, permiten al jugador aceptar o rechazar la invitación, y en el caso de una partida de torneo, avisan al jugador si está distraído o espectando otra partida.

9.12. Integración de Ejercicios y Torneos al módulo de Calendario

El módulo de calendario fue desarrollado por el colaborador del proyecto. La integración consistió en conectar los ejercicios y torneos creados en sus respectivos módulos para que aparezcan automáticamente como eventos en el calendario de cada clase sin que el profesor tenga que añadirlos manualmente.

La integración se basa primero en obtener los datos de ejercicios y torneos mediante peticiones independientes a la API del *backend* al inicializar la página del calendario. Una vez obtenidos los datos, mediante dos funciones de conversión, se transforman los datos de ejercicios y torneos al formato de evento que utiliza el componente de calendario. El resultado se combina en una lista antes de pasarse al componente *CalendarioMensual* para su renderizado.

Una decisión de diseño importante fue tratar los eventos de torneo y ejercicio como eventos de solo lectura en el calendario, que, a diferencia de los eventos creados manualmente, los torneos y entregas no pueden borrarse desde esta vista, ya que su ciclo de vida se gestiona desde sus propios módulos. Esto se implementa comprobando si el identificador del evento comienza por el prefijo torneo- o ejercicio-.

9.13. Integración de Stockfish en Aula virtual y Partidas

El componente base de Stockfish fue desarrollado por el colaborador del proyecto, quien implementó la integración del motor mediante *WebAssembly* y su encapsulación en un *hook* reutilizable.

La tarea en este apartado se basó en añadir ese componente en los módulos del aula virtual y la revisión de partidas finalizadas.

En el aula virtual, la integración le ofrece al profesor la capacidad de activar el análisis de Stockfish sobre la posición actual del tablero compartido. Las líneas de evaluación y las flechas de variantes sugeridas se pintan en el tablero en tiempo real conforme el motor profundiza en el análisis. Para esta integración se necesitó conectar el estado del tablero sincronizado con el *hook* de Stockfish. Haciendo esto, cada vez que la posición del tablero cambia, el motor recibe automáticamente el nuevo FEN y reinicia el análisis.

En las partidas finalizadas, el motor se activa una vez que la partida haya finalizado, ofreciendo al usuario la posibilidad de analizar cualquier posición del historial. Esta integración fue más fácil, ya que el tablero no está sincronizado en tiempo real y el FEN simplemente se actualiza al navegar por la planilla de movimientos.

9.14.Despliegue

Para el despliegue de la aplicación, el *frontend* está alojado en Vercel, que es la plataforma de despliegue recomendada oficialmente para proyectos Next.js, y el *backend* en Render, que ofrece un soporte sencillo para aplicaciones Node.js sin tener que gestionar la infraestructura de servidores.

El proceso de despliegue está automatizado mediante un pipeline de CI/CD en GitHub. Tanto Vercel como Render están conectados al repositorio del proyecto y se configuraron para desplegar automáticamente cada vez que se hace un *push* a la rama principal.

La gestión de variables de entorno se hizo de forma específica para cada plataforma. En Vercel se configuró la URL de la API del backend y las claves públicas de Supabase. En Render se configuró la clave de servicio de Supabase, la clave secreta JWT y la URL de la base de datos.

Un detalle importante a tener en cuenta es el plan gratuito de Render, el cual provoca que el servidor entre en modo de suspensión tras un período de inactividad. Esto hace que una petición tarde hasta cincuenta segundos en completarse, ya que se está arrancando el servidor *backend*. Esto no afecta al funcionamiento de la aplicación web durante el uso activo, pero puede ser incómodo al acceder a la plataforma tras un tiempo sin actividad.

10. Pruebas

En este capítulo se describe el proceso de pruebas llevado a cabo para verificar el correcto funcionamiento de ChessClassroom. Se presenta en primer lugar la estrategia de pruebas adoptada, seguida de los resultados obtenidos en cada tipo de prueba: pruebas funcionales por módulo, pruebas del árbol de variantes, pruebas de rendimiento, pruebas del sistema en tiempo real y pruebas de validación con usuarios reales. Para cada tipo de prueba se incluyen tablas con casos representativos que muestran el resultado esperado y el resultado obtenido, incluyendo los defectos encontrados y las correcciones aplicadas.

10.1. Estrategia de pruebas

La estrategia de pruebas aplicadas en el proyecto une pruebas unitarias para los componentes más importantes del sistema, pruebas funcionales por módulo, pruebas de rendimiento y una validación final con usuarios reales.

Dado que en el proyecto se aplicó una metodología de desarrollo ágil con *sprints* iterativos, las pruebas se hicieron de forma continua a lo largo del desarrollo, asegurándose de que cada módulo funciona antes de integrarlo con el resto de la aplicación o subirlo a *main*. La documentación de los casos de prueba se elaboró tomando como referencia el estándar IEEE 829 (IEEE, 1998) el cual establece las reglas para la especificación y el registro de pruebas de software.

Para los componentes y funcionalidad más importantes, se diseñaron pruebas unitarias utilizando Jest, por ejemplo, en el *backend* se probó de forma aislada la lógica de validación de movimientos y el motor de emparejamiento de torneos. En el *frontend* se probó especialmente el hook encargado de la lógica del tablero, testeando las funciones de construcción del árbol de variantes, serialización a PGN y navegación entre nodos.

Las pruebas funcionales por módulo se diseñaron siguiendo la técnica de la caja negra, testeando que cada módulo genere las salidas esperadas a partir de las entradas, sin tomar en cuenta la implementación. Este permite comprobar el cumplimiento de los requisitos funcionales especificados en el capítulo 6.

Las pruebas del árbol de variantes se hicieron siguiendo la técnica de la caja blanca para diseñar casos de prueba que cubran cada rama del algoritmo, incluyendo casos límite como variantes anidadas a varios niveles o archivos PGN con formato incorrecto.

Las pruebas de rendimiento se realizaron con Lighthouse. Sobre la página desplegada, evaluando métricas como el tiempo de carga inicial, el *First Contentful Paint*, la puntuación general de rendimiento, etc.

Las pruebas del sistema en tiempo real requieren la interacción de varios usuarios al mismo tiempo. Para simularlo se utilizaron varios navegadores y sesiones en dispositivos distintos, comprobando que los eventos se propagasen correctamente entre todos los clientes conectados. Específicamente, para el *backend* se utilizó Postman, que permite mandar petición a los *endpoints* de la API y para el *frontend* se utilizaron las herramientas de desarrollo de Chrome para inspeccionar el estado de los componentes y los eventos de Supabase Realtime.

Las pruebas de validación con usuarios reales se hicieron mediante sesiones de prueba de la aplicación final desplegada y encuestas de opinión.

Por último, también se hicieron pruebas de regresión al final de cada *sprint* para comprobar que los cambios de un módulo no afectan al comportamiento de los módulos testeados anteriormente.

10.2 Pruebas funcionales de modulo

Las pruebas funcionales se diseñaron siguiendo la técnica de la caja negra. Para cada módulo se definieron casos de prueba que cubren las acciones principales, alternativos y los casos de error más comunes.

Como ejemplo, en la Tabla 5 se presenta los casos de prueba del módulo de ejercicios. Este módulo es uno de los más completos por tener acciones para alumnos y profesores.

Tabla 5: Pruebas del módulo de ejercicios

Identificador	Descripción	Entrada	Resultado esperado	Resultado obtenido
P01	Crear ejercicio con archivo PGN válido	Archivo PGN individual, categoría y fechas de disponibilidad	El ejercicio queda registrado y aparece en el explorador	Superado
P02	Crear ejercicio con archivo PGN inválido	Archivo con extensión incorrecta	El sistema muestra un mensaje de error y no almacena el archivo	Fallido: el sistema aceptaba el archivo sin validar la extensión. Corregido añadiendo validación en el <i>frontend</i>
P03	Grabar secuencia de solución	Secuencia de movimientos válida sobre el tablero	La solución queda almacenada y asociada al ejercicio	Superado

P04	Asignar ejercicio a un alumno con fecha futura	Fecha de inicio posterior a la fecha actual	El ejercicio no es visible para el alumno hasta la fecha de inicio	Superado
P05	Resolver ejercicio con movimiento correcto	Movimiento que coincide con la solución pregrabada	Retroalimentación visual positiva y aplicación del siguiente movimiento de la solución	Superado
P06	Resolver ejercicio con movimiento incorrecto	Movimiento que no coincide con la solución	Retroalimentación visual negativa e incremento del contador de fallos	Fallido: el contador de fallos no se incrementaba en el primer intento. Corregido inicializando correctamente el estado del contador
P07	Entregar ejercicio completado	Todos los movimientos de la solución completados correctamente	El ejercicio se marca como completado y el profesor puede consultarlo	Superado
P08	Consultar entrega como profesor	Acceso a la vista de entregas del ejercicio	El sistema muestra el tiempo, los fallos y el PGN jugado por el alumno	Superado
P09	Evaluar entrega con puntuación y comentario	Puntuación del 1 al 5 y comentario de texto	La evaluación queda registrada y el alumno puede consultarla	Superado
P10	Acceder a ejercicio de otro alumno	Alumno intenta acceder a la entrega de un compañero	El sistema deniega el acceso y no muestra la entrega	Superado

10.3. Pruebas del árbol de variantes

Las pruebas del árbol de variantes se diseñaron siguiendo la técnica de la caja blanca. El objetivo fue verificar que la función `construirArbol` genere correctamente la estructura de nodos en todos los casos, y que las operaciones de navegación, inserción de variantes y serialización produzcan resultados correctos. En la Tabla 6 se pueden observar los casos de prueba.

Tabla 6: Pruebas del árbol de variantes

Identificador	Descripción	Entrada	Resultado esperado	Resultado obtenido
P01	Construir árbol desde PGN sin variantes	Archivo PGN con línea principal únicamente	Árbol con un único camino lineal de nodos	Superado
P02	Construir árbol desde PGN con una variante	Archivo PGN con un par de paréntesis	Árbol con nodo raíz con dos hijos en el punto de bifurcación	Superado
P03	Construir árbol con variantes anidadas	Archivo PGN con variantes dentro de variantes	Árbol con bifurcaciones a varios niveles de profundidad	Fallido: las variantes de segundo nivel no se construían correctamente al restaurar el estado de la pila. Corregido ajustando la lógica de restauración del estado
P04	Construir árbol desde FEN personalizado	Cadena FEN válida como entrada	Árbol con nodo raíz cuyo FEN coincide con la posición indicada	Superado
P05	Ignorar comentarios y anotaciones NAG	PGN con bloques entre llaves y símbolos \$	Árbol construido correctamente sin incluir los comentarios como nodos	Superado

P06	Cargar PGN con formato parcialmente incorrecto	PGN con un movimiento inválido intercalado	El <i>parser</i> omite el movimiento inválido y construye el árbol con los movimientos válidos	Superado
P07	Insertar movimiento existente en el árbol	Movimiento que ya existe como hijo del nodo actual	El sistema navega al nodo existente sin crear un duplicado	Superado
P08	Insertar variante desde punto intermedio	Movimiento diferente al de la línea principal desde un nodo con hijos	Se crea un nuevo nodo hijo manteniendo la línea principal intacta	Fallido: al insertar la variante se sobrescribía el primer hijo. Corregido añadiendo el nuevo nodo al final del <i>array</i> de hijos en lugar de reemplazar el primero
P09	Serializar árbol sin variantes a PGN	Árbol con línea principal únicamente	PGN lineal sin paréntesis que reproduce los mismos movimientos	Superado
P10	Ciclo completo carga-modificación-guardado	PGN con variantes, inserción de nuevo movimiento y serialización	El PGN generado cargado de nuevo produce el mismo árbol	Superado

10.4. Pruebas de rendimiento

Las pruebas de rendimiento se hicieron utilizando Lighthouse en la aplicación desplegada. Se ejecutaron pruebas que evalúan cuatro características: rendimiento, accesibilidad, buenas prácticas y SEO. En la Tabla 7 se muestran los resultados obtenidos de algunas páginas y módulos.

Tabla 7: Resultados de pruebas con Lighthouse

Módulo	Rendimiento	Accesibilidad	Buenas prácticas	SEO
Página de inicio	87	91	95	89
Menú de clases	82	88	92	85
Módulo de estudios	74	85	90	82
Módulo de aula virtual	71	83	90	80

En general, los resultados son buenos (Google, 2023), sin embargo, las páginas con más componentes y funcionalidades, como el módulo de estudios y el aula virtual, tienen puntuaciones de rendimiento algo menores.

Respecto a los resultados de tiempo de carga, los más relevantes fueron el *First Contentful Paint*, que ronda entre 1.2 y 1.8 segundos según la página, y el *Largest Contentful Paint*, que llega hasta 2.4 segundos en el aula virtual por el tiempo de inicialización y sincronización del componente del tablero.

10.5. Pruebas del sistema en tiempo real

En las pruebas de tiempo real, se simuló utilizando varios navegadores abiertos en paralelo, en varios dispositivos y con las sesiones de usuarios diferentes en el mismo aula y torneo. En la Tabla 8 se muestran las pruebas para el canal de Supabase Realtime (sincronización del tablero), la sala de voz de LiveKit y el módulo de partidas y torneos en tiempo real.

Tabla 8: Pruebas del sistema en tiempo real

Identificador	Descripción	Escenario	Resultado esperado	Resultado obtenido
P01	Sincronización de movimiento en el aula	Profesor mueve una pieza con dos alumnos conectados	El movimiento aparece en el tablero de ambos alumnos en menos de 2s	Superado
P02	Persistencia de permisos tras reconexión	Alumno con permiso de mover se desconecta y reconecta	El alumno recupera sus permisos al volver a entrar	Fallido: el alumno perdía los permisos al reconectarse. Corregido cargando los permisos desde la base de datos al inicializar el aula
P03	Sistema de presencia con múltiples usuarios	Tres usuarios conectados al aula, uno se desconecta	La lista de participantes se actualiza correctamente en menos de 30 segundos	Superado
P04	Audio entre participantes en sala de voz	Dos usuarios conectados a la sala de voz	Ambos se escuchan mutuamente sin eco ni cortes perceptibles	Superado
P05	Silenciado remoto por el profesor	Profesor silencia a un alumno desde el panel de moderación	El micrófono del alumno se desactiva y el botón queda deshabilitado	Superado
P06	Nuevo participante en sala de voz con usuario ensordecido	Usuario ensordecido, nuevo participante se une	El nuevo participante no se escucha hasta que el usuario desactiva el ensordecimiento	Fallido: el nuevo <i>track</i> se reproduce, aunque el usuario estuviera ensordecido. Corregido controlando el volumen mediante la propiedad <i>volume</i> del elemento HTML

P07	Sincronización del reloj en partida	Dos jugadores realizan movimientos alternos durante dos minutos	El tiempo restante de ambos jugadores es coherente entre las dos sesiones	Superado
P08	Emparejamiento automático en torneo Arena	Dos jugadores libres con ping activo en el torneo	El sistema genera una nueva partida en menos de 10 segundos	Superado
P09	Aborto por inactividad en torneo	Jugador no realiza su primer movimiento en 30 segundos	La partida se aborta y el jugador recibe una derrota	Superado
P10	Fin de torneo por fecha	Fecha de fin del torneo alcanzada con partidas en curso	El torneo pasa a estado finalizado y las partidas en curso continúan hasta su conclusión	Superado

10.6. Pruebas de validación con usuarios

Para las pruebas de validación con usuarios se diseñó una encuesta estructurada en tres bloques y una sección de comentarios libres donde para cada pregunta se valoró mediante una escala del uno al cinco, en la cual uno corresponde a muy en desacuerdo y cinco a muy de acuerdo. Encuesta disponible en el Anexo F.

En estas pruebas participaron veinte personas: cinco profesores/entrenadores de ajedrez, diez alumnos/practicantes habituales y cinco personas sin relación con el ajedrez. Dieciséis de los veinte participantes habían utilizado Lichess o Chess.com.

Para la encuesta se utilizó la versión final del TFG desplegado en producción y pidiendo a los participantes que exploren la aplicación libremente antes de responder.

En la Figura 26 se muestran los datos obtenidos para el bloque de valoración general.

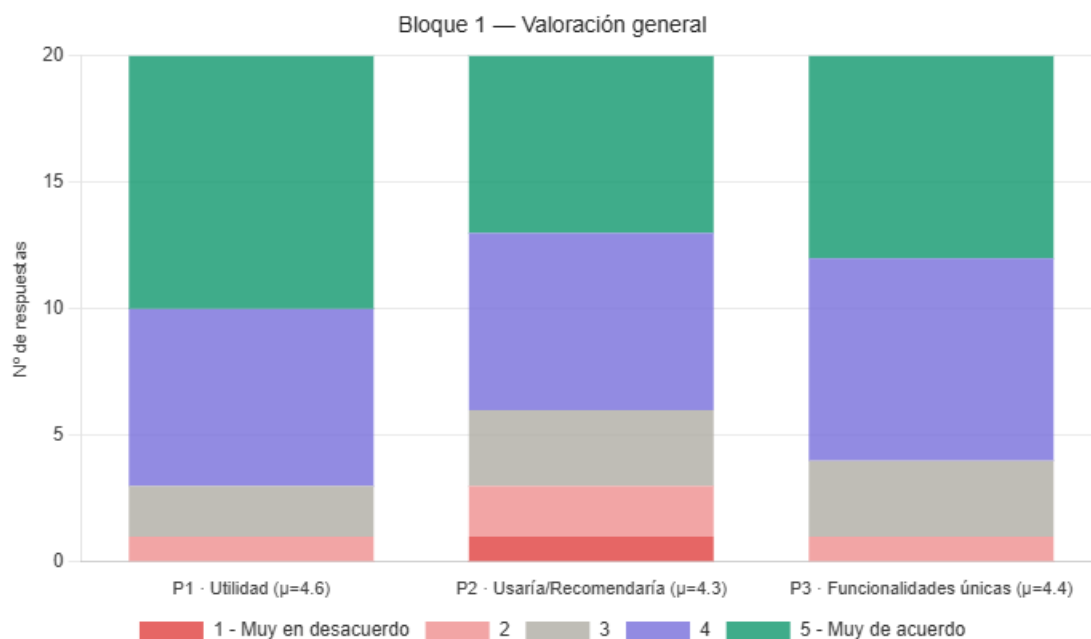


Figura 26: Distribución de respuestas para la valoración general

Los resultados de la valoración general son muy positivos.

- La pregunta sobre la utilidad de ChessClassroom tiene una media más alta con un 4.6/5, siendo bien valorada por los profesores y alumnos de ajedrez.
- La pregunta sobre si utilizarían o recomendarían la plataforma tiene una media de 4.3/5 con puntuaciones más bajas por parte de los usuarios sin relación al ajedrez.
- La pregunta sobre si ofrece funcionalidades no disponibles en otras plataformas tiene una media de 4.4/5, siendo valorado muy positivamente por los profesores.

En la Figura 27 se muestran los datos obtenidos para el bloque de funcionalidades concretas.

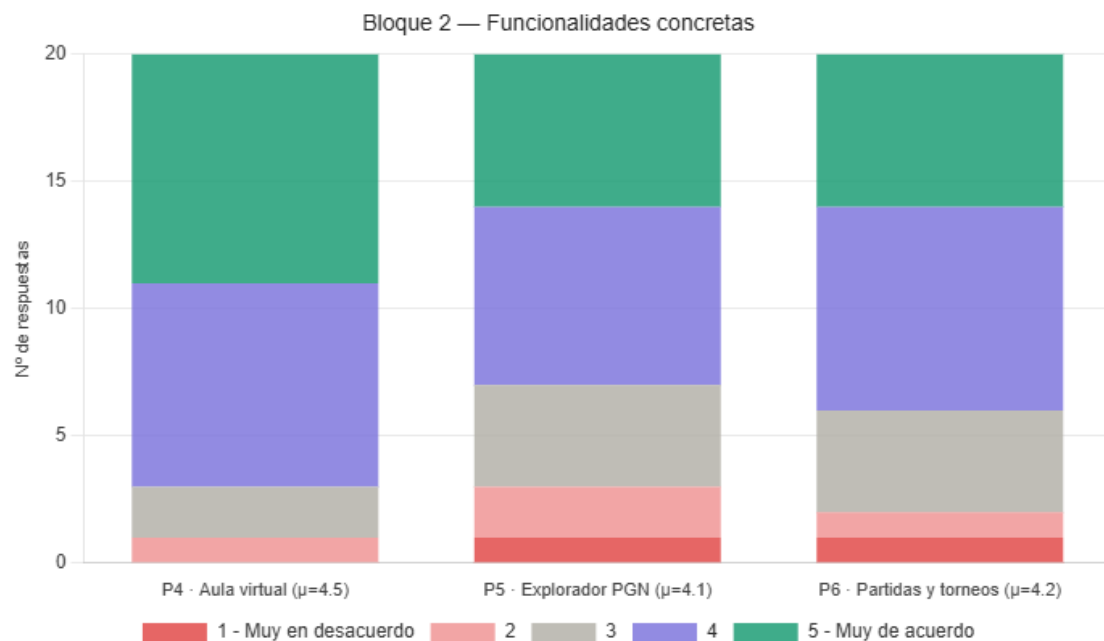


Figura 27: Distribución de respuestas para las funcionalidades

Los resultados de la valoración de funcionalidades son mayormente positivos debido a que los usuarios sin relación con el ajedrez tienen menos contexto para valorar estas preguntas.

- El aula virtual con tablero sincronizado y sala de voz tiene una media de 4.5/5 siendo valorado muy positivamente por los profesores, que la destacaron como la funcionalidad más importante comparado con otras plataformas.
- El explorador de archivos PGN tiene una media de 4.1/5, con valoraciones más bajas por parte de usuario alumno y los que no juegan ajedrez, ya que no suelen trabajar con estos archivos.
- El sistema de partidas y torneos Arena tiene una media de 4.2/5, siendo bien valorado por profesores y alumnos, pero no destaca por encima de las principales plataformas *online*.

En la Figura 28 se muestran los datos obtenidos para el bloque de usabilidad.

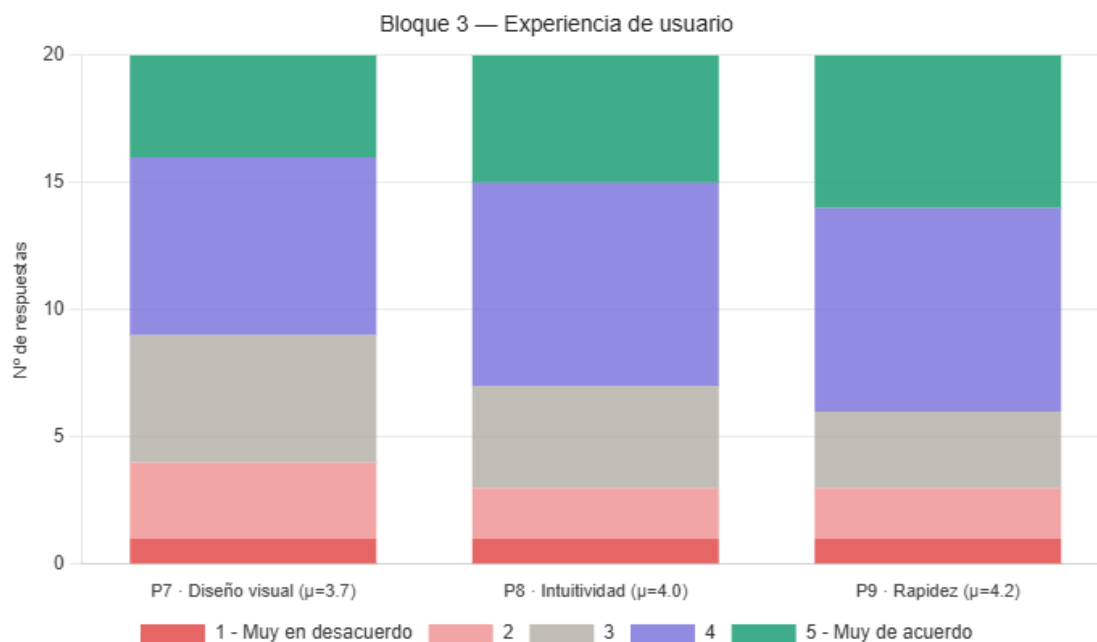


Figura 28: Distribución de respuestas para la usabilidad

Los resultados de la experiencia de usuario son positivos y son los más similares entre los diferentes tipos de usuarios.

- El diseño visual de la plataforma tiene una media de 3.7/5, ha sido pregunta peor valorada, lo cual indica que hay mucho margen de mejora en el diseño de la interfaz.
- La intuitividad de la interfaz tiene una media de 4.0/5, varios usuarios indicaron en los comentarios que algunos módulos como el torneo o ejercicios necesitan una pequeña curva de aprendizaje.
- La rapidez de respuesta de la plataforma tiene una media de 4.2/5, con alguna valoración baja por el retardo de la inicialización del servidor *backend* en Render.

En los comentarios libres se mencionaron que las mejores funcionalidades son el aula virtual y el sistema de ejercicios con corrección automática. Entre las sugerencias de mejora aparecen principalmente mejoras de interfaz, uso de la plataforma desde el móvil con mayor comodidad, más opciones de personalización del perfil y alguna integración con otras plataformas para importar partidas directamente desde Lichess o Chess.com.

En términos generales, los resultados de la encuesta demuestran que ChessClassroom cubre una necesidad real en la enseñanza del ajedrez y que sus funcionalidades principales son útiles y adecuadas para el público objetivo. La media global de todas las preguntas es un 4.17/5, con la interfaz como principal área de mejora.

11. Conclusiones y futuras líneas de trabajo

En este capítulo se presentan las conclusiones del proyecto. En primer lugar, se delimita el alcance final de la plataforma y se identifican sus limitaciones actuales. A continuación, se evalúa el grado de cumplimiento de los objetivos definidos al inicio del proyecto. Tras ello, se exponen las conclusiones generales extraídas del desarrollo y la validación. El capítulo finaliza con las futuras líneas de trabajo identificadas durante el proceso y las aportaciones personales del autor.

11.1. Alcance y limitaciones del proyecto

El alcance del proyecto es el diseño e implementación de una plataforma web de enseñanza de ajedrez orientada a profesores, con todos los módulos descritos en la propuesta de solución disponibles en la versión desplegada en producción. La plataforma está creada para un entorno de escritorio, aunque es accesible desde dispositivos móviles.

Respecto a las limitaciones, la plataforma presenta algunas restricciones que conviene mencionar. El servidor *backend* está desplegado en el plan gratuito de Render, lo que puede provocar tiempos de inicialización de varios segundos tras períodos de inactividad prolongada. Los archivos almacenados en la BD están condicionados por los límites del plan gratuito de Supabase Storage, establecidos a 50 MB por archivo. Por último, no existe un panel de administración global para gestionar la plataforma a nivel de administrador del sistema.

Estas limitaciones no afectan al funcionamiento de las funcionalidades principales de la plataforma ni a su validez como sistema educativo.

11.2. Cumplimiento de los objetivos

El objetivo de diseñar e implementar una aplicación web completa con arquitectura cliente-servidor desacoplada se ha cumplido. El proyecto tiene un *frontend* en Next.js desplegado en Vercel y un *backend* en Node.js con Express desplegado en Render, comunicados mediante una API REST y Supabase como infraestructura de base de datos, autenticación y tiempo real.

El objetivo de ofrecer al profesor herramientas para la enseñanza se ha cumplido. La plataforma permite crear y gestionar clases particulares y grupales, organizar material educativo, asignar y corregir ejercicios con fechas de entrega, etc.

El objetivo de implementar funcionalidades de comunicación y enseñanza en tiempo real se ha cumplido. El aula virtual implementa un tablero de ajedrez sincronizado entre todos los participantes, comunicación por voz mediante LiveKit, chat de texto en tiempo real mediante Supabase Realtime y controles de moderación para el profesor.

El objetivo de ofrecer herramientas específicas de ajedrez se ha cumplido. La plataforma implementa un tablero interactivo reutilizable con posibilidad de ejecutar variantes, el motor de análisis Stockfish integrado mediante WebAssembly, un módulo de ejercicios con corrección automática y un sistema de torneos en formato Arena con emparejamiento automático.

El objetivo de ofrecer una experiencia de usuario intuitiva y accesible se ha cumplido parcialmente. Los resultados de la encuesta de validación indican una valoración media de 4.0/5 para la usabilidad, sin embargo, el diseño visual es mejorable.

El objetivo de integrar las funcionalidades de la plataforma con un calendario se ha cumplido. Los ejercicios y torneos creados en los módulos aparecen automáticamente en el calendario de la clase y el calendario general.

El objetivo de validar el sistema mediante pruebas funcionales y con usuarios reales se ha cumplido. Se diseñaron y ejecutaron casos de prueba funcionales, pruebas del árbol de variantes, pruebas del sistema en tiempo real, pruebas de rendimiento y varias sesiones de prueba de validación con veinte usuarios.

11.3. Conclusiones

El desarrollo de ChessClassroom ha creado una plataforma web docente de ajedrez funcional y desplegada en producción que cubre la ausencia de una herramienta diseñada para el profesorado de ajedrez que integra en un único entorno la gestión de clases, la enseñanza online y el seguimiento del progreso del alumno.

Desde la perspectiva del desarrollador, el proyecto ha sido un gran reto por la cantidad de tecnologías y herramientas integradas, además de la complejidad de algunos módulos, como el árbol de variantes con soporte para PGN anidado, la integración de LiveKit para la sala de voz con todos los problemas del ciclo de vida de los componentes, el diseño del *job* para la gestión de partidas y torneos en el *backend*, etc. Todos ellos son ejemplos de problemas técnicos que han necesitado de soluciones.

El proyecto también ha demostrado que es posible construir una aplicación web completa y escalable únicamente con servicios *cloud* con planes gratuitos, esto lo convierte en un punto de comienzo para determinar si la idea es realmente viable.

Los resultados de la validación con usuarios han sido generalmente positivos, con una valoración media global de 4.17/5 y han confirmado que las funcionalidades principales de la plataforma cumplen sus necesidades.

En conclusión, ChessClassroom es una plataforma funcional, técnicamente apropiada y validada por usuarios reales, que demuestra la viabilidad de crear una plataforma educativa especializada en ajedrez que, agrupando diversas funcionalidades, termina superando a las herramientas actualmente disponibles en el mercado.

11.4. Aportaciones personales

El desarrollo de ChessClassroom ha sido una de las experiencias más enriquecedoras de mi etapa universitaria, tanto desde el punto de vista técnico como personal.

Fácilmente puedo destacar el hecho de aprender a usar nuevas herramientas y tecnologías, como Next.js y su sistema de App Router, Supabase como plataforma de *backend*, LiveKit para la comunicación de audio en tiempo real basada en el protocolo WebRTC y todo el proceso de despliegue en la nube con Vercel y Render. Aprender a usar e integrar estas herramientas un único proyecto ha sido un proceso complicado y que me ha obligado a leer documentación, resolver problemas y tomar decisiones de diseño con criterio propio.

El proyecto también ha sido una prueba de los conocimientos que he adquirido durante la carrera y mi experiencia en prácticas. Conceptos como la especificación de requisitos, el diseño de la arquitectura, los patrones de diseño, las metodologías de trabajo, el diseño de bases de datos o el proceso de pruebas, son ideas que había estudiado de forma teórica o aplicado en contextos donde recibía mucha ayuda externa, pero en este proyecto he tenido que aplicarlos de forma autónoma.

Trabajar en equipo con un colaborador también ha sido muy provechoso. Gracias a Alejandro Gil López, he podido aprender mucho sobre los aspectos académicos y profesionales que no he estudiado, como el diseño de soluciones, desarrollo de una idea de producto, gestión de proyectos, etc. Del mismo modo, he podido compartir mis conocimientos técnicos sobre el desarrollo software, selección de herramientas o estimación de complejidad de funcionalidades. Más allá de lo académico y técnico, esta colaboración ha desarrollado una cantidad innumerable de competencias transversales, como la comunicación, la organización, la toma de decisiones en equipo, entre muchas otras.

Por último, el proyecto me ha introducido en un ámbito completamente nuevo: el mundo del ajedrez y su docencia. Aunque ocasionalmente haya jugado al ajedrez, el proceso de entender sus necesidades educativas, hablar con profesores y alumnos, analizar las herramientas existentes y diseñar una solución adaptada a sus necesidades me ha abierto los ojos, demostrando que a veces, la ingeniería del software no solo consiste en escribir código, sino también en entender profundamente el problema que se quiere resolver y a las personas involucradas.

11.5. Futuras líneas de trabajo

A lo largo del desarrollo del proyecto se identificaron funcionalidades y mejoras que, por limitaciones de tiempo o alcance, quedaron fuera de la versión final del proyecto.

La mejora más solicitada es el diseño visual de la plataforma, la cual requiere un nuevo estilo para mejorar la coherencia entre módulos, la paleta de colores y la tipografía.

La adaptación a dispositivos móviles es otra de las mejoras importantes. Aunque actualmente la plataforma es accesible, la experiencia en pantallas pequeñas es mejorable, especialmente en los módulos que tengan que interactuar con el tablero de ajedrez. Se debe mejorar el reescalado o desarrollar una aplicación móvil nativa, aprovechando la arquitectura desacoplada del *backend*.

Los torneos con formato suizo quedaron como *backlog* para priorizar el formato Arena. Actualmente, el formato suizo es el más popular en competiciones de ajedrez, sin embargo, es más complejo y habría que tomar en cuenta el nivel individual de cada jugador.

La integración con Lichess para importar automáticamente las partidas ayudaría al profesor a migrar el material educativo sin tener que exportar e importar archivos PGN manualmente.

La grabación de sesiones de clase del aula virtual permitiría al alumno revisar las clases a las que no pudo asistir o repasar el contenido de sesiones anteriores.

La migración a planes de pago en Render y Supabase eliminarían los tiempos de inicialización del servidor y límites actuales de almacenamiento, mejorando la experiencia de uso en situaciones reales de producción con cientos o miles de usuarios al mismo tiempo. A más largo plazo, habría que desvincularse de estas plataformas y realizar la gestión en servidores y bases de datos propios.

Por último, un clasificador táctico automático basado en inteligencia artificial, podría analizar las posiciones de los ejercicios y las etiquetaría automáticamente por temática (ataque doble, clavada, sacrificios, etc.). Es una funcionalidad muy útil que permitiría la creación automática de ejercicios basados en las partidas jugadas por los usuarios. Sin embargo, su implementación requiere de un trabajo de investigación y desarrollo específico que supera el alcance de este TFG.

Referencias bibliográficas

- Balsamiq. (s.f.). *Balsamiq Wireframes*. Obtenido de Balsamiq Wireframes: <https://balsamiq.com/product/wireframes/>
- Bart, W. M. (2014). *On the effect of chess training on scholastic achievement*. *Frontiers in Psychology*.
- Bernal Martí, H. (2025). *Adaptación de modelos de aprendizaje profundo para ajedrez*. Universitat Politècnica de València.
- Bower, M. D. (2015). Design and implementation factors in blended synchronous learning environments. *Computers and Education* 86, 1–17.
- Campbell, M. H. (2002). Deep Blue. *Artificial Intelligence*, 134(1–2), 57–83.
- Chacon, S. y. (2014). *Pro Git*. Apress.
- Chess.com. (s.f.). *Chess.com*. Obtenido de Chess.com: <https://www.chess.com/>
- Chessable. (s.f.). *Chessable*. Obtenido de Chessable: <https://www.chessable.com/es>
- ChessKid. (s.f.). *ChessKid*. Obtenido de ChessKid: <https://www.chesskid.com/>
- CircleChess. (s.f.). *CircleChess*. Obtenido de CircleChess: <https://circlechess.com/>
- Clariity. (s.f.). *React-chessboard*. Obtenido de React-chessboard: <https://www.npmjs.com/package/react-chessboard>
- Cooper, A. R. (2007). *About Face 3: The Essentials of Interaction Design*. Wiley Publishing.
- Discord. (s.f.). *Discord*. Obtenido de Discord: <https://discord.com/>
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Excalidraw. (s.f.). *Excalidraw*. Obtenido de Excalidraw: <https://excalidraw.com/>
- Express.js. (s.f.). *Express.js*. Obtenido de Express.js: <https://expressjs.com/en/>
- Fette, I. y. (2011). *The WebSocket Protocol. RFC 6455*. Internet Engineering Task Force.
- FIDE. (2023). *Annual Report 2023*. Fédération Internationale des Échecs.
- FIDE. (2023). *Handbook: Laws of Chess*. Fédération Internationale des Échecs. Obtenido de <https://www.fide.com/FIDE/handbook/LawsOfChess.pdf>
- Freeman, E. y. (2021). *Head First Design Patterns*. O'Reilly Media.
- Git. (s.f.). *Git*. Obtenido de Git: <https://git-scm.com/>
- GitHub. (s.f.). *GitHub*. Obtenido de GitHub: <https://github.com/>



- Google. (2023). *Lighthouse performance scoring*. Obtenido de Lighthouse performance scoring: <https://developer.chrome.com/docs/lighthouse/performance/performance-scoring>
- Google. (s.f.). *Google Classroom*. Obtenido de Google Classroom: <https://classroom.google.com/>
- Google. (s.f.). *Lighthouse*. Obtenido de Lighthouse: <https://developer.chrome.com/docs/lighthouse?hl=es-419>
- Grigorik, I. (2013). *High Performance Browser Networking*. O'Reilly Media.
- Grok. (s.f.). *Grok*. Obtenido de Grok: <https://grok.com/imagine>
- IEEE. (1998). *IEEE Std 829-1998: IEEE Standard for Software Test Documentation*. IEEE.
- IEEE. (s.f.). *IEEE Std 830: Recommended Practice for Software Requirements Specifications*.
- ISO. (2011). *ISO 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.
- Jennings, C. H. (2013). Real-time communications for the web. *IEEE Communications Magazine*, 51(4), 20–26.
- Jest.js. (s.f.). *Jest.js*. Obtenido de Jest.js: <https://jestjs.io/>
- jhlywa. (s.f.). *Chess.js*. Obtenido de Chess.js: <https://jhlywa.github.io/chess.js/>
- Jiménez Rivera, D. (2026). *Sistema web de inventario y préstamo bibliotecario de la Federación de Ajedrez de la Comunidad Valenciana*. Universitat Politècnica de València.
- Knuth, D. E. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence* 6(4), 293–326.
- Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley.
- Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. Prentice Hall.
- Lichess. (s.f.). *Lichess*. Obtenido de Lichess: <https://lichess.org/>
- LiveKit. (s.f.). *LiveKit*. Obtenido de LiveKit: <https://livekit.com/>
- LucidChart. (s.f.). *LucidChart*. Obtenido de LucidChart: <https://www.lucidchart.com/pages/es>
- Mauricio, J. (2017). Chess in schools: The Armenian experience and its international impact. *Journal of Chess Education* 3(1), 12–19.

- Microsoft. (s.f.). *Microsoft Teams*. Obtenido de Microsoft Teams: <http://microsoft.com/teams>
- Microsoft. (s.f.). *Visual Studio Code*. Obtenido de Visual Studio Code: <https://code.visualstudio.com/>
- Moodle. (s.f.). *Moodle*. Obtenido de Moodle: <https://moodle.org/>
- Murray, H. J. (1913). *A History of Chess*. Oxford University Press.
- Nasu, Y. (2018). *Efficiently updatable neural network based evaluation functions for computer shogi*. The 28th World Computer Shogi Championship Appeal Document.
- Navarro López, R. (2006). *Programación en Java de un gestor de torneos de ajedrez*. Universitat Politècnica de València.
- Nielsen, J. (1993). *Usability Engineering*. Morgan Kaufmann.
- Node.js. (s.f.). *Node.js*. Obtenido de Node.js: <https://nodejs.org/es>
- Object Management Group. (2017). *Unified Modeling Language Specification, version 2.5.1*. Obtenido de OMG: <https://www.omg.org/spec/UML/2.5.1>
- OnlyChessAcademy. (s.f.). *OnlyChessAcademy*. Obtenido de OnlyChessAcademy: <https://www.onlychessacademy.com/>
- Postman. (s.f.). *Postman*. Obtenido de Postman: <https://www.postman.com/>
- Pressman, R. S. (2020). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
- React. (s.f.). *React*. Obtenido de React: <https://es.react.dev/>
- React-chessboard. (s.f.). *Upgrading to V5*. Obtenido de React-chessboard: <https://react-chessboard.vercel.app/?path=/docs/upgrade-guides-upgrading-to-v5--docs>
- Render. (s.f.). *Render*. Obtenido de Render: <https://render.com/>
- Sala, G. y. (2016). Do the benefits of chess instruction transfer to academic and cognitive skills? A meta-analysis. *Educational Research Review*, 18, 46–57.
- Schwaber, K. y. (2020). *La Guía de Scrum*. Scrum.org.
- Silver, D. H. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* 362(6419), 1140–1144.
- Skarin, H. K. (2010). *Kanban y Scrum – Obteniendo lo mejor de ambos*. C4Media Inc.
- Sommerville, I. (2016). *Software Engineering*. Pearson.
- Statista. (2023). *Number of registered users of Chess.com worldwide*. Statista Research Department. Statista.

- Stockfishchess. (s.f.). *Stockfishchess*. Obtenido de Stockfishchess:
<https://stockfishchess.org/>
- Supabase. (s.f.). *Supabase*. Obtenido de Supabase: <https://supabase.com/>
- tailwindcss. (s.f.). *Tailwindcss*. Obtenido de Tailwindcss: <https://tailwindcss.com/>
- Tarazona Franco, J. E. (2001). *Desarrollo de un motor de ajedrez*. Universitat Politècnica de València.
- Trello. (s.f.). *Trello*. Obtenido de Trello: <https://trello.com/>
- Trinchero, R. (2013). *Can chess training improve Pisa scores in mathematics?* Kasparov Chess Foundation Europe.
- Turing, A. M. (1950). Computing machinery and intelligence. *Mind*, 59(236), 433–460.
- TypeScript . (s.f.). *TypeScript* . Obtenido de TypeScript :
<https://www.typescriptlang.org/>
- Vanaclocha Llorens, J. L. (1999). *Implementación de un juego de ajedrez con capacidad de aprendizaje basado en técnicas de identificación de patrones*. Universitat Politècnica de València.
- Vercel. (s.f.). *Next.js*. Obtenido de Next.js: <https://nextjs.org/>
- Vercel. (s.f.). *Vercel*. Obtenido de Vercel: <https://vercel.com/>
- Wieruch, R. (2020). *The Road to React*. Leanpub.

Anexos

Anexo A. Glosario

App Router: Sistema de enrutamiento en el *framework* Next.js basado en carpetas y componentes, mejora el renderizado del servidor y la estructura de la aplicación.

Broadcast: Mecanismo de comunicación en red donde un evento o mensaje se transmite simultáneamente a todos los clientes suscritos a un mismo canal.

Client-Only Render: Técnica de renderizado en React que fuerza a un componente a montarse y dibujarse solo en el navegador del usuario, evitando así errores de desajuste de hidratación (SSR mismatch).

Endpoint: URL de un servidor que responde a las peticiones de los clientes, actúa como punto de acceso a un servicio web o a una API REST.

FEN (Forsyth-Edwards Notation): Formato de texto estándar compuesto por una sola línea, usado para describir de forma precisa una posición estática exacta (sin historial de previos movimientos) en un tablero de ajedrez.

Lifting State Up (Elevación de estado): Patrón de diseño en React que consiste en trasladar el estado local de varios componentes hijos a su componente padre o contenedor común, con el objetivo de centralizar la lógica y mantener la aplicación sincronizada.

LiveKit: Infraestructura tecnológica y plataforma de código abierto para la transmisión de audio y vídeo en tiempo real.

Middleware: Capa de software que está entre el servidor y las aplicaciones, interceptando las peticiones HTTP para realizar tareas intermedias como la validación de tokens de autenticación o la configuración de CORS.

PGN (Portable Game Notation): Formato de texto estructurado y estandarizado que registra partidas de ajedrez completas, incluyendo metadatos descriptivos, el resultado y la secuencia de movimientos, con soporte para variantes anidadas.

SAN (Standard Algebraic Notation): Sistema formal e internacional utilizado para registrar los movimientos de las piezas de ajedrez de forma legible (por ejemplo, "Nf3" o "e4").



Stockfish: Motor de análisis y evaluación de ajedrez de código abierto, reconocido internacionalmente por su precisión matemática.

Supabase: Plataforma de desarrollo que opera como infraestructura de *backend* (BaaS). Proporciona bases de datos PostgreSQL, sistemas de autenticación, almacenamiento en la nube y sincronización mediante WebSockets.

Tree (Árbol de variantes): Estructura de datos no lineal utilizada para almacenar y gestionar el historial de una partida, donde cada nodo representa una posición y sus hijos las distintas continuaciones de jugadas posibles (línea principal y variantes).

UCI (Universal Chess Interface): Protocolo de comunicación que estandariza la interacción entre las interfaces gráficas de usuario (GUI) y los motores de ajedrez como Stockfish.

Web Worker: Funcionalidad de los navegadores que permite ejecutar procesos de código en segundo plano mediante hilos paralelos, evitando que cálculos pesados detengan el hilo principal de la interfaz gráfica de usuario.

Anexo B. Pseudocódigo de Proceso de construcción del árbol

Eliminar cabeceras, comentarios y anotaciones

Para cada token en el PGN:

si el token es "(":

guardar estado actual en la pila

retroceder al nodo padre para iniciar variante

si el token es ")":

restaurar estado desde la pila

si el token es un movimiento:

aplicar movimiento sobre el juego actual

si ya existe un hijo con ese FEN:

navegar a ese hijo existente

si no:

crear nuevo nodo y añadirlo como hijo del nodo actual

Anexo C. Pseudocódigo de Aplicar movimiento en el tablero

Aplicar movimiento sobre la posición actual con chess.js

si el movimiento no es legal:

devolver error

calcular FEN resultante

si existe un hijo del nodo actual con ese mismo FEN:

navegar a ese hijo existente

devolver éxito sin modificar el árbol

crear nuevo nodo con los datos del movimiento

añadir nuevo nodo como hijo del nodo actual

navegar al nuevo nodo

devolver éxito

Anexo D. Pseudocódigo de Serialización del árbol a PGN:

```
escribir movimiento del nodo en notación SAN
para cada variante en los hijos del nodo desde el índice 1:
    escribir "("
    escribir serializarDesdeNodo(variante)
    escribir ")"
    si el nodo tiene hijo principal:
        escribir serializarDesdeNodo(hijo principal)
```

Anexo E. Pseudocódigo de Generación de filas para la planilla

```
si el nodo es de color blanco:
    emparejar con su respuesta negra si existe
añadir fila con número de jugada, movimiento blanco y movimiento negro
para cada variante del movimiento negro:
    añadir cabecera de variante con profundidad + 1
    añadir filas generarFilas(variante, profundidad + 1)
si el nodo es de color negro:
    añadir fila con casilla blanca vacía y movimiento negro
continuar con el siguiente nodo de la línea principal
```

Anexo F. Encuesta de satisfacción

¿Cuál es tu relación con el ajedrez?

- Profesor o entrenador de ajedrez
- Alumno o practicante habitual
- Jugador ocasional
- Otro

¿Has utilizado previamente alguna plataforma de ajedrez online como Lichess o Chess.com?

- Sí
- No

Bloque 1 - Valoración general

Valora las siguientes afirmaciones del 1 al 5, donde 1 significa "muy en desacuerdo" y 5 "muy de acuerdo".

1. ChessClassroom me parece una herramienta útil para la enseñanza del ajedrez.
2. Utilizaría o recomendaría ChessClassroom a otros profesores o alumnos de ajedrez.
3. ChessClassroom ofrece funcionalidades que no encuentro en otras plataformas del mercado.

Bloque 2 - Funcionalidades

Valora las siguientes afirmaciones del 1 al 5, donde 1 significa "muy en desacuerdo" y 5 "muy de acuerdo".

1. El aula virtual con tablero sincronizado y sala de voz es adecuada para dar clases de ajedrez online.
2. El explorador de archivos para organizar y visualizar partidas en formato PGN es cómodo y fácil de usar.
3. El sistema de partidas y torneos Arena es adecuado para organizar competiciones dentro de una clase.

Bloque 3 - Experiencia de usuario

Valora las siguientes afirmaciones del 1 al 5, donde 1 significa "muy en desacuerdo" y 5 "muy de acuerdo".

1. El diseño visual de la plataforma me parece atractivo y cuidado.
2. La interfaz es intuitiva y fácil de entender sin necesidad de formación previa.
3. La plataforma responde con rapidez a las acciones del usuario.

Bloque 4 - Comentarios

¿Qué es lo que más te ha gustado de la plataforma? ¿Qué mejorarías o añadirías? ¿Algún comentario adicional?

Anexo G. Endpoints de la API REST

POST	/api/auth/registro
POST	/api/auth/login

GET	/api/chats/
GET	/api/chats/{salaId}/mensajes
POST	/api/chats/{salaId}/mensajes

POST	/api/clases/crear
PATCH	/api/clases/{id}/estado
GET	/api/clases/invite/{codigo}
POST	/api/clases/join
DELETE	/api/clases/{id}
GET	/api/clases/{id}
GET	/api/clases/profesor/{profesorId}
GET	/api/clases/alumno/{alumnoId}
GET	/api/clases/{claseId}/alumnos
PATCH	/api/clases/{claseId}/alumnos/{alumnoId}/alias
DELETE	/api/clases/{claseId}/alumnos/{alumnoId}
GET	/api/clases/{claseId}/miembros

GET	/api/recursos/carpetas/{id}
PATCH	/api/recursos/carpetas/{id}
DELETE	/api/recursos/carpetas/{id}
GET	/api/recursos/archivos/carpetas/{carpeta_id}
DELETE	/api/recursos/archivos/{archivo_id}/partidas
PATCH	/api/recursos/archivos/{archivo_id}/partida/{partida_index}/visibilidad
DELETE	/api/recursos/archivos/{archivo_id}/partida/{partida_index}
GET	/api/recursos/archivos/{id}
PATCH	/api/recursos/archivos/{id}
DELETE	/api/recursos/archivos/{id}
POST	/api/recursos/upload-pgn
GET	/api/recursos/descargar/{id}

Figura 29: Lista de endpoints parte 1

GET	/api/aula/clase/{claseId}
GET	/api/aula/{aulaId}
GET	/api/aula/{aulaId}/permisos
PATCH	/api/aula/{aulaId}/tablero
PATCH	/api/aula/{aulaId}/orientacion
PATCH	/api/aula/{aulaId}/cerrar
PUT	/api/aula/{aulaId}/permisos/{alumnoId}
DELETE	/api/aula/{aulaId}/permisos/{alumnoId}
GET	/api/livekit/token/{aulaId}
PATCH	/api/livekit/{aulaId}/mutear/{participanteId}
DELETE	/api/livekit/{aulaId}/expulsar/{participanteId}

PATCH	/api/ejercicios/respuestas/{respuesta_id}/evaluar
GET	/api/ejercicios/archivo/{archivo_id}
DELETE	/api/ejercicios/bloque
POST	/api/ejercicios/
GET	/api/ejercicios/entregas-calendario/{claseId}
GET	/api/ejercicios/{id}
DELETE	/api/ejercicios/{id}
PATCH	/api/ejercicios/{id}/solucion
PATCH	/api/ejercicios/{id}/fechas
PATCH	/api/ejercicios/{id}/asignar
GET	/api/ejercicios/{id}/respuestas
PATCH	/api/ejercicios/{id}/visibilidad
POST	/api/ejercicios/{id}/iniciar
GET	/api/ejercicios/{id}/mi-progreso
PATCH	/api/ejercicios/{id}/movimiento
PATCH	/api/ejercicios/{id}/comentario-alumno
PATCH	/api/ejercicios/{id}/tiempo

GET	/api/partidas/clase/{claseId}
POST	/api/partidas/
GET	/api/partidas/{partidaId}
DELETE	/api/partidas/{partidaId}
POST	/api/partidas/{partidaId}/unirse
POST	/api/partidas/{partidaId}/iniciar
POST	/api/partidas/{partidaId}/movimiento
POST	/api/partidas/{partidaId}/tablas
POST	/api/partidas/{partidaId}/abandonar
POST	/api/partidas/{partidaId}/guardar-estudio
GET	/api/partidas/{partidaId}/chat
POST	/api/partidas/{partidaId}/chat

GET	/api/torneos/clase/{claseId}
POST	/api/torneos/
GET	/api/torneos/{torneoId}
PATCH	/api/torneos/{torneoId}
DELETE	/api/torneos/{torneoId}
POST	/api/torneos/{torneoId}/iniciar
POST	/api/torneos/{torneoId}/ping
POST	/api/torneos/{torneoId}/participantes
DELETE	/api/torneos/{torneoId}/participantes/{usuarioId}
GET	/api/torneos/{torneoId}/partidas
GET	/api/torneos/{torneoId}/chat
POST	/api/torneos/{torneoId}/chat

Figura 30: Lista de endpoints parte 2

Anexo H. Imágenes de la aplicación



Figura 31: Pantalla de inicio

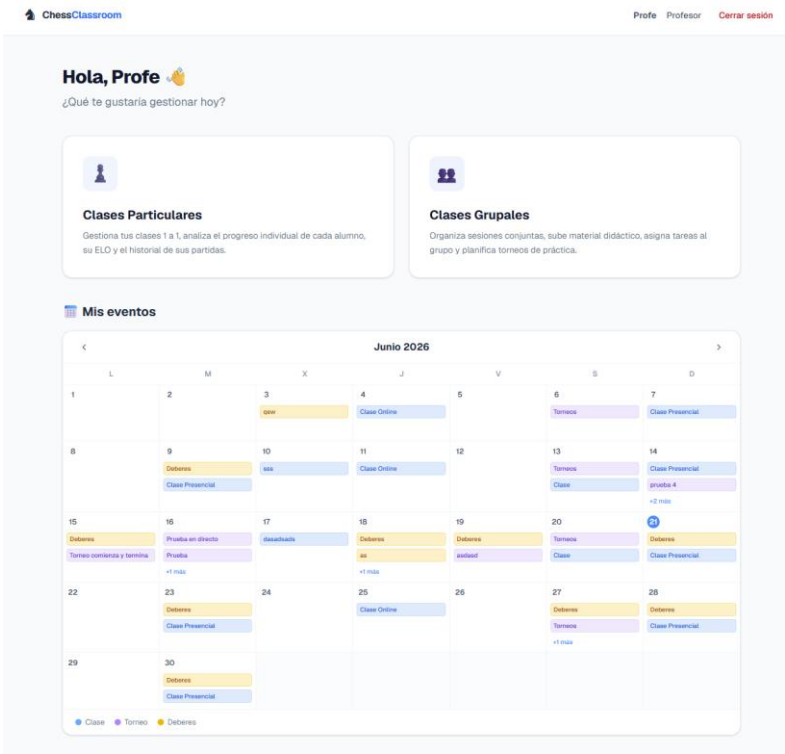


Figura 32: Selección de clases

 **Listado de clases grupales**

grupal1

Código: 206149fe [Copiar Enlace](#)

Inactivo 

chat1

Código: c6a94518 [Copiar Enlace](#)

Activo 

+ Crear clase

Nombre del grupo (Ej: Grupo Avanzados - Tácticas)

Descripción (opcional)

Cancelar

Crear clase

Figura 33: Listado de clases y creación de clase

 **ChessClassroom**



TE HAN INVITADO A UNA CLASE PARTICULAR

Grupo Avanzado Sub-16

Ya tengo cuenta (Iniciar sesión)

Crear cuenta para unirte

Figura 34: Invitación para unirse a clase

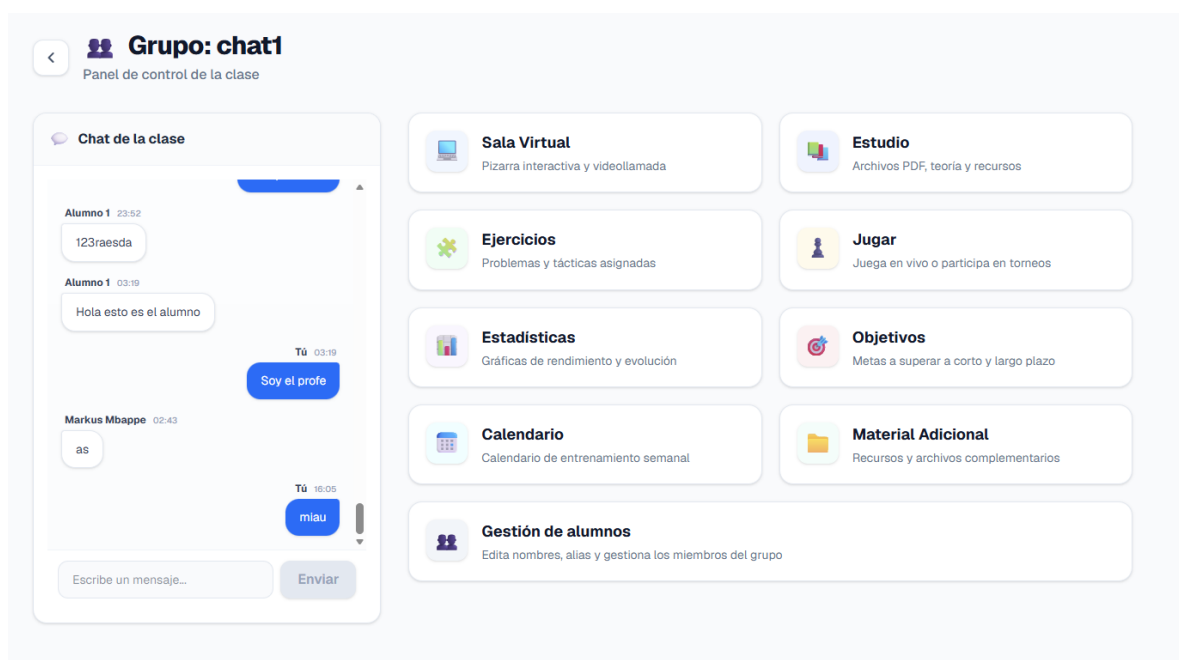


Figura 35: Menú de clase

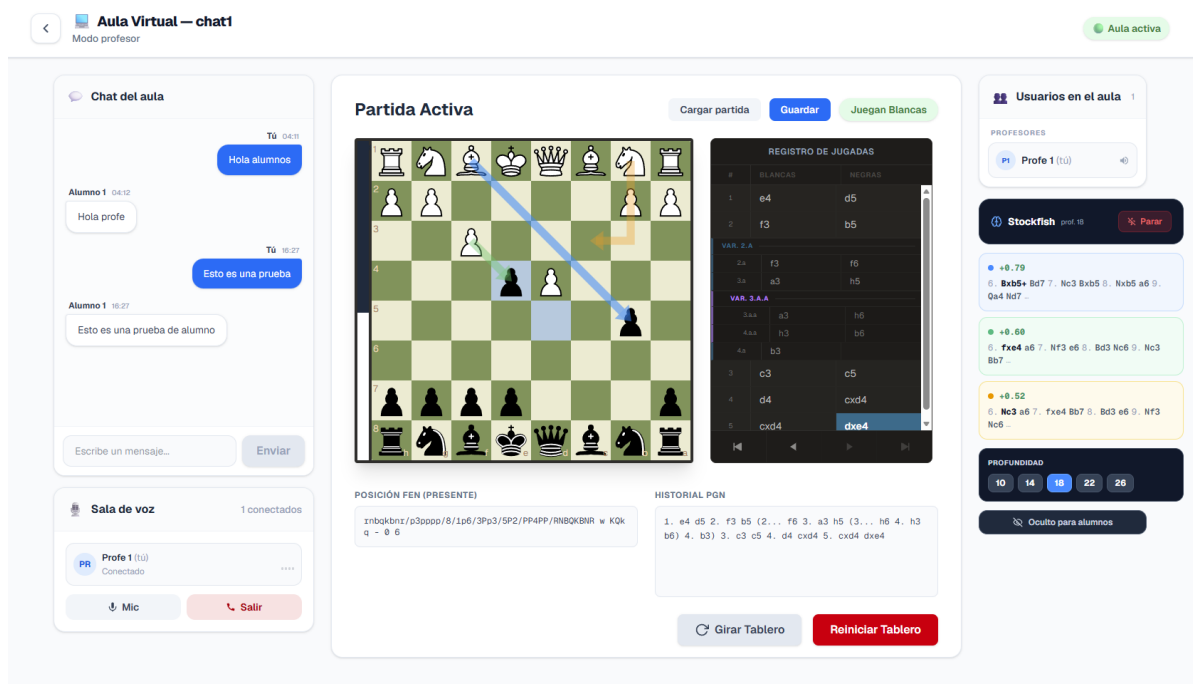


Figura 36: Aula virtual

Cargar partida ✕

OPCIÓN A — ARCHIVO PGN / FEN

Haz clic para seleccionar un archivo .pgn
Solo partidas individuales

OPCIÓN B — PEGAR FEN O PGN

FEN: rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3
0 1

o PGN: 1. e4 e5 2. Nf3 ...

Solo partidas individuales — para databases usa el explorador

OPCIÓN C — DESDE MIS ARCHIVOS

Estudios Ejercicios

Cancelar Cargar partida

Figura 37: Formulario de carga de posiciones en el tablero del aula

← **Guardar en Estudios** ✕

Raíz

+ Nueva carpeta

Aperturas >

Estudios 9 >

as >

Carpeta 8 >

Carpeta 7 >

Carpeta 6 >

Carpeta 5 >

Carpeta 3 >

Carpeta 2 >

Cancelar Guardar aquí

Figura 38: Formulario de guardado de la posición del aula

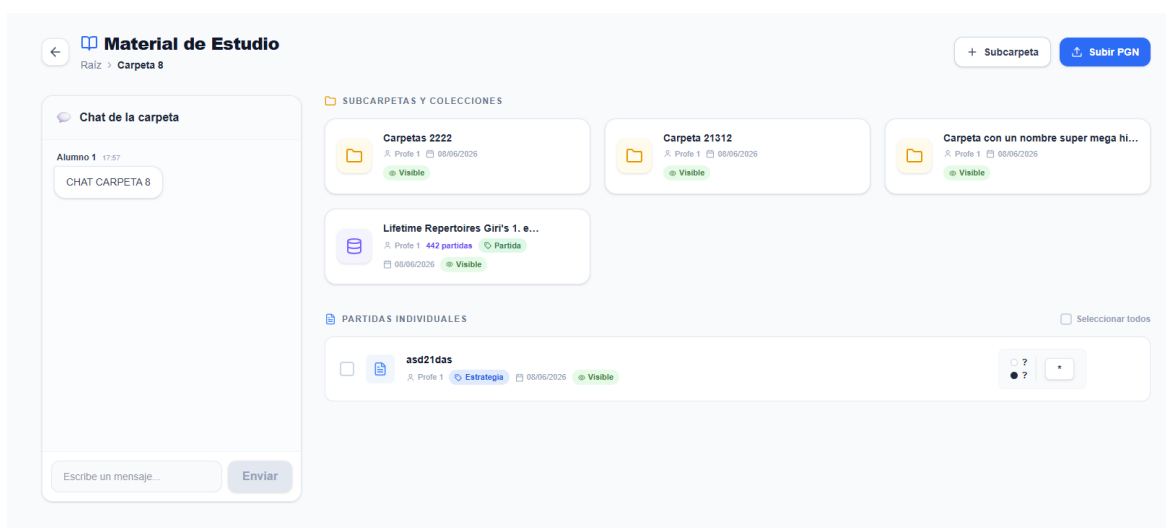


Figura 39: Explorador de archivos

Subir ejercicio

OPCIÓN A — ARCHIVO(S) PGN / FEN

Haz clic para seleccionar uno o varios archivos .pgn

Cada archivo se sube como un ejercicio independiente

OPCIÓN B — PEGAR FEN O PGN COMO TEXTO

FEN: rnbqkbnz/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1

o PGN: 1. e4 e5 2. Nf3 ...

NOMBRE (opcional — si se deja vacío se usa el nombre del archivo)

Ej: Táctica del alfil

FECHA DE INICIO (opcional)

FECHA DE ENTREGA (opcional)

mm/dd/yyyy

mm/dd/yyyy

CATEGORÍA *

Apertura

Táctica

Estrategia

Final

Partida

Cálculo

Cancelar

Subir ejercicio

Figura 40: Formulario de subida de ejercicios

Ejercicio

TIEMPO

00:01:32

★ Evaluado

ERRORES

6

ENTREGA

21 jun 2026

TU COMENTARIO PARA EL PROFESOR

afdafasfasfasfas/safa

24/350

Guardar comentario

REVISIÓN DEL PROFESOR

PUNTUACIÓN

1

2

3

4

5

5/5

COMENTARIO

Muy bien hecho!

1

2

3

4

5

6

7

8

a

b

c

d

e

f

g

h

REGISTRO DE JUGADAS

#	BLANCAS	NEGRAS
1	e4	d6
2	f3	c5
3	Qe2	d5
4	exd5	Qxd5

Girar Tablero

Figura 41: Ejercicio evaluado

Nueva partida

♔ BLANCAS

Sin asignar

♚ NEGRAS

Sin asignar

⌚ TIEMPO (min, 1–90)

5

⏰ INCREMENTO (seg, 0–30)

0

📄 POSICIÓN INICIAL (opcional — FEN o PGN)

FEN: rnbqkbnr/... o PGN: 1. e4 e5 ...

Cancelar

Crear partida

Figura 42: Formulario de creación de partida

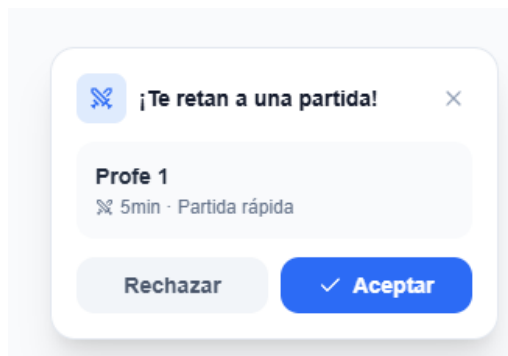


Figura 43: Widget de invitación a partida amistosa



Figura 44: Partida

Nuevo torneo

NOMBRE *
Ej: Torneo de diciembre

FECHA INICIO * **FECHA FIN ***
mm/dd/yyyy mm/dd/yyyy

TIEMPO (min, 1-90) INCREMENTO (seg, 0-30)
5 0

PARTICIPANTES (0 seleccionados) **Añadir toda la clase**

☐ Profe 1 Profesor
☐ Alumno 1
☐ Alumno 2

POSICIÓN INICIAL PARA TODAS LAS PARTIDAS (opcional)
FEN: rnbqkbnz/... o PGN: 1. e4 e5 ...

Cancelar **Crear torneo**

Figura 45: Formulario de creación de torneo

torneo 5

3 participantes
5 min
Inicio: 14 jun 2026, 19:35
Fin: 20 jun 2026, 19:35

Finalizado

Chat del torneo

Tu 17:36

Hola alumnos

Alumno 1 17:38

Hola profe

Escribe un mensaje...

Enviar

Activas Historial

Todos los participantes

Profe 1

Alumno 1

5 min

Tu partida

Abortada

Profe 1

Alumno 2

5 min

Tu partida

Abortada

Alumno 2

Alumno 1

5 min

Finalizada

Empate - tablas

Alumno 2

Alumno 1

5 min

Abortada

Alumno 1

Profe 1

5 min

Finalizada

Gana Alumno 1 - abandono

Profe 1

Alumno 1

5 min

Tu partida

Abortada

Alumno 1

Alumno 2

5 min

Finalizada

Gana Alumno 1 - abandono

Profe 1

Alumno 1

5 min

Tu partida

Abortada

Alumno 2

Profe 1

5 min

Tu partida

Abortada

Clasificación

#	JUGADOR	PTS	PART
1	Alumno 1 LIDER	103	103
2	Profe 1 (tu)	87	88
3	Alumno 2	80	79

Figura 46: Torneo

trofeo

¡Has sido emparejado!

Torneo arena - redirección automática

Nueva partida lista

Juegas con Negras

Redirigiendo en...

10s

Ir a la partida ahora

Figura 47: Widget de notificación de emparejamiento de torneo

Anexo I. Relación con los objetivos de desarrollo sostenible

Tabla 9: Relación con los ODS

Objetivos de Desarrollo Sostenibles	Alto	Medio	Bajo	No Procede
ODS 1. Fin de la pobreza.				X
ODS 2. Hambre cero.				X
ODS 3. Salud y bienestar.				X
ODS 4. Educación de calidad.	X			
ODS 5. Igualdad de género.				X
ODS 6. Agua limpia y saneamiento.				X
ODS 7. Energía asequible y no contaminante.				X
ODS 8. Trabajo decente y crecimiento económico.			X	
ODS 9. Industria, innovación e infraestructuras.		X		
ODS 10. Reducción de las desigualdades.		X		
ODS 11. Ciudades y comunidades sostenibles.				X
ODS 12. Producción y consumo responsables.				X
ODS 13. Acción por el clima.				X
ODS 14. Vida submarina.				X
ODS 15. Vida de ecosistemas terrestres.				X
ODS 16. Paz, justicia e instituciones sólidas.				X
ODS 17. Alianzas para lograr objetivos.		X		

Los Objetivos de Desarrollo Sostenible, establecidos por las Naciones Unidas en la Agenda 2030, forman un marco de referencia global para orientar el desarrollo de proyectos hacia un futuro más justo y sostenible. En la Tabla 9 se ha indicado cierto grado de relación con alguno de los objetos. A continuación, se explica los ODS que ChessClassroom cumple.

ODS 4 - Educación de calidad (Alto)

La relación más directa de ChessClassroom es con el ODS4, que cuyo objetivo es ofrecer una educación inclusiva, igualitaria y de calidad.

ChessClassroom surge a partir de la identificación de una carencia en el entorno educativo del ajedrez, la ausencia de herramientas digitales diseñadas para el docente. El proyecto cubre esta necesidad, y en el proceso mejora la calidad de la enseñanza a varios niveles.

Facilita la personalización del aprendizaje mediante el seguimiento del progreso de cada alumno e iguala el acceso a una educación estructurada del ajedrez al permitir dar clases online de calidad a alumnos de todo el mundo.

La corrección automática de ejercicios, el sistema de estadísticas y el aula virtual con tablero sincronizado contribuyen directamente a mejorar la experiencia educativa, reduciendo la carga del profesor y permitiéndole dedicar más tiempo a la enseñanza.

ODS 9 - Industria, innovación e infraestructuras (Medio)

El ODS 9 promueve la innovación y el desarrollo de infraestructuras. ChessClassroom se relaciona con este objetivo al ser una solución innovadora para un sector que hasta ahora carecía de herramientas digitales especializadas.

El proyecto integra tecnologías modernas como WebSockets para la sincronización en tiempo real, WebRTC para la comunicación de audio y WebAssembly para la ejecución del motor de análisis en el navegador, aplicándolas en un entorno educativo concreto.

La arquitectura desacoplada y escalable del sistema crea una base que puede crecer y adaptarse a las necesidades de academias y federaciones más grandes.

Todos estos aspectos contribuyen a la digitalización de la infraestructura educativa del sector.

ODS 10 - Reducción de las desigualdades (Medio)

ChessClassroom cumple el ODS 10 al eliminar barreras geográficas en el acceso a la enseñanza del ajedrez.

Tradicionalmente, para recibir clases de calidad se necesita estar cerca de un club o academia. Un alumno en una zona rural o con poca costumbre de jugar ajedrez tenía un acceso muy limitado a formación. El proyecto rompe esta barrera al permitir que un profesor pueda dar clases online con las mismas herramientas interactivas que tendría en una clase presencial.

Al tratarse de una aplicación web accesible desde cualquier dispositivo con navegador, tampoco obliga al usuario tener hardware especializado, lo que reduce la barrera tecnológica de acceso.

ODS 17 - Alianzas para lograr objetivos (Medio)

El ODS 17 promueve la colaboración entre personas para lograr objetivos comunes. ChessClassroom se relaciona con este objetivo desde dos puntos de vista.

En primer lugar, el proyecto ha sido el resultado de una colaboración entre dos estudiantes con perfiles complementarios, donde la combinación de conocimientos técnicos y de dominio del ajedrez ha sido indispensable para desarrollar el proyecto.

En segundo lugar, la propia plataforma forma alianzas en el entorno educativo, permitiendo a profesores conectar con alumnos y organizar actividades que de otro modo serían más complicadas.

ODS 8 - Trabajo decente y crecimiento económico (Bajo)

La relación con el ODS 8 es indirecta. La plataforma ofrece a los profesores de ajedrez una herramienta que les permite escalar su actividad docente, ampliando su base de alumnos a través de clases online. Esto puede contribuir a mejorar la su situación económica, especialmente para profesores independientes o de academias menos importantes.

En conclusión, ChessClassroom es un proyecto con un objetivo claramente educativo cuyo impacto principal se relaciona con el ODS 4. Su desarrollo como una solución tecnológica innovadora lo conecta con el ODS 9, su potencial para reducir barreras de acceso con el ODS 10, sus características colaborativas con el ODS 17 y su apoyo a la actividad docente con el ODS 8. Por todos estos razonamientos, se puede concluir que este trabajo de fin de grado está alineado con los valores de sostenibilidad, inclusión e innovación que promueve las Naciones Unidas mediante la Agenda 2030.